

CCA-Secure Traceable Threshold (ID-based) Encryption and Application

Rishiraj Bhattacharyya
University of Birmingham, UK
rishiraj.bhattacharyya@gmail.com

Jan Bormet
TU Darmstadt, Germany
jan.bormet@tu-darmstadt.de

Sebastian Faust
TU Darmstadt, Germany
sebastian.faust@tu-darmstadt.de

Pratyay Mukherjee
Supra Research, India
pratyay85@gmail.com

Hussien Othman
TU Darmstadt, Germany
hussien.othman@gmail.com

ABSTRACT

A recent work by Boneh, Partap, and Rotem [Crypto’24] introduced the concept of traceable threshold encryption, in that if t or more parties collude to construct a decryption box, which performs decryptions, then at least one party’s identity can be traced by making a few black-box queries to the box. This has important applications, e.g., in blockchain mempool privacy, where collusion yields high financial gain through MEVs without any consequence – the possibility of tracing discourages collusion. Nevertheless, their definitions leave room for exploitation as they only achieve CPA security and do not consider inconsistency in decryption via different participating sets.

This paper proposes stronger definitions of traceable threshold encryption, which supports CCA-security and consistency. Our main approach considers identity-based variants of traceable encryption (which we also define). It converts that to a CCA-secure construction, adapting two generic transformations, first using a one-time signature and then a fingerprinting code. We put forward two efficient instantiations of our identity-based scheme with different merits: our first construction is based on Boneh-Franklin IBE [Crypto’01] and has constant size ciphertexts but quadratic size public keys – this is proven secure based on XDH and BDDH. Our second construction is based on Boneh-Boyen IBE [Eurocrypt’04]. It supports both constant-size ciphertexts and constant-size public keys – this is proven secure based on a variant of the uber assumption over bilinear pairings. Our concrete analysis shows that the first construction’s ciphertext is much ($\sim 6x$) smaller than the second construction. Finally, we extend the definitions to support consistency and achieve it by adjoining an efficient, non-interactive proof of correct encryption.

1 INTRODUCTION

Threshold cryptographic divides a cryptographic procedure into multiple subprotocols, distributing them among several (say n) participants. This ensures that no single participant holds the entire secret, avoiding a single point of failure. Only a subset of participants, meeting a predefined “threshold” (say, $t \leq n$) can collaboratively perform the cryptographic operation or reconstruct the secret. Threshold cryptography traditionally guarantees security when at most $(t - 1)$ parties are corrupt (and possibly colluding). Specifically, if more than t parties are corrupted by the adversary, then all bets are off, as the adversary gets to know the entire secret! However, in many practical applications of threshold cryptography if t or more parties collude, then the colluders can exploit the

system for significant financial gain. Without a mechanism for discouraging or even detecting such behavior, this possibility may turn out to be rationally viable as well.

For example, let us consider the problem of mempool privacy. In many blockchain ecosystems, such as Ethereum, front-running attacks, and variants [34, 54, 55], on mempools¹ are commonplace. Here, the validators of a blockchain leverage their advantage of observing transactions before they are finalized. They selectively finalize specific blocks or put transactions from the mempool in a specific order and, depending on that, may extract substantial financial gains (aka MEVs [30]). Existing solutions [2, 19, 25, 26, 50] broadly incorporate threshold decryption techniques, in which transactions in the mempool are encrypted, and the decryption keys are distributed among the validators in a t out of n threshold access structure. Transactions are decrypted only when at least t validators agree to decrypt, which takes place only after a certain time when no further MEVs can be derived. However, in practice, without additional mechanisms, there is no way to enforce $< t$ corruption. In particular, if $\geq t$ validators continue to extract MEVs by just decrypting transactions earlier than scheduled, no one can even detect that. In fact, it is clearly a rational choice for the validators to derive MEVs via collusion.

Towards mitigating this kind of problem in the context of threshold encryption, the work of Boneh, Partap, and Rotem [15] (henceforth, BPR24) recently put forward the notion of *traceable threshold encryption*, which allows a designated party, called the *tracer* (who may possess a secret tracing key) to catch at least one traitor from the collusion of at least t parties in the context of threshold decryption. More precisely, if the colluding group (of $\geq t$ parties) constructs a decryption box that, on a given ciphertext, returns the corresponding decrypted value, then with a few queries to the box, the tracer can output the identity of at least one of the colluders. Therefore, with an adequate penalty in place (for example, via slashing), validators are much less likely to collude, as the financial benefit of colluding over not colluding is not clear anymore. In the same paper [15], the authors proposed two traceable threshold encryption schemes with different merits, both based on finger-printing codes [18].

While the constructions in BPR24 [15] are quite efficient, their security notions only provide weaker security guarantee – they achieve only CPA-security, while CCA-security is considered the “gold standard” for encryptions [5]. In this work, we address this

¹Roughly, validators locally maintain a mempool that contains all transactions sent to a blockchain that have not been finalized in a block.

natural problem by extending the notion of traceable threshold encryption to CCA-security. Moreover, from a practical perspective in the context of mempool privacy, two concrete practical attacks are not captured by BPR24: (i) Since their definition only considers CPA security, a non-malleability attack is easy to execute, for example, by blindly mauling an encrypted transaction to something related. This related transaction may be decrypted earlier, thereby violating transaction privacy. We stress that such an attack also works when the plaintexts are signed, which is usually the case for mempool data. This is because, even if blind mauling makes the signature invalid, the plaintext would still get exposed. Hence, a malicious validator can frontrun the original transaction in the next block. Instead, CCA security would guarantee that no information is leaked when the ciphertext is mauled. (ii) The notion does not guarantee a *consistency* of decryption, which ensures that the decryption results into the same plaintext as long as *any* set of t decryptors participate, regardless of the specific members of the set – this can be exploited, for example, by adversarially crafting a ciphertext which results into different decryptions based on the set of decryptors. In fact, these attacks are not only outside their definitions but they can be easily executed on their constructions, simply exploiting the homomorphic structures (see Appendix A for more details). This begets not only an extension of their notions but also new, stronger constructions secure therein.

1.1 Our Contribution

In this paper, we primarily focus on addressing these issues comprehensively first by extending BPR24’s notion to *CCA-security* and then adding *consistency*; finally, we propose two new efficient constructions satisfying our notions. In particular, our contributions are as follows:

- **New Stronger Definitions.** We formalize the notion of *CCA-security* in the setting of traceable threshold key encapsulation mechanisms (TT-KEM).² En route, we also define the notion of identity-based traceable KEM, which supports a threshold key-derivation – we use this as a building block towards CCA security. Finally, we augment our CCA-security definitions to support the crucial *consistency* property. Our new definitions capture the above.
- **Constructing Stronger TT-KEM Schemes.** We construct CCA-secure TT-KEM schemes generically from a CCA-secure bipartite threshold KEM scheme (BT-KEM) – a CPA-secure version of this primitive was defined in BPR24, we augment that to CCA-security. Our generic transformation is an adaptation from BPR24’s techniques in the context of CPA-security and also relies on the fingerprinting codes [17]. However, as we explain later in Section 1.3, the CPA-secure construction from BPR24 is not compatible with the existing CPA-to-CCA transformation. So we resort to a new approach via IBE. We construct CCA-secure BT-KEM *generically* from any identity-based BT-KEM (i.e., BTIB-KEM, which we define here adapting the PKE definition from BPR24) lifting the one-time-signature based

technique due to [11]. Finally, we instantiate BTIB-KEM with two concrete constructions:

- (i) Our first construction is based on the Boneh-Franklin IBE [12] and relies on XDH and BDDH assumptions on bilinear pairing groups. Instantiating with this construction, we obtain a CCA-TT-KEM, which has a constant size ciphertext but a larger public key, which scales quadratically with the number of parties.
 - (ii) Our second construction is based on the Boneh-Boyen IBE [7]. Instantiating with this, we obtain a TT-KEM scheme, where both ciphertext and public key are of constant size. However, the ciphertext is concretely larger than the first construction. The security proof becomes much more involved and relies on a variant of Uber assumptions [8, 20], which hold in generic groups. This construction additionally supports hierarchical IBE, and using a generic transformation using fingerprinting codes, similar to the aforementioned BT-KEM to TT-KEM transformation, we can obtain a CCA-secure traceable threshold IBE (TTIB-KEM) scheme – this may be of independent interest.³
- **Achieving Consistency.** Finally, we extend our CCA-secure TT-KEM constructions to achieve the consistency property by using an efficient non-interactive zero-knowledge (NIZK) proof of the well-formedness of the ciphertext. However, the challenging part is to resolve the apparently contradictory requirements of public verifiability of ciphertext well-formedness and traceability, which relies on the colluding party’s inability to detect a malformed ciphertext. We resolve this by including a simulation trapdoor (for NIZK) into the tracing key which enables only the tracer to produce “false proofs”. We also note that a similar strategy cannot be incorporated to achieve CCA-security, because that should hold also against a malicious tracer. We discuss this in more detail in the next section (Section 1.3).

Our work has significant implications for both theory and practice through defining, designing and analyzing CCA-security for traceable threshold encryption. On the theoretical side, we introduce the first formal security notion for CCA-security in the context of traceable threshold encryption and present a novel construction, accompanied by rigorous proofs, that realizes this notion. In doing so, we address a key open problem left unresolved in prior work [15]. From a practical standpoint, CCA-security is natural and essential for most real-world applications of threshold encryption, including encrypted mempools, secure voting, secure auctions etc. Moreover, the core assumption in threshold cryptography, that at most $t - 1$ parties are corrupted, can be violated in practice, especially in settings where parties may be financially motivated to collude, as elaborated above. By providing a practically efficient construction for CCA-secure traceable threshold encryption,

²Similar to BPR24 [15] we work with the notion of KEM for cleaner constructions and presentations, all our constructions can be converted to usual encryption schemes following the standard notion of hybrid encryption (KEM-DEM paradigm).

³We stress that, for the transformation from BTIB-KEM to CCA-BT-KEM only CPA security of the former suffices.

our paper has taken an important step in addressing these practical concerns.

1.2 The Traceability Model in Practice

We follow [15] and consider a stateless decoder (alternatively, decryptor), that does not retain any state between queries and can be reset and re-executed at will. In this model, the colluders jointly construct such a decoder that returns the decryption of a given ciphertext. To perform the tracing procedure, the tracer (who holds a tracing key) needs black-box access to this decoder; the tracer is allowed to submit any (potentially malformed) ciphertexts to this decoder and observes its responses. From the responses, the tracer identifies at least one of the colluders. Note that the privacy requirement holds against the tracer, despite the knowledge of tracing key.

As elaborated in [15], this modeling captures the scenario, in that the colluding adversaries build this decoder box and sell it on the dark market. When this decoder box is seized, the tracer gets involved and starts the tracing procedure by running this box at their disposal. In fact, even without guaranteed access to the decoder, the mere possibility of efficient tracing based on leaked decoders serves as an anti-collusion mechanism. This is because colluders cannot be certain that one of them will not leak the decoder or that the buyer (of the decoder box) is not collaborating with the tracer.

We note that our model excludes attacks in which the colluders provide a decryption service, i.e., a scenario where the adversary submits ciphertexts to the colluders for decryption, due to its possible statefulness.⁴ In such a model, the colluders could jointly execute, for example, a secure multi-party computation (MPC) protocol and return the decryption result to the adversary. However, as discussed in [15], this decryption-as-a-service approach is often less desirable. For example, in the encrypted mempool application, an adversarial buyer (of the service or decryption box) may intend to learn the decryption ahead of everyone else, including the colluding validators. But, with decryption-as-a-service approach, the colluding validators may be able to access the decryption result before the buyer. Additionally, it requires some form of interaction between the colluders and the buyer for every decryption, making it both more likely to be discovered and less efficient than selling a single decoder. Nevertheless, formally modeling and constructing traceability guarantees against such stateful and interactive decryption services remains an important open problem in traceability.

1.3 Technical Overview

In this section, we provide an overview of our techniques. We start with the setting, which is the same as BPR24.

The Setting. We consider an arbitrary threshold access structure for decryption in that any t out of n parties together can decrypt any ciphertext and reconstruct the whole decryption key. Among them, any set of parties (possibly $\geq t$) can collude to construct

a decryption/decoder box, called D .⁵ BPR24 allows for decoders that can be used to learn *any* non-trivial information about the encrypted message (not even the full message). This is formally modeled through the distinguishing advantage. To allow for traceability, the threshold encryption scheme is equipped with a tracing algorithm, which would find out the identity of at least one colluder while making a limited number of black-box queries to D , possibly using a private tracing key. However, note that as long as the corruption is within the threshold t , the privacy (CCA-security in our case) should hold even against the tracer, who owns the tracing key. In the CCA-security game, the adversary is given the tracing key to model this. Nonetheless, once t or more parties collude to construct a decryption box, no privacy can be guaranteed. Instead, in that case, the tracer is responsible for finding a colluder by using the decoder box as an oracle. In reality, this captures the scenario when the tracer (e.g., law enforcement) buys a copy of the decoder box and runs it with various inputs at their own disposal.

Overview of BPR24 [15]. Boneh and Naor [14] constructed a public-key encryption scheme with traitor-tracing using fingerprinting codes [17]. Intuitively, fingerprinting codes enable one to trace information about the original codewords from noisy ones – this helps in tracing when secret keys are encoded with such codes. The constructions of BPR24 used fingerprinting codes in a similar fashion, with adequate changes to make it work in a threshold setting. Technically, the main idea is to first construct a building block (defined in BPR24) called bipartite threshold KEM (BT-KEM) and then convert it to a TT-KEM using a generic transform. Informally, in BT-KEM, a secret key consists of 2ℓ elements for some parameter ℓ . For each index $i \in [\ell]$, there are two keys, a left key and a right key. Each ciphertext also contains a left and a right part and is associated with an index $j \in [\ell]$. The correctness is *two-sided*, in that any collection of keys from t parties possibly holding a mix of left or right keys for index j , are sufficient to decrypt any ciphertext; security is *one-sided*, in that a set (possibly of size $\geq t$) of parties, all of them holding only the left key (resp. right key) for index j cannot distinguish the corresponding right part (resp. left part) of the ciphertext from a random value. In particular, this one-side security enables the tracing functionality similar to [14].

Specifically, BPR24’s generic transformation from BT-KEM to TT-KEM works as follows: The setup phase samples a fingerprinting code of n words of size ℓ . It runs the key-generation of BT-KEM with parameter ℓ to generate 2ℓ secret keys, a pair for each index $j \in [\ell]$. Each party- i receives a unique secret key share which consists of exactly ℓ elements chosen among the 2ℓ pairs according to the i -th finger-printing codeword, such that a party obtains exactly one key (either left or right) for each index $i \in [\ell]$. To encrypt a message m first a random index $j \in [\ell]$ is chosen, and then it is encrypted with respect to the j -th index to generate the left and right ciphertext pair (c_0, c_1) . Decryption works with any subset of shares of left or right keys. Traceability is achieved using one-sided security plus the tracing mechanism of the fingerprinting codes: carefully crafted combinations of invalid and valid ciphertexts are queried to D , and based on whether invalidity is

⁴As an example, a stateful decryption service could refuse to decrypt any further ciphertexts after it realizes it is being traced by observing a request with a malformed ciphertext.

⁵This captures the practical scenario, in which the colluding parties may collaborate to produce and sell a device containing everyone’s shares. The device can be purchased, possibly in a black market, and then used by anyone to decrypt any ciphertext.

detected or not, the tracer can recover (partial) information about the codewords corresponding to the colluders, thereby narrowing down the plausible set of the colluders, and eventually a colluder's identity. The intuition is, if, e.g., all the colluders have only left shares for index j , then, by the one-sided security of BT-KEM, the decoder D cannot distinguish valid c_1 from an invalid one, and hence would return a valid decryption. So the tracer can conclude that all colluders correspond to j -th index being 0 and so on.

Failed Attempts for CCA. Now, the most natural way to try to achieve CCA-security from the state of the art (namely BPR24) is to apply existing CPA-to-CCA transformation techniques to the BPR24's constructions. However, the main challenge is to ensure that CCA-security, which guarantees non-malleability, does not interfere with traceability, which does require some sort of malleability, as explained above. The existing CPA-to-CCA approaches fail to overcome precisely this issue. To illustrate let us first consider the Fujisaki-Okamoto [35, 36] transformation: the encapsulation mechanism encrypts a random string r via a CPA-secure encryption (where the encryption randomness is deterministically derived from r) to produce the ciphertext c . Then the random message (or key) is derived by hashing r . That is, $c = \text{Encrypt}(pk, r)$ and $k = \mathcal{H}(r)$. The main argument in proving CCA security is, as the adversary never gets to decrypt the challenge ciphertext c , it is unable to find the value of r , even with the decryption of some other $c' = \text{Encrypt}(pk, r')$, and thus k appears completely random assuming the hash to be a random oracle. Let us now try to apply this to the above CPA TT-KEM: encrypt the random string r using BT-KEM to get a ciphertext pair (c_0, c_1) with respect to some index j . Any of c_0 or c_1 can be decrypted to derive the random string r and hence the correct key. Given a decryption oracle, one can query a mauled ciphertext (c_0, c_1^*) . Since the BT-KEM decryption decrypts one of the ciphertexts, it may just decrypt the left one and return r , breaking security. The crux of the problem here is, decryption in the BPR24 construction does not have a mechanism to publicly verify that the pair (c_0, c_1) is indeed a valid ciphertext.⁶

An alternative is to consider the Naor-Yung [45] paradigm (or its variants, such as the Cramer-Shoup transform [29] using hash-proof), which essentially addresses this public verifiability problem by attaching a non-interactive zero-knowledge proof of the fact that two ciphertexts encrypt the same message. Unfortunately, this approach does not work because the decoder box can just detect if an invalid ciphertext is queried, and return $\perp$. Hence, one-sided security cannot be guaranteed, and it is unclear how to make the tracing algorithm work.⁷

Our Approach to achieve CCA-security. The main technical challenge is to ensure that CCA-security, which implies non-malleability, does not interfere with traceability, which does require some sort of malleability, as explained above. We resolve this issue using a different route. In particular, we first extend the notion of BT-KEM to support identity – we call this new notion the bipartite threshold

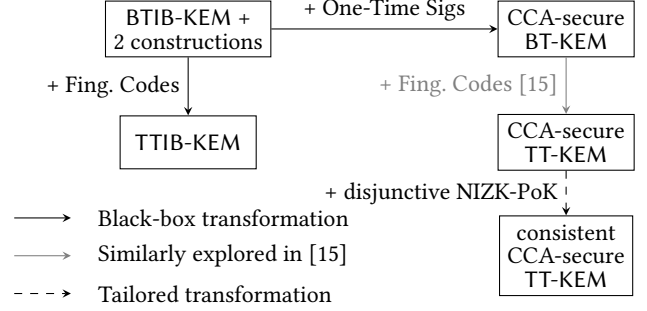


Figure 1: Overview of the techniques used in this paper.

identity-based KEM (BTIB-KEM). Then we use a variant of IBE-to-CCA generic transformation by Boneh et al. [11] to construct a CCA-secure BT-KEM. Finally, adapting the generic transformation of BPR24 to the CCA setting, we convert CCA-secure BT-KEM to CCA secure TT-KEM using fingerprinting like the above.

From BTIB-KEM to CCA-BT-KEM. Let us start by recalling the idea of Boneh et al. [11] which uses a one-time signature to generically transform an IBE to a CCA-secure PKE scheme. Let the master key pair of the IBE scheme be (mpk, msk) . Then the encryption algorithm of the PKE works as follows: sample a fresh signing-verification key pair (sk, vk) , and use vk as the identity to encrypt the message m to ciphertext c . Then sign c with sk to compute signature σ . The final ciphertext is (vk, c, σ) . Decryption works by first checking the signature, then deriving the identity key using the master key, and decrypting c . If the signature fails to verify, it outputs $\perp$. Intuitively, CCA-security follows because (i) if the adversary makes decryption queries with a ciphertext which has $\text{vk}' \neq \text{vk}$, then the ciphertext is unrelated to the challenge ciphertext; (ii) if the adversary makes a decryption query with a ciphertext which has $\text{vk}' = \text{vk}$, the ciphertext is either invalid or can serve as a forgery to the signature scheme. Now we apply the same technique to BTIB-KEM in a way such that one is able to generate a correct signature for a malformed ciphertext such that traceability is possible. To see this, we note that in our modified one-sided security game for CCA secure BT-KEM, the challenger generates two ciphertexts $c^{(0)} = (c_0^{(0)}, c_1^{(0)})$ and $c^{(1)} = (c_0^{(1)}, c_1^{(1)})$. Then, it challenges the adversary with a possibly malformed ciphertext c^* , such as a mix of $c^{(0)}$ and $c^{(1)}$ (i.e., takes one side from each ciphertext). Here, the adversary can distinguish between the cases when the ciphertext is valid and when it is malformed by verifying the signature. This is fixed by modifying the one-sided security definition, in that the two ciphertexts $c^{(0)}$ and $c^{(1)}$ are encrypted to the same identity key vk (as tags) and then sign the possibly malformed c^* using sk . Finally, we show that our modified one-sided security game does not break tracing by discussing the required changes needed to the tracing algorithm of [15].

To conclude, we argue in Section 3.3 that the bipartite-to-traceable transformation of [15] and this work both preserve CCA-security. Hence, plugging any CCA-secure BT-KEM in the black-box TT-KEM construction in [15] results in a CCA-secure TT-KEM.

⁶We note that, since the BT-KEM decryption uses one of the left or right keys, but not both, it cannot check equality after decryption.

⁷One might attempt to resolve this issue by including a simulation trapdoor into the private tracing key – this would enable the tracer to produce simulated proofs of false statements. Nonetheless, this would not work because the CCA-adversary gets access to the tracing key, thus is able to produce false proofs and break CCA security.

Efficient BTIB-KEM. Finally we give two instantiations of BTIB-KEM. We achieve our first construction based on Boneh-Franklin IBE scheme [12] – this can be thought of as an identity-based extension of BPR24. This construction enjoys a short ciphertext of only 2 group elements. However, the construction requires a quadratic-size public key. Our second construction enjoys a constant-size public key. We achieve this by using Boneh-Boyen two-level (hierarchical) IBE [7]. The construction is more involved. However, in concrete terms, this construction has larger ciphertext (12 group elements compared to 2 in the first), though it offers better asymptotic parameters. In practical settings, such as mempool privacy, typically short ciphertexts are more desired. Nevertheless, the second construction supports a hierarchical feature. Note that from a two-level hierarchical BTIB-KEM, we can achieve a CCA-secure traceable IBE using the BPR24 transformation. This may be of independent interest as identity-based encryption schemes are useful in popular mempool privacy solutions as well [50].

A summary of our techniques and results is given in Figure 1.

1.4 Applications Beyond Blockchains

While blockchain mempool privacy remains our primary motivating application, we stress that designing a mechanism for $\geq t$ collusion remains a fundamental challenge across *all* applications of threshold encryption (and in fact, threshold cryptography in general). Notable examples include secure electronic voting [28] and secure auctions [48]. For example, in voting systems threshold encryption ensures secure and confidential decryption of ballots by requiring the cooperation of multiple parties before revealing the results. Since votes are highly sensitive, a collusion of $\geq t$ would have devastating consequences. A traceability mechanism in such a system may be helpful to prevent (or disincentivize) such a scenario. Similarly, in auction systems, the bids are highly sensitive and a collusion of $\geq t$ decryptors may reap huge unfair financial benefits for the colluders. Our traceability mechanism puts some safeguards to prevent such an attack.

1.5 Related Works

To the best of our knowledge, there are no other traceable threshold encryption constructions besides [15]. Some other recent works study collusion resistance of secret sharing [16, 32, 39], but it is not clear how to extend these techniques to construct threshold cryptographic schemes.

There has been extensive research on traitor tracing in the broadcast encryption setting (see, e.g., [15, 52] for a good overview). Traitor tracing in this context is mostly motivated by the goal of catching individuals who illegally sell or replicate their decryption device for, say, pay TV broadcast. In this setting, the work of [31] is the first to consider CCA-secure traitor tracing, using the Cramer-Shoup transformation [29]. In particular, they present a k -resilient construction with ciphertext overhead of $O(k)$, where k is an upper bound on the number of colluders. This is less efficient than our construction, since we achieve a constant-size ciphertext with full-resiliency. Other works [1, 40, 46] studied traitor tracing IBE in the broadcast encryption setting. Specifically, they extend the traitor tracing in broadcast encryption to the identity-based setting, such that each user gets a decryption key for some identity group.

Hence, the decoder box that is constructed by a set of users can decrypt only for their corresponding identity groups (as users cannot derive new identity keys). Our model (and that of [15]) differs from all these works, as we aim to trace colluding master-key shareholders of thresholdized committees, whereas traitor tracing for broadcast encryption focuses on tracing end-users that construct illegal decryption devices in the non-threshold setting.

In [44], they construct a traceable PRF and use it as building block for constructing CCA-secure fingerprinting-based traitor tracing in the symmetric key setting, whereas in this paper we consider the public key setting. Consistency, which is another property that we study, was defined and studied for the non-threshold setting in [53]. In their paper they suggest working in the white-box decoder model, in which the tracer can also access the code of the decoder, rather than only having black-box access.

1.6 Roadmap

In Section 2 we present some preliminaries. Then, in Section 3 we discuss the definition of CCA-secure TT-KEM and present a transformation from CCA-secure BT-KEM to CCA-secure TT-KEM. In Section 4, we introduce identity-based BT-KEM (BTIB-KEM) as our main building block and show how to construct CCA-secure BT-KEM from BTIB-KEM. We present two BTIB-KEM constructions in Section 5. In Section 6, we discuss the efficiency of our constructions. Finally, we discuss consistency in Section 7.

2 PRELIMINARIES

Notation. We denote the security parameter as $\lambda \in \mathbb{N}$, the key space for KEM protocols as $\mathcal{K}(\lambda)$, and the identity space for identity-based protocols as $\text{IDSpace}(\lambda)$. All protocols have an implicit algorithm $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ that, given the security parameter, outputs public parameters pp . The public parameters are implicitly input to all subsequent algorithms. We write $x \leftarrow y$ to assign an expression y to variable x and $x \leftarrow S$ to sample uniformly from the set S . For a probabilistic algorithm A we denote by $y \leftarrow A(x; r)$ the execution of A with input x and randomness r , assigning the result to y . In short, we usually write $y \leftarrow A(x)$ to express the execution of A with uniform randomness. We denote the computational indistinguishability of two distributions X and Y as $X \approx_c Y$. For a positive integer n , we denote the set $\{1, \dots, n\}$ as $[n]$. By $\{a_i\}_{i \in S}$ we denote the indexed set $\{(i, a_i) | i \in S\}$. We denote as $l_{i,S}(x)$ the Lagrange coefficient for interpolation among a set S at position x with $l_{i,S}(x) = \prod_{k \in S, k \neq i} \frac{x - x_k}{x_i - x_k}$. We write l_i instead when S is clear from context and $x = 0$. We assume that all shareholders in a (t, n) -threshold cryptosystem have participant indices in $[n]$.

Cryptography Building Blocks. We rely on three cryptographic building blocks, namely one-time signature schemes $\Sigma = (\text{KGen}, \text{Sign}, \text{Verify})$ and non-interactive zero-knowledge proofs of knowledge (NIZK-PoK) $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$. For the sake of brevity, we refer the reader to Appendix B.1 for detailed definitions. Additionally, we use hash functions in the random oracle model.

Bilinear Pairing Ensembles. Our constructions rely on *bilinear pairing ensembles*. A bilinear pairing ensemble $E = (\mathbb{G}_1, g_1, \mathbb{G}_2, g_2, \mathbb{G}_T, p, e)$ is an ensemble of cyclic groups $\mathbb{G}_1, \mathbb{G}_2$ with generators g_1 and g_2 and $\mathbb{G}_T$ of prime order p along with a pairing operators

$e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ that satisfies bilinearity and non-degeneracy. We rely on the Bilinear (Decisional) Diffie-Hellman (BDDH) [12] assumption as well as the XDH assumption [4, 10], which are both well-established and have been used in numerous works. We define them formally in Appendix B.2.

3 CCA-SECURE TRACEABLE THRESHOLD KEM

We first define our goal of CCA-secure Traceable Threshold KEM (TT-KEM). Then, in Section 3.2, we introduce some interesting technical modifications to the definitions of BT-KEM primitive to accommodate for CCA security. Finally, in Section 3.3, we show how to transform CCA-secure BT-KEM to CCA-secure TT-KEM.

3.1 Definition: TT-KEM with CCA-security

First, we introduce traceable threshold KEM, which was first defined in [15]. We make minor modifications to also define CCA-security.

Definition 3.1 (Traceable Threshold KEM). A traceable threshold KEM (TT-KEM) protocol is a tuple of algorithms $\Pi = (\text{KGen}, \text{Enc}, \text{Verify}, \text{PDec}, \text{Comb}, \text{Trace})$ with key space $\mathcal{K}(\lambda)$. It extends the standard threshold KEM by a tracing algorithm. The differences compared to standard threshold KEM are highlighted in gray.

- $(\text{pk}, \{\text{sk}_i\}_{i \in [n]}, \text{tk}) \leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\epsilon(\lambda)})$. The KGen algorithm receives the number of parties n as well as the threshold t with $n \geq t > 0$ as well as an additional parameter ϵ , which is essentially a lower bound on the distinguishing advantage of decoders that must be traceable by this scheme (see Definitions B.8 and B.9). KGen outputs a public key pk , the secret key shares $\text{sk} = \{\text{sk}_i\}_{i \in [n]}$, and a tracing key tk .
- $(k, c, \tau) \leftarrow \text{Enc}(\text{pk})$. Given a public key pk , the Enc algorithm returns a key $k \in \mathcal{K}(\lambda)$, a ciphertext c , and a tag τ .
- $1/0 \leftarrow \text{Verify}(c, \tau)$. Given a ciphertext c and a tag τ , the Verify algorithm returns 1 if τ is valid for c and 0 otherwise.
- $d_i \leftarrow \text{PDec}(\text{sk}_i, c)$. The partial decryption algorithm receives a secret key share sk_i of party i and a ciphertext c as input and outputs a decryption share d_i .
- $k/\perp \leftarrow \text{Comb}(S, \{d_i\}_{i \in S}, c)$. The Comb algorithm aggregates decryption shares d_i from a set of at least t parties S and outputs the key k encapsulated in the ciphertext c .
- $\mathcal{J} \leftarrow \text{Trace}^{D(\cdot)}(\text{pk}, \text{tk}, 1^{1/\epsilon(\lambda)})$. This algorithm receives a public key pk , a tracing key tk , the parameter ϵ , and black-box access to a decoder D . It outputs a set of traitors $\mathcal{J} \subseteq [n]$.

We defer the correctness and IND-CCA definitions to Appendix B.3, but we note that they are the same as standard threshold KEM. In summary, correctness states that every honestly generated ciphertext decrypts correctly, using shares from any set of at least t parties. For IND-CCA security, we require that an adversary who controls up to $t - 1$ parties is unable to distinguish if a challenge ciphertext belongs to a valid key or a random key from $\mathcal{K}(\lambda)$ instead. This needs to hold even if the adversary has access to a partial decryption oracle that allows him to learn partial decryptions on any ciphertext, but the challenge ciphertext. Crucially, the adversary

gets access to the tracing key tk to model that CCA-security must hold even if the tracer is malicious.

Traceability. As in [15] traceability guarantees that if an adversary corrupts an arbitrary set of parties $\mathcal{J} \subseteq [n]$ (the traitors) and uses their shares to construct a decoder D that has decryption advantage ϵ , then the $\text{Trace}^{D(\cdot)}$ algorithm, when given black-box access to the decoder, will output a nonempty subset $\mathcal{J}' \subseteq \mathcal{J}$ of traitors. It also captures unframeability, that is, Trace 's output $\mathcal{J}'$ cannot contain a non-traitor with non-negligible probability. Since we add tags τ for CCA-security, the decoder now also receives tags along with ciphertexts. The formal definition can be found in Appendix B.3 (Definition B.9).

Distinguishing Decoders. Following [15], our traceability definition considers distinguishing decoders rather than decrypting decoders. That is, instead of decrypting a given ciphertext c , decoders in our security games also receive an element $k \in \mathcal{K}(\lambda)$ and output 1 or 0, distinguishing whether c encapsulates k or not. This is a more general model, since any decrypting decoder can be trivially transformed into a distinguishing decoder. It is stronger because it even captures decoders that do not learn the entire key k but any non-trivial information about k .

3.2 Definition: BT-KEM with CCA-security

Next, we extend the BT-KEM definition from [15] to accommodate for CCA-security. Recall that in [15], the one-sided security notion was introduced in order to allow traceability. Hence, the main technical challenge is to achieve CCA-security while maintaining one-sided security of the BT-KEM scheme.

Defining Bipartite Threshold Key Encapsulation (BT-KEM). A natural way to define CCA security for BT-KEM is to let Enc algorithm output an additional tag τ to prove the integrity of the ciphertext. In our definition, we need to deviate from this approach to allow for schemes that offer both one-sided security and CCA security. More concretely, in the one-sided security game, the challenger generates two ciphertexts $c^{(i)} = (c_0^{(i)}, c_1^{(i)})$ for $i \in \{0, 1\}$ and ensembles the challenge, either $c^* = (c_0^{(0)}, c_1^{(1)})$ when $b = 0$, or $c^* = (c_0^{(d)}, c_1^{(d)})$ when $b = 1$ (here $d \in \{0, 1\}$ is chosen by the adversary). If we let Enc output τ , then syntactically, we cannot receive tags for a malformed ciphertext c^* generated when $b = 0$.

To address this issue, we add algorithms to BT-KEM that allow us to generate tags both for valid ciphertexts during standard encryption and for malformed ciphertexts, as encountered in the one-sided security game. Concretely, we split encryption and tag generation into three distinct algorithms: TagKeys, Enc, and GenTag. The TagKeys algorithm generates a key pair $(\text{sk}_e, \text{vk}_e)$, where vk_e serves as a verification key. Encryption proceeds as $(k, (c_0, c_1)) \leftarrow \text{Enc}(\text{pk}, j, \text{vk}_e)$ ⁸. Finally, GenTag uses the secret key sk_e to produce a tag $\tau \leftarrow \text{GenTag}(c, \text{sk}_e)$. Looking ahead, when constructing TT-KEM from BT-KEM, this mechanism is employed in two distinct ways:

⁸Notice that the encryption takes vk_e as input. Conceptually, given $(\text{sk}_e, \text{vk}_e)$, one can generate a valid tag using sk_e for any ciphertext that was encrypted with the corresponding vk_e as additional input.

- (1) **Tags for Normal Encryption:** The three algorithms operate as described above. The encryptor first uses TagKeys to prepare a fresh key pair, then invokes Enc with vk_e to encrypt, and produces a tag within GenTag using sk_e .
- (2) **Tags during Tracing with Malformed Ciphertexts:** Malformed ciphertexts are constructed by mixing the left and right parts of two *different* ciphertexts (refer to the one-sided security game in Figure 3). To attach a valid tag, the tracer prepares a single key pair $(sk_e, vk_e) \leftarrow \text{TagKeys}(1^\lambda)$. Then, it generates both $c^{(0)}$ and $c^{(1)}$ using two independent Enc operations, *but with the same* vk_e . Finally, the tracer uses GenTag to obtain a valid tag for the resulting malformed ciphertext, given that both original ciphertexts were generated under the same vk_e that corresponds to sk_e .

Additionally, we add a Verify algorithm to validate ciphertext-tag pairs prior to partial decryption. In practice, shareholders will only release partial decryptions for ciphertexts with valid tags.

The requirement that tags, as generated in normal encryption, verify is captured by two-sided correctness, while the latter requirement that tags verify for malformed ciphertexts is implicitly captured by our modified one-sided security (Figure 3) and two-sided correctness. In particular, one-sided security also requires that the additional tag does not gain the adversary any advantage in distinguishing whether it receives a malformed ciphertext or not.

Definition 3.2 (Bipartite Threshold Key Encapsulation (BT-KEM)). A BT-KEM scheme is a tuple of algorithms $\Pi = (\text{KGen}, \text{TagKeys}, \text{Enc}, \text{GenTag}, \text{Verify}, \text{PDec}, \text{Comb})$ with key space $\mathcal{K}(\lambda)$. The necessary modifications of BT-KEM to achieve CCA-Security are highlighted in gray.

- $(pk, \{(sk_{i,0}^{(j)}, sk_{i,1}^{(j)})\}_{i \in [n], j \in [\ell]}) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$. KGen receives the security parameter λ , the number of parties n , a threshold t such that $0 < t \leq n$, and the number of positions $\ell \in \mathbb{N}$. It outputs a public key pk and, for every $(i, j) \in [n] \times [\ell]$, secret shares $(sk_{i,0}^{(j)}, sk_{i,1}^{(j)})$. We call $sk_{i,0}^{(j)}$ a *left* key share and $sk_{i,1}^{(j)}$ a *right* key share. As a shorthand, we denote the set of all secret key shares as sk and all secret key shares for party i as sk_i . Further, $sk_{i,0}$ denotes all the left shares in sk_i while $sk_{i,1}$ denotes all the right shares in sk_i .
- $(sk_e, vk_e) \leftarrow \text{TagKeys}(1^\lambda)$. This algorithm outputs a key pair (sk_e, vk_e) that is used to generate tags for ciphertexts.
- $(k, c := (c_0, c_1)) \leftarrow \text{Enc}(pk, j, vk_e)$. The Enc algorithm gets the public key pk , a position $j \in [\ell]$, and a verification key vk_e . It outputs a key $k \in \mathcal{K}(\lambda)$ and a ciphertext $c := (c_0, c_1)$.
- $\tau \leftarrow \text{GenTag}(c, sk_e)$. The GenTag algorithm receives a ciphertext c and a key sk_e . It outputs a tag τ .
- $1/0 \leftarrow \text{Verify}(c, \tau)$. Verify outputs 1, iff the tag is valid for c .
- $d_i \leftarrow \text{PDec}(sk_{i,b_{i,j}}^{(j)}, c)$. The partial decryption algorithm receives a left ($b_{i,j} = 0$) or right ($b_{i,j} = 1$) secret share of i for a position $j \in [\ell]$ and a ciphertext c . It outputs a partial decryption d_i .
- $k/\perp \leftarrow \text{Comb}(S, \{d_i\}_{i \in S}, c)$. Given decryption shares d_i from a set of at least t parties S and a ciphertext c , the Comb algorithm returns the KEM key k or $\perp$ upon failure.

Definition 3.3 (Two-Sided Correctness of BT-KEM). A BT-KEM protocol is two-sided correct if for all $\lambda, \ell > 0$, all $j \in [\ell]$ and $n \geq t > 0$ as well as all $S \in [n]$ with $|S| \geq t$ and all bitstrings $b \in \{0, 1\}^n \times \{0, 1\}^\ell$ it holds that

$$\Pr \left[\begin{array}{l} (pk, sk) \leftarrow \text{KGen}(1^\lambda, n, t, \ell) \\ (sk_e, vk_e) \leftarrow \text{TagKeys}(1^\lambda) \\ (k, c) \leftarrow \text{Enc}(pk, j, vk_e) \\ \tau \leftarrow \text{GenTag}(c, sk_e) \\ \forall i \in S: d_i \leftarrow \text{PDec}(sk_{i,b_{i,j}}^{(j)}, c) \\ k' \leftarrow \text{Dec}(S, \{d_i\}_{i \in S}, c) \end{array} : \begin{array}{l} k = k' \wedge \\ \text{Verify}(c, \tau) = 1 \end{array} \right] = 1.$$

3.2.1 CCA and One-Sided Security. Next, we define the required security games for modeling CCA-security in BT-KEM. First, we define a CCA-security game. Then, we present a one-sided security game. Our one-sided security game follows [15] but with slight modifications that are needed to achieve CCA-security.

CCA-security Game. In Game-IND-CCA (Figure 2), the adversary can corrupt a set of up to $t - 1$ parties C . We also give the adversary access to a partial decryption oracle $\mathcal{O}^{\text{PDec}}$ that derives the left and right partial decryptions for ciphertexts on behalf of honest parties⁹. The only restriction is that the adversary cannot use $\mathcal{O}^{\text{PDec}}$ to obtain decryption shares on the challenge ciphertext c^* . Additionally, the challenge ciphertext now also contains the tag τ^* . The adversary wins if it can successfully distinguish between a random KEM-key and the one encapsulated in c^* .

Game-IND-CCA $_{\mathcal{A}}(1^\lambda)$	$\mathcal{O}^{\text{PDec}}((c, \tau), i)$
$b \leftarrow \{0, 1\}$	if $\text{Verify}(c, \tau) = 0$ then
$(\ell, st_1) \leftarrow \mathcal{A}_1(1^\lambda)$	return $\perp$
$(pk, \{sk_i\}_{i \in [n]}) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$	if $c = c^*$ then
$(C, st_2) \leftarrow \mathcal{A}_2(st_1, pk)$	return $\perp$
if $C \not\subseteq [n] \vee C \geq t$ then return 0	$d_{i,0} \leftarrow \text{PDec}(c, sk_{i,0})$
$j^* \leftarrow [\ell]; k_0 \leftarrow \mathcal{K}(\lambda)$	$d_{i,1} \leftarrow \text{PDec}(c, sk_{i,1})$
$(sk_e^*, vk_e^*) \leftarrow \text{TagKeys}(1^\lambda)$	return $(d_{i,0}, d_{i,1})$
$(k_1, c^*) \leftarrow \text{Enc}(pk, j^*, vk_e^*)$	
$\tau^* \leftarrow \text{GenTag}(c^*, sk_e^*)$	
$b' \leftarrow \mathcal{A}_3^{\text{PDec}}(st_2, (c^*, \tau^*), k_b), \{sk_i\}_{i \in C}$	
return $b \stackrel{?}{=} b'$	

Figure 2: IND-CCA security game for BT-KEM. The changes to the IND-CPA game in [15] are highlighted in gray.

Definition 3.4 (IND-CCA Security of BT-KEM). A BT-KEM protocol achieves IND-CCA security, if for all PPT adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr \left[\text{Game-IND-CCA}_{\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where Game-IND-CCA is defined in Figure 2.

⁹Note that in normal decryptions, the parties contribute either a left or a right share, not both.

Modified One-Sided Security Game. We define one-sided security similarly to [15], but account for the CCA-tags that are attached to ciphertexts. The adjusted Game-OSS experiment (Figure 3) first prepares a key pair $(sk_e, vk_e) \leftarrow \text{TagKeys}(1^\lambda)$, assembles the challenge ciphertext c^* as in [15], and finally uses sk_e to generate a tag τ^* for c^* as $\tau^* \leftarrow \text{GenTag}(c^*, sk_e)$. We give τ^* to the adversary; thus in our security analysis, we need to show that the tag does not help the adversary to distinguish whether it receives a valid ciphertext (case $b = 0$) or malformed ciphertext (case $b = 1$). We recall that the “natural” approach of generating τ^* within Enc does not work even on a definition level: If the two encryptions to ciphertexts $c^{(0)}$ and $c^{(1)}$ constructed in Game-OSS generated separate tags $\tau^{(0)}$ and $\tau^{(1)}$, then it is unclear which tag should be given to the adversary $\mathcal{A}_2$ together with the challenge ciphertext c^* . Moreover, these tags are not valid for a mixed ciphertext c^* (i.e., when $b = 0$).

Game-OSS $_{\mathcal{A}}(1^\lambda)$

$(\ell, u, d, st_1) \leftarrow \mathcal{A}_1(1^\lambda)$

$(pk, sk) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$

$(sk_e, vk_e) \leftarrow \text{TagKeys}(1^\lambda)$

$(k^{(0)}, c^{(0)}) = (c_0^{(0)}, c_1^{(0)}) \leftarrow \text{Enc}(pk, u, vk_e)$

$(k^{(1)}, c^{(1)}) = (c_0^{(1)}, c_1^{(1)}) \leftarrow \text{Enc}(pk, u, vk_e)$

$b \leftarrow \{0, 1\}$

if $b = 0$ then $c^* \leftarrow (c_0^{(0)}, c_1^{(1)})$

if $b = 1$ then $c^* \leftarrow (c_0^{(d)}, c_1^{(d)})$

$\tau^* \leftarrow \text{GenTag}(c^*, sk_e)$

shares $\leftarrow \left(\{ (sk_{i,0}^{(j)}, sk_{i,1}^{(j)}) \}_{i \in [n], j \in [\ell] \setminus \{u\}}, \{ sk_{i,d}^{(u)} \}_{i \in [n]} \right)$

$b' \leftarrow \mathcal{A}_2(st_1, pk, (c^*, \tau^*), k^{(0)}, \text{shares})$

return $b \stackrel{?}{=} b'$

Figure 3: One-sided security game for BT-KEM. The modifications compared to [15] are highlighted in gray.

Definition 3.5 (One-sided Security of BT-KEM). A BT-KEM protocol achieves one-sided security, if for all PPT adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2)$ there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr \left[\text{Game-OSS}_{\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where Game-OSS is defined in Figure 3.

3.3 Generic Construction: CCA-BT-KEM to CCA-TT-KEM

In this section, we show that plugging any CCA-secure BT-KEM, as presented in Section 3.2, in the black-box TT-KEM construction of [15] results in a CCA-secure TT-KEM (later, in Section 4.2 we construct a CCA-secure BT-KEM). We need to apply some minor modifications to TT-KEM in order to account for the changes we made to BT-KEM. Next, we will overview the transformation and discuss our modifications.

The BT-KEM to TT-KEM Transformation [15]. The transformation samples a fingerprinting code $\Gamma_{n,\ell}$ of n codewords of size ℓ , sets the tracing key tk to be the tracing key in Γ , and then receives the secret shares from BT-KEM, where the number of positions is set to ℓ (i.e., the size of each codeword). Each party is assigned a codeword, and the secret shares are distributed according to the corresponding codeword; if the j 'th bit of the i 'th codeword is 0, then party i receives the left share at position j , and the right share otherwise. Hence, each party receives exactly one share for every position $j \in [\ell]$. In encryption, TT-KEM.Enc samples a position $j \leftarrow [\ell]$ at random and invokes BT-KEM.Enc(pk, j) with the sampled position.

In our case, we adjust how TT-KEM.Enc uses BT-KEM for encryption. That is, instead of simply invoking BT-KEM.Enc(pk, j) with a random $j \in [\ell]$, it should first prepare a key pair $(sk_e, vk_e) \leftarrow \text{TagKeys}(1^\lambda)$, then encrypt $(k, c) \leftarrow \text{Enc}(pk, j, vk_e)$ for a random $j \in [\ell]$, and finally compute the tag $\tau \leftarrow \text{GenTag}(c, sk_e)$. The resulting encryption algorithm outputs the pair (c, τ) . Also, we set TT-KEM.Verify to BT-KEM.Verify so shareholders can check the tag's validity before releasing partial decryptions on a ciphertext.

Note the resulting TT-KEM scheme inherits its IND-CCA-security directly from that of our CCA-secure BT-KEM. We next discuss traceability.

Modifications to Tracing. We modify the Tr subroutine of the tracing algorithm in order to generate tags for the malformed ciphertexts. The new Tr subroutine is shown in Figure 4. Observe that the modified Tr subroutine generates the tag in exactly the same manner as for the challenge ciphertext in the modified one-sided security game (Figure 3). The Trace algorithm is otherwise unchanged from that of [15]. Intuitively, our modifications to Tr do not break traceability because our modified one-sided security notion guarantees that for a secure BT-KEM, the additional tag τ^* does not help the adversary to distinguish malformed ciphertexts from normal ciphertexts. We refer to Appendix D.3 for a detailed overview of how the proof of traceability in [15] needs to be adjusted for our needs. We next overview how tracing uses the Tr subroutine.

Overview of Tracing [15]. The tracing works by running the $\text{Tr}^{D(\cdot)}$ subroutine for every index $j \in \ell$ with either valid ciphertexts or ciphertexts that are malformed in some way. Specifically, it queries the Tr subroutine for every position j with three different configurations: A completely valid ciphertext $((b_k, b_0, b_1) = (1, 1, 1))$, a valid left and invalid right ciphertext $((b_k, b_0, b_1) = (0, 0, 1))$, and an invalid left and right ciphertext $((b_k, b_0, b_1) = (1, 0, 0))$. For every position j and each one of the three configurations, the tracer runs the decoder N times and computes the number of times it returns 1 (indicating an invalid ciphertext). The outputs for every position j are used to determine whether the decoder contains left keys for j , right keys for j , or whether it cannot be determined. That is, let p_{001} be the number of times the decoder returns 1 when the configuration is $(0, 0, 1)$. The values p_{111} and p_{100} are defined similarly. Then, if $|p_{001} - p_{111}|$ is large enough, the tracer concludes that the decoder has right keys for position j since this indicates that it was able to distinguish that the right ciphertext is valid in one case and invalid in the other. If, on the other hand, $|p_{001} - p_{100}|$ is large enough, then the tracer concludes

```

 $\text{Tr}^{D(\cdot)}(\text{pk}, j, N, (b_k, b_0, b_1))$ 
ctr  $\leftarrow$  0
for  $r = 1, \dots, N$  do
   $(\text{sk}_e, \text{vk}_e) \leftarrow \text{TagKeys}(1^\lambda)$ 
   $(k^{(0)}, c^{(0)} \leftarrow (c_0^{(0)}, c_1^{(0)})) \leftarrow \text{BT-KEM.Enc}(\text{pk}, j, \text{vk}_e)$ 
   $(k^{(1)}, c^{(1)} \leftarrow (c_0^{(1)}, c_1^{(1)})) \leftarrow \text{BT-KEM.Enc}(\text{pk}, j, \text{vk}_e)$ 
   $c^* \leftarrow (c_0^{(b_0)}, c_1^{(b_1)})$ 
   $\tau^* \leftarrow \text{GenTag}(c^*, \text{sk}_e)$ 
  if  $D((c^*, \tau^*), k^{(b_k)}) = 1$  then ctr  $\leftarrow$  ctr + 1
return ctr

```

Figure 4: The adjusted Tr subroutine of Trace. Changes compared to that of [15] are highlighted in gray. $(b_k, b_0, b_1) \in \{0, 1\}^3$ and N is the number of iterations fixed by the tracer. The Trace algorithm uses Tr to determine whether the traitors have left or right keys for index j .

that the decoder has left keys since this indicates that it was able to distinguish that the left ciphertext is valid in one case and invalid in the other. If neither condition holds (that is, the number of times is almost equal in all configurations), then the tracer cannot determine whether the decoder has left keys, right keys, or both in that position.

The results are compiled into a noisy codeword $\bar{w}$, which is essentially just a string of length ℓ , where 0 encodes “left keys at positions j ”, 1 encodes right keys at position j , and ‘?’ encodes that the tracing algorithm cannot determine. The ‘?’ positions are called *noisy*. In the end, the codeword $\bar{w}$ is fed into the tracing algorithm of the fingerprinting code along with the tracing key tk to determine the set of traitors. The underlying fingerprinting scheme then allows to identify a subset of colluders from the noisy codeword.

COROLLARY 3.6. *Let Π be a BT-KEM protocol with two-sided correctness (Definition 3.3), CCA-security (Definition 3.4), and one-sided security (Definition 3.5). Applying the traceability transformation of [15] to Π with the above changes to Enc, Verify, and Tr yields a CCA-secure TT-KEM as defined in Section 3.1.*

4 CCA-BT-KEM FROM IDENTITY-BASED-BT-KEM

In this section, we introduce our main building block of Bipartite Threshold Identity-based KEM (BTIB-KEM). It is central to our results, as we use it both to construct CCA-secure BT-KEM (and CCA-secure TT-KEM as a consequence) and traceable threshold IBE. We start with a new definition of BTIB-KEM and show how to generically transform that to CCA-secure BT-KEM in Section 4.2.

4.1 Definition: Bipartite Threshold Identity Based Key Encapsulation (BTIB-KEM)

Our BTIB-KEM definition is an adaptation of the bipartite threshold KEM definition of [15] to the identity-based setting.

Definition 4.1 (Bipartite Threshold Identity Based Key Encapsulation). A BTIB-KEM protocol is a tuple of algorithms $\Pi = (\text{KGen}, \text{Enc}, \text{Dldk}, \text{Combldek}, \text{Dec})$ with key space $\mathcal{K}(\lambda)$ and identity space $\text{IDSpace}(\lambda)$ such that:

- $(\text{pk}, \{\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)}\}_{i \in [n], j \in [\ell]}) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$. This algorithm has the same syntax as BT-KEM.KGen (Definition 3.2).
- $(k, c \leftarrow (c_0, c_1)) \leftarrow \text{Enc}(\text{pk}, j, \text{ID})$. Given a public key pk , index $j \in [\ell]$, and an identity $\text{ID} \in \text{IDSpace}(\lambda)$, Enc outputs an encapsulation key $k \in \mathcal{K}(\lambda)$ and an encryption of k to identity ID. The ciphertext $c = (c_0, c_1)$, is split into a *left ciphertext* c_0 and *right ciphertext* c_1 .
- $\text{idk}_i \leftarrow \text{Dldk}(\{\text{sk}_{i,b_{i,j}}^{(j)}\}_{j \in [\ell]}, \text{ID})$. The identity key derivation algorithm Dldk receives i ’s secret key share $\text{sk}_{i,b_{i,j}}^{(j)}$ for every index $j \in [\ell]$. If $b_{i,j} = 0$, then the given share in position j is the left share. Otherwise, it is the right share. Dldk outputs an identity key share idk_i for identity ID. We also define a version of this algorithm that only derives the identity-key share for position j as $\text{idk}_i^{(j)} \leftarrow \text{Dldk}_j(\text{sk}_{i,b_{i,j}}^{(j)}, \text{ID}, j)$.
- $\text{idk} \leftarrow \text{Combldek}(S, \{\text{idk}_i\}_{i \in S})$. The Combldek algorithm combines at least t identity key shares idk_i from parties $i \in S$ where $S \subseteq [n]$ and $|S| \geq t$ into a full identity key idk . Again, we define a version of Combldek specifically for position j as $\text{idk}^{(j)} \leftarrow \text{Combldek}_j(S, \{\text{idk}_i^{(j)}\}_{i \in S})$.
- $k/\perp \leftarrow \text{Dec}(\text{idk}, c, j)$. Given an identity key idk , a ciphertext $c = (c_0, c_1)$, and an index $j \in [\ell]$, Dec decrypts c and returns the encapsulation key k or $\perp$ in case of failure.

A BTIB-KEM must satisfy two-sided correctness, semantic security, and one-sided security.

Definition 4.2 (Two-Sided Correctness of BTIB-KEM). A BTIB-KEM protocol is two-sided correct if for all $\lambda, \ell > 0, j \in [\ell]$ and $n \geq t > 0$ as well as all $S \subseteq [n]$, where $|S| \geq t$, all bitstrings $b \in \{0, 1\}^n \times \{0, 1\}^\ell$ and all identities $\text{ID} \in \text{IDSpace}(\lambda)$ it holds that

$$\Pr \left[k = k' : \forall i \in S: \text{idk}_i \leftarrow \text{Dldk}(\{\text{sk}_{i,b_{i,j}}^{(j)}\}_{j \in [\ell]}, \text{ID}), \text{idk} \leftarrow \text{Combldek}(S, \{\text{idk}_i\}_{i \in S}), k' \leftarrow \text{Dec}(\text{idk}, c, j) \right] = 1.$$

Semantic Security. For semantic security, we work in the selective identity model [11, 22], where the adversary is required to commit to the identity it intends to attack ahead of time. While this is a relaxation, we later show that *selective-identity* CPA-secure BTIB-KEM is sufficient for our transformation to BT-KEM to achieve CCA-security. Both of our BTIB-KEM constructions can be proven without selective identity using standard techniques (i.e. guessing the challenge identity in the random oracle with a tightness loss in the number of random oracle queries q_H [7]).

Definition 4.3 (Selective ID Semantic Security). A BTIB-KEM protocol achieves selective-identity semantic security (IND-SID-CPA),

Game-IND-SID-CPA $_{\mathcal{A}}(1^\lambda)$	$\mathcal{O}^{\text{DIdk}}(\text{ID}, i)$
$b \leftarrow \{0, 1\}$	if $\text{ID} = \text{ID}^*$ then
$(\text{ID}^*, \ell, \text{st}_1) \leftarrow \mathcal{A}_1(1^\lambda)$	return $\perp$
$(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$	$\text{idk}_0 \leftarrow \text{DIdk}(\text{sk}_{i,0}, \text{ID})$
$(C, \text{st}_2) \leftarrow \mathcal{A}_2(\text{st}_1, \text{pk})$	$\text{idk}_1 \leftarrow \text{DIdk}(\text{sk}_{i,1}, \text{ID})$
if $C \notin [n] \vee C \geq t$ then return 0	return $(\text{idk}_0, \text{idk}_1)$
$j^* \leftarrow [\ell]; k_0 \leftarrow \mathcal{K}(\lambda)$	
$(k_1, c) \leftarrow \text{Enc}(\text{pk}, j^*, \text{ID}^*)$	
$b' \leftarrow \mathcal{A}_3^{\mathcal{O}^{\text{DIdk}}}(\text{st}_2, (c, k_b), \{\text{sk}_i\}_{i \in C})$	
return $b \stackrel{?}{=} b'$	

Figure 5: IND-SID-CPA security game for BTIB-KEM.

if for all PPT adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr \left[\text{Game-IND-SID-CPA}_{\mathcal{A}}^{\text{BTIB-KEM}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where Game-IND-SID-CPA is defined in Figure 5.

Game-OSS-ID $_{\mathcal{A}}(1^\lambda)$
$(\ell, u, d, \text{ID}, \text{st}_1) \leftarrow \mathcal{A}_1(1^\lambda)$
$(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$
$(k^{(0)}, c^{(0)}) \leftarrow (c_0^{(0)}, c_1^{(0)}) \leftarrow \text{Enc}(\text{pk}, u, \text{ID})$
$(k^{(1)}, c^{(1)}) \leftarrow (c_0^{(1)}, c_1^{(1)}) \leftarrow \text{Enc}(\text{pk}, u, \text{ID})$
$b \leftarrow \{0, 1\}$
if $b = 0$ then $c^* \leftarrow (c_0^{(0)}, c_1^{(1)})$
if $b = 1$ then $c^* \leftarrow (c_0^{(d)}, c_1^{(d)})$
shares $\leftarrow \left(\{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{i \in [n], j \in [\ell] \setminus \{u\}}, \{\text{sk}_{i,d}^{(u)}\}_{i \in [n]} \right)$
$b' \leftarrow \mathcal{A}_2(\text{st}_1, \text{pk}, c^*, k^{(0)}, \text{shares})$
return $b \stackrel{?}{=} b'$

Figure 6: One-sided security game for BTIB-KEM.

One-Sided Security. Following [15], we define one-sided security for BTIB-KEM through Game-OSS-ID (Figure 6), but in the identity-based setting. When we later use bipartite identity-based key encapsulation as a building block for traceable threshold identity-based KEM, we will rely on one-sided security of BTIB-KEM to ensure that we can identify a set of traitors. Roughly speaking, we will use one-sided security to argue that if a decoder has only left keys ($d = 0$) for some index u , then it cannot distinguish, given $k, (c_0, c_1)$, whether c_1 fits c_0 and k or not. Importantly, if BTIB-KEM lacks this property, then a decoder could avoid being traced at all.

Observe that the one-sided security game does not provide an identity key derivation oracle. This is reasonable, as the adversary obtains sufficient information to derive any identity key and, in consequence, decrypt any ciphertext using the master secret key shares

he is provided in Game-OSS-ID. One could imagine a stronger definition of one-sided security, where, e.g., the adversary who only obtains left keys for u gets to see *partial right-side identity-keys*, provided they are for a different identity than the challenge identity. This oracle, however, is not required to prove our transformation to CCA-secure BT-KEM. We leave it as an open problem to find constructions that can provably satisfy identity-based one-sided security with an additional identity key derivation oracle. As a direct implication, one would get even stronger traceability guarantees for TTIB-KEM and TT-KEM. We elaborate on this in Appendix C.1.

Definition 4.4 (One-sided security of BTIB-KEM). A BTIB-KEM protocol achieves one-sided security, if for all PPT adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2)$ there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr \left[\text{Game-OSS-ID}_{\mathcal{A}}^{\text{BTIB-KEM}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where Game-OSS-ID is defined in Figure 6.

Looking ahead, we will show how to construct our main building block of BTIB-KEM in Section 5.

4.2 Generic Construction: BTIB-KEM to CCA-BT-KEM

In this section, we present a black-box CCA-secure BT-KEM construction. The underlying schemes in this construction are a CPA-secure BTIB-KEM and a one-time signature scheme.

Let $\Sigma = (\text{KGen}, \text{Sign}, \text{Verify})$ be a one-time signature scheme that is EUF-OT-CMA-secure (Definition B.2) and let $\mathcal{I}$ be a BTIB-KEM that fulfills semantic security (Definition 4.3) and one-sided security (Definition 4.4).

We construct CCA-secure BT-KEM by adapting the transformation from CPA-IBE to CCA-secure encryption [11] to the bipartite setting while ensuring that our adjusted notion of one-sided security holds. The construction is depicted in Figure 7.

In our transformation, the encryptor samples a fresh signing key-pair $(\text{sk}_e, \text{vk}_e) \leftarrow \Sigma.\text{KGen}(1^\lambda)$ per encryption operation. Then it calls $(k, c_0, c_1) \leftarrow \mathcal{I}.\text{Enc}(\text{pk}, j, \text{vk}_e)$, where it encrypts to the identity “ vk_e ”. The encryptor assembles $c := ((k, c_0, c_1), \text{vk}_e, j)$ and finally calls GenTag to sign the entire ciphertext under sk_e . The signature σ is attached as CCA-tag to c . The Verify algorithm verifies σ for c under vk_e , and should be called prior to partial decryption to ensure CCA-security. During partial decryption of a ciphertext $c = ((k, c_0, c_1), \text{vk}_e, j)$, the partial identity-key for “ vk_e ” is derived using $\mathcal{I}.\text{DIdk}$. One can run Comb to combine at least t partial identity keys for vk_e and subsequently decrypt c . We highlight that it is sufficient to derive the j ’th position of the identity key during PDec. Hence, the partial decryptions do not depend on the number of positions ℓ .

Correctness and Security. The resulting BT-KEM clearly has two-sided correctness, given the two-sided correctness of BTIB-KEM and the correctness of the one-time signature scheme Σ .

For IND-CCA-security, notice that the challenge ciphertext is encrypted to the identity “ vk_e ” in $\mathcal{I}$. Due to the semantic security of $\mathcal{I}$, the adversary cannot learn any information about the challenge ciphertext on its own. Furthermore, the decryption oracle $\mathcal{O}^{\text{PDec}}$ does not give the adversary any advantage. Indeed, if the queried ciphertext contains another vk_e , then $\mathcal{O}^{\text{PDec}}$ derives shares of a

KGen ($1^\lambda, n, t, \ell$)	Verify (c, τ)
return $\mathcal{I}.\text{KGen}(1^\lambda, n, t, \ell)$	Parse c as $((c_0, c_1), j, \text{vk}_e)$ and τ as σ
TagKeys (1^λ)	return $\Sigma.\text{Verify}(\text{vk}_e, c, \sigma)$
$(\text{sk}_e, \text{vk}_e) \leftarrow \Sigma.\text{KGen}(1^\lambda)$	PDec ($\text{sk}_{i,b_{i,j}}^{(j)}, c$)
return $(\text{sk}_e, \text{vk}_e)$	$\text{ID} \leftarrow \text{vk}_e$
Enc ($\text{pk}, j, \text{vk}_e$)	$\text{idk}_i^{(j)} \leftarrow \mathcal{I}.\text{Dldk}_j(\text{sk}_{i,b_{i,j}}^{(j)}, \text{ID}, j)$
$\text{ID} \leftarrow \text{vk}_e$	return $d_i \equiv \text{idk}_i^{(j)}$
$(k, c_0, c_1) \leftarrow \mathcal{I}.\text{Enc}(\text{pk}, j, \text{ID})$	Comb ($S, \{d_i\}_{i \in S}, c$)
return $k, c = ((c_0, c_1), j, \text{vk}_e)$	Parse $d_i \equiv \text{idk}_i^{(j)}; c := ((c_0, c_1), j, \text{vk}_e)$
GenTag (c, sk_e)	$\text{idk} \leftarrow \mathcal{I}.\text{Combldk}(S, \{\text{idk}_i^{(j)}\}_{i \in S})$
$\sigma \leftarrow \Sigma.\text{Sign}(\text{sk}_e, c)$	return $\mathcal{I}.\text{Dec}(\text{idk}, (c_0, c_1), j)$
return $\tau \equiv \sigma$	

Figure 7: Black-box CCA-secure BT-KEM from any BTIB-KEM scheme $\mathcal{I}$ and signature scheme Σ .

different identity key. If it contains the same vk_e then the adversary would need to forge a signature under vk_e , which is infeasible according to the unforgeability of Σ . A detailed proof for the following lemma is presented in Appendix D.1.

LEMMA 4.5 (IND-CCA SECURITY OF BT-KEM). *The BT-KEM construction as presented in Figure 7 is IND-CCA secure if the underlying BTIB-KEM is IND-SID-CPA secure and the signature scheme is EUF-OT-CMA secure.*

LEMMA 4.6 (ONE-SIDED SECURITY OF BT-KEM). *The BT-KEM construction as presented in Figure 7 is one-sided secure (Definition 3.5) if the underlying BTIB-KEM is one-sided secure (Definition 4.4).*

The one-sided security reduces straightforwardly to one-sided security of BTIB-KEM. The detailed proof is given in Appendix D.2

5 CONCRETE INSTANTIATIONS OF BTIB-KEM

Both our results for CCA-secure TT-KEM (Section 3.3) and traceable threshold IBE (Appendix C) rely on the existence of efficient constructions for our main building block of bipartite threshold identity-based KEM (BTIB-KEM). To this end, we present two constructions based on bilinear pairings within this section. Henceforth, we refer to them as BTIB-KEM-1 and BTIB-KEM-2.

In both of our constructions, we combine techniques used in [15] with an appropriate IBE scheme. In particular, the constructions of [15] are based on the following idea. First, generate two secret keys, y, z , for the left and right sides. In order to achieve two-sided correctness, a correlation must exist between the two sides. This correlation is achieved via a master secret α . That is, the left secret shares correspond to αz , and the right secret shares correspond to αy . The encapsulated key is then $e(g_1, g_2)^{\alpha y z r}$, where r is sampled in the encryption. The one-sided security is guaranteed by the argument that αz looks random to parties that hold shares of αy , and vice versa. In BTIB-KEM, we need to accommodate for ℓ secret keys for every side. In our first construction, we treat each one of the ℓ instances independently, which results in a public key of

size ℓ . We improve this in our second construction by using a 2-Layer IBE, but at the cost of a longer ciphertext (i.e., 12 vs 2 group elements) and more complicated construction. Interestingly, as we already discussed, our second construction can be easily extended to multi-layer IBE using techniques from [7], which allows us to achieve CCA-secure traceable threshold IBE (TTIB-KEM).

In Appendix C.1, we define TTIB-KEM and show a TTIB-KEM black-box construction from BTIB-KEM, which can be instantiated with both BTIB-KEM-1 and BTIB-KEM-2. We note that the resulting TTIB-KEM scheme is only selective-ID-secure.

5.1 BTIB-KEM-1 (Short Ciphertext)

Our first BTIB-KEM construction (BTIB-KEM-1), depicted in Figure 8, is based on Boneh-Franklin IBE [12].

KGen ($1^\lambda, n, t, \ell$)	Enc (pk, j, ID)
for $j \in [\ell]$ do	$r \leftarrow \mathbb{Z}_p$
$\alpha_j, y_j, z_j \leftarrow \mathbb{Z}_p$	$k \leftarrow e(X_j^r, \mathcal{H}(j, \text{ID}))$
$(X_j, Y_j, Z_j) \leftarrow (g_1^{\alpha_j y_j z_j}, g_1^{y_j}, g_1^{z_j})$	$(c_0, c_1) \leftarrow (Y_j^r, Z_j^r)$
$\{s_i^{(j)}\}_{i \in [n]} \leftarrow \text{Shamir.Share}(\alpha_j, n, t)$	return $(k, (c_0, c_1))$
for $i \in [n]$ do	Combldk ($S, \{\text{idk}_i\}_{i \in S}$)
$\text{sk}_{i,0}^{(j)} \leftarrow z_j \cdot s_i^{(j)}$	for $j \in [\ell]$ do
$\text{sk}_{i,1}^{(j)} \leftarrow y_j \cdot s_i^{(j)}$	$S_0 \leftarrow \{i \in S \mid b_{i,j} = 0\}$
$\text{pk} \leftarrow \{(X_j, Y_j, Z_j)\}_{j \in [\ell]}$	$S_1 \leftarrow S \setminus S_0$
$\text{sk} \leftarrow \{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{j \in [\ell], i \in [n]}$	$L_j \leftarrow \prod_{i \in S_0} \text{idk}_i^{(j) L_{i,S}}$
return (pk, sk)	$R_j \leftarrow \prod_{i \in S_1} \text{idk}_i^{(j) L_{i,S}}$
Dldk ($\{\text{sk}_{i,b_{i,j}}^{(j)}\}_{j \in [\ell]}, \text{ID}$)	return $\text{idk} \equiv \{(L_j, R_j)\}_{j \in [\ell]}$
for $j \in [\ell]$ do	Dec ($\text{idk}, c = (c_0, c_1), j$)
$\text{idk}_i^{(j)} \leftarrow \mathcal{H}(j, \text{ID})^{\text{sk}_{i,b_{i,j}}^{(j)}}$	$k \leftarrow e(c_0, L_j) \cdot e(c_1, R_j)$
return $\text{idk}_i \equiv \{(b_{i,j}, \text{idk}_i^{(j)})\}_{j \in [\ell]}$	return k

Figure 8: Our BTIB-KEM-1 Construction.

Basically, we extend Boneh-Franklin IBE to achieve the requirements of two-sided correctness and one-sided security.

Setup and Key Generation. The BTIB-KEM-1 Setup generates an asymmetric pairing ensemble E as public parameters. The key space of this construction is the target group $\mathcal{K}(\lambda) = \mathbb{G}_T$ while the identity space is arbitrary. Further, we also use a cryptographic hash function $\mathcal{H}$ with output domain $\mathbb{G}_2$. During KGen we sample α_j, y_j and z_j for every $j \in [\ell]$. Intuitively, this is the equivalent of ℓ separate Boneh-Franklin IBE instances with $\alpha_j y_j z_j$ as master secret key and $X_j = g_1^{\alpha_j y_j z_j}$ as public key. For every j , we share this master key into left and right key shares by generating t -out-of- n shares of α_j , denoted by $\{s_i^{(j)}\}_{i \in [n]}$. Then, the left share party i gets for index j is $z_j \cdot s_i^{(j)}$ and the right share is $y_j \cdot s_i^{(j)}$.

Encryption. For encryption and identity key derivation, we follow Boneh-Franklin IBE and extend it to work with correlated keys.

For encryption to a specific $j \in [\ell]$ and an identity ID, one generates the KEM key as $k \leftarrow e(X_j^r, \mathcal{H}(j, \text{ID})) = e(g_1, \mathcal{H}(j, \text{ID}))^{\alpha_j y_j z_j^r}$ for a random $r \leftarrow \mathbb{Z}_p$. Instead of a single ciphertext as g_1^r , we separate this into a left and right ciphertext $(c_0, c_1) \leftarrow (Y_j^r, Z_j^r)$. While during key generation, the left shares contain z_j , here, the left ciphertext contains y_j . This allows us to combine those parts of the identity key that stem from left shares with the left part of the ciphertext and vice versa in order to decrypt.

Identity Key Derivation. Party i calls Dldk to derive the key share for ID and provides a left or right key share (the side of the share is determined by the bit $b_{i,j}$). The identity key share would be $\mathcal{H}(j, \text{ID})^{\text{sk}_{i,b_{i,j}}^{(j)}}$. That is, party i sends to the entity with ID the share $\mathcal{H}(j, \text{ID})^{z_j \cdot s_i^{(j)}}$ or $\mathcal{H}(j, \text{ID})^{y_j \cdot s_i^{(j)}}$. Using Comblmk, for every $j \in [\ell]$, one can locally aggregate t or more identity key shares from a set of parties $S = S_0 \cup S_1$ with mixed sides (left shares from S_0 and right shares from S_1) into an identity key (L_j, R_j) , where L_j represents the aggregation over S_0 and R_j the aggregation over S_1 . Note that the aggregation is computed in the exponent.

Decryption. The identity key holder can decrypt a ciphertext (c_0, c_1) for an index j and identity ID using the j 'th component of the identity key (L_j, R_j) , by computing $e(c_0, L_j) \cdot e(c_1, R_j)$. Two-sided correctness follows from the decryption equation:

$$\begin{aligned} & e(c_0, L_j) \cdot e(c_1, R_j) \\ &= e(Y_j^r, \mathcal{H}(j, \text{ID})^{z_j \sum_i^{S_0} l_{i,S} \cdot s_i^{(j)}}) \cdot e(Z_j^r, \mathcal{H}(j, \text{ID})^{y_j \sum_i^{S_1} l_{i,S} \cdot s_i^{(j)}}) \\ &= e(g_1, \mathcal{H}(j, \text{ID}))^{r y_j z_j \sum_i^S l_{i,S} \cdot s_i^{(j)}} = e(g_1, \mathcal{H}(j, \text{ID}))^{r y_j z_j \alpha_j} = k \end{aligned}$$

Security of BTIB-KEM-1. In addition to two-sided correctness, we need to prove (selective-identity) semantic security and one-sided security. Note that the main challenge in the proof over [15] is that we must simulate the identity key derivation oracle $\mathcal{O}^{\text{Dldk}}$ for semantic security as we work in the identity-based setting.

LEMMA 5.1 (IND-SID-CPA SECURITY OF BTIB-KEM-1). *BTIB-KEM-1 is IND-SID-CPA secure (Definition 4.3) if the BDDH assumption (Definition B.4) holds in the underlying pairing ensemble.*

LEMMA 5.2 (ONE-SIDED SECURITY OF BTIB-KEM-1). *BTIB-KEM-1 is one-sided secure (Definition 4.4) if the XDH assumption (Definition B.5) holds in the underlying pairing ensemble.*

The proofs are deferred to Appendix E.1 and E.2 respectively.

5.2 BTIB-KEM-2 (Constant-Size Public Key)

For simplicity, we restrict ourselves to the non-threshold case in this presentation, but we note that it can be thresholdized using the same techniques as in our first construction. We give a formal construction, along with formal security proofs, in Appendix G.

Our construction is based on the Boneh-Boyen 2-layer IBE construction [7]. In a 2-layer (or hierarchical) IBE [41], there are two layers of identities (e.g., (I_1, I_2)). With the master key, one can derive the identity key for any first-layer identity I_1 . The first-layer identity key can then be used to derive the identity key for any layer-two identity that has I_1 as prefix, i.e., $(I_1, *)$, but not $(I_1', *)$ for $I_1 \neq I_1'$. Hence, the layers of identities act as hierarchical domains.

Inspired by [15, 52], we use 2-layers in order to compress all public keys for the ℓ different positions in one layer. That is, instead of encrypting under pk_j to ID, we encrypt under a global pk and to identity (j, ID) . As secret keys, we distribute the identity keys for the positions $j \in [\ell]$. Hence, one can derive an identity key for ID at position j (that is, the identity key for (j, ID)) using the j 'th secret key. Note that the encryptor needs to know only the master public key pk, which is independent of position j .

5.2.1 Boneh-Boyen IBE [7]. We recall the non-threshold Boneh-Boyen IBE construction for two layers. For threshold version, see [9].

Setup and key generation. Choose random $\alpha, b \leftarrow \mathbb{Z}_p$ and $u_1, u_2 \leftarrow \mathbb{G}_1$. Define functions $L_1, L_2: \mathbb{Z}_p \rightarrow \mathbb{G}_1$ such that $L_1(x) = g_1^{\alpha x} \cdot u_1$ and $L_2(x) = g_1^{\alpha x} \cdot u_2$. Publish g_1^α, g_2^b, u_1 , and u_2 as public parameters. The master secret key is g_1^{ab} .

Identity key derivation. Given an identity I_1 on the first layer, sample $r_1 \leftarrow \mathbb{Z}_p$ and set $\text{idk}^{I_1} = (g_1^{ab} \cdot L_1(I_1)^{r_1}, g_2^{r_1})$. And for the second layer, given $\text{ID} = (I_1, I_2)$, sample $r_2 \leftarrow \mathbb{Z}_p$ and set $\text{idk}^{\text{ID}} = (\text{idk}_1^{I_1} \cdot L_2(I_2)^{r_2}, \text{idk}_2^{I_1} \cdot g_2^{r_2})$.

Encryption. To encrypt to identity (I_1, I_2) , choose a random $s \in \mathbb{Z}_p$, set $k = e(g_1^\alpha, g_2^b)^s$ and $ct = (g_2^s, L_1(I_1)^s, L_2(I_2)^s)$.

Decryption. Given idk^{ID} and ct that is encrypted under ID, compute k as $k \leftarrow e(\text{idk}_1^{\text{ID}}, ct_1) / (e(ct_2, \text{idk}_2^{\text{ID}}) \cdot e(ct_3, \text{idk}_3^{\text{ID}}))$.

5.2.2 Our BTIB-KEM-2 Construction. In our construction, we use two instances of 2-layer Boneh-Boyen IBE (one per side) with modifications to account for our new correctness and security notions, as follows.

Setup and Key Generation. The Setup algorithm generates an asymmetric pairing ensemble E . The key space is $\mathcal{K}(\lambda) = \mathbb{G}_T$. The identity space in our second construction is $\text{IDSpace} = \mathbb{Z}_p$, but can be arbitrarily extended using a hash function to $\mathbb{Z}_p$. We also use a cryptographic hash function $\mathcal{H}$ with output domain $\mathbb{G}_1$. Similarly to our first construction, we choose random $\alpha, y, z \in \mathbb{Z}_p$. As we consider two sides, we use an instance for the left side in which we set the master key for position j to be $\mathcal{H}(j)^{\alpha z}$ and for the right side $\mathcal{H}(j)^{\alpha y}$. That is, from the notation above, g_1^b is $g_1^{w_j z}$ where $\mathcal{H}(j) = g_1^{w_j}$. Denote the parameters used in the first instance by u_1, u_2, L_1, L_2 and for the second instance by v_1, v_2, R_1, R_2 (i.e., representation of the functions). The public key is set to $(g_1^\alpha, g_2^y, g_2^z, g_2^{\alpha y z}, u_1, u_2, v_1, v_2)$, which is independent of ℓ .

Secret keys. In our construction, the secret keys for each position $j \in [\ell]$ conceptually corresponds to an identity key for identity j . Hence, we generate the secret keys similar to how one would generate first-layer identity keys in the Boneh-Boyen IBE from the master secret key, except that we extend the secret key by two elements (the second and the fourth), which are needed for our two-sided correctness. That is, for every j , we first sample $r_{1,0}, r_{1,1} \leftarrow \mathbb{Z}_p$ and set left secret key for position j as

$$\text{sk}_0^{(j)} = (\mathcal{H}(j)^{\alpha z} \cdot L_1(j)^{r_{1,0}}, g_2^{y r_{1,0}}, g_2^{r_{1,0}}, g_2^{\alpha z}),$$

and the right key as $\text{sk}_1^{(j)} = (\mathcal{H}(j)^{\alpha y} \cdot R_1(j)^{r_{1,1}}, g_2^{z r_{1,1}}, g_2^{r_{1,1}}, g_2^{\alpha y})$.

Key derivation. Again, we follow the key derivation in Boneh-Boyen scheme, except with new elements similar to the first layer. For a given identity ID, we sample $r_{2,0}, r_{2,1} \leftarrow \mathbb{Z}_p$ and set the left identity key for position j as

$$\text{idk}_{\text{ID},0}^{(j)} = (\text{sk}_{0,1}^{(j)} \cdot L_2(\text{ID})^{r_{2,0}}, \text{sk}_{0,2}^{(j)}, g_2^{y r_{2,0}}, \text{sk}_{0,3}^{(j)}, g_2^{r_{2,0}}, \text{sk}_{0,4}^{(j)}),$$

$$\text{and } \text{idk}_{\text{ID},1}^{(j)} = (\text{sk}_{1,1}^{(j)} \cdot R_2(\text{ID})^{r_{2,1}}, \text{sk}_{1,2}^{(j)}, g_2^{z r_{2,1}}, \text{sk}_{1,3}^{(j)}, g_2^{r_{2,1}}, \text{sk}_{1,4}^{(j)}).$$

Encryption. To account for two-sided correctness, similar to our first construction, we modify the ciphertext such that we replace g_2^s by g_2^{ys} in c_0 and by g_2^{zs} in c_1 . Furthermore, in this case we need to mask these values with random elements t_0 and t_1 , respectively, in order to account for one-sided security. This modification entails additional elements in the ciphertext that are needed to cancel the masks t_0 and t_1 during decryption. That is, in encryption we sample $s, t_0, t_1 \leftarrow \mathbb{Z}_p$, and set the key $k = e(\mathcal{H}(j)^s, g_2^{\alpha y z})$. The resulting ciphertext is $c = (c_0, c_1)$, where

$$c_0 = (g_2^{ys+t_0}, L_1(j)^s, L_2(\text{ID})^s, L_1(j)^{t_0}, L_2(\text{ID})^{t_0}, \mathcal{H}(j)^{t_0}),$$

and $c_1 = (g_2^{zs+t_1}, R_1(j)^s, R_2(\text{ID})^s, R_1(j)^{t_1}, R_2(\text{ID})^{t_1}, \mathcal{H}(j)^{t_1})$. We note that the last three elements on each side are necessary to cancel the masks t_0 and t_1 during decryption.

Decryption. The decryption follows from a similar formula as Boneh-Boyen, but in the bipartite setting: Let $\text{idk}_0, \text{idk}_1$ be $\text{idk}_{\text{ID},0}^{(j)}, \text{idk}_{\text{ID},1}^{(j)}$, respectively. We compute k as

$$k \leftarrow \frac{e(\text{idk}_{b,1}, c_{b,1})}{\prod_{i=2}^6 e(c_{b,i}, \text{idk}_{b,i})}$$

for $b \in \{0, 1\}$. It can be computed by the left or right key since, in this simplified version, the threshold is 1. In the thresholdized version, the final key is computed as the multiplication of two decryption outputs, similar to our first construction in Figure 8.

Correctness and Security. The two-sided correctness immediately follows the decryption formula when applying the same thresholding technique as in the first construction.

Intuitively, the semantic security follows from the semantic security of each one of 2-layer IBE instances. While we use ideas from [9], due to our changes some challenges need to be resolved.

The one-sided security relies on the masks t_0 and t_1 that we applied to the ciphertexts. Note that without masking, the one-sided security is broken by, e.g., the check that $e(R_1(j)^s, g_2^y) = e(R_1(j), g_2^{ys})$. Intuitively, masking prevents this attack since the adversary does not know $R_1(j)^{t_0}$ or $L_1(j)^{t_1}$. Also, we had to introduce the random oracle in the master secret keys ($\mathcal{H}(j)^{\alpha z}$ and $\mathcal{H}(j)^{\alpha y}$) in order to simulate all secret keys but those for position j^* on side $1 - d$ in the one-sided security proof. Our proofs rely on assumptions that can be proven secure in the Generic Group Model. The full proofs appear in Appendix G.

6 EFFICIENCY ANALYSIS

We next discuss the efficiency of our constructions with respect to the ciphertext and key size, as well as computational efficiency.

Parameter Size. The table in Figure 9 summarizes the parameter sizes in the resulting CCA-secure TT-KEM construction that is obtained by plugging both of our BTIB-KEM construction into the transformation. Observe that both constructions are relatively

Param	With BTIB-KEM-1	With BTIB-KEM-2
$ c $	$2\mathbb{G}_1 + \text{vk} + \sigma + \log(\ell)$	$10\mathbb{G}_1 + 2\mathbb{G}_2 + \text{vk} + \sigma + \log(\ell)$
$ \text{pk} $	$3\ell\mathbb{G}_1$	$5\mathbb{G}_1 + 3\mathbb{G}_2$
$ \text{sk}_i $	$2\ell\mathbb{Z}_p$	$\ell(\mathbb{G}_1 + 3\mathbb{G}_2)$
$ d_i $	$1\mathbb{G}_2$	$1\mathbb{G}_1 + 5\mathbb{G}_2$
$ d $	$2\mathbb{G}_2$	$2(1\mathbb{G}_1 + 5\mathbb{G}_2)$

Figure 9: Summary of the parameter sizes of CCA-secure TT-KEM, when instantiating with BTIB-KEM-1 or BTIB-KEM-2 and a signature scheme Σ as building blocks. $|\text{vk}|$ and $|\sigma|$ refer to the length of the verification key and signature of Σ .

size-efficient in the important parameters¹⁰ that is the ciphertext c , the public key pk and the partial decryptions d_i . Also, observe that it is possible to combine at least t partial decryptions $\{d_i\}_{i \in S}$ from a set of parties S into a single decryption key d to save even more space. This decryption key d is only twice the size of any individual d_i . This works because both our constructions support the pre-aggregation of identity key shares through ComblDk.

In the TT-KEM construction, the number of positions ℓ is determined according to the codeword size in the underlying fingerprinting code. When we use the fingerprinting code due to Tardos [51], the size of ℓ is $O(n^2 \log^2 n)$.

Both of our constructions are concretely computationally efficient, with execution times in the order of milliseconds. An exact overview is laid out in Appendix H.

7 CONSISTENCY

In this section, we discuss the consistency property in TT-KEM.

Recall that the consistency requirement ensures that any coalition of parties will get the same decryption result, even with adversarially generated ciphertext and decryption shares. Our consistency notion is very strong, in the sense that it even holds against adversaries that corrupt *all* shareholders. On the other hand, we can guarantee consistency only against adversaries who do not know the tracing key tk . Intuitively, this is because we want to ensure through a trapdoored proof that the ciphertext is *well-formed* while still allowing tracing, where the tracer sends queries to the decoder box D with ciphertexts that have mismatching left and right parts.¹¹

Consistency Definition. In the consistency experiment Game-Consist (Figure 10) the adversary obtains all secret key shares for all share holders. Then, the adversary outputs a ciphertext c with the attached proof π along with two sets $S, S' \in [n]$ that each contain at least t parties. Furthermore, the adversary outputs partial

¹⁰We consider them most important since they are the most likely to be stored on the blockchain when using our protocol in scenarios such as mempool privacy.

¹¹Note that according to [53], consistency with public tracing (i.e., with no tracing key) is hard. Thus, relying on the tracing key to achieve consistency is reasonable.

decryptions (shares) $\{d_i\}_{i \in S}$ and $\{d'_i\}_{i \in S'}$ on behalf of the parties in S and S' . The game verifies the proof as well as the partial decryption shares. The proof is verified using the Verify algorithm, which already exists for TT-KEM according to definition 3.1. We add an algorithm PVerify to verify partial decryptions. If all verifications pass, the game combines the decryption shares of S and the decryption shares of S' . The adversary wins, if the resulting KEM-keys are different or either decryption fails (returns $\perp$). Observe that without the proof π , the adversary can win the game in TT-KEM, which is based on BT-KEM, as the adversary can output a ciphertext with mismatching left and right parts.

Game-Consist $\mathcal{A}(1^\lambda)$
$(pk, \{sk_i\}_{i \in [n]}, tk) \leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\epsilon(\lambda)})$
$(c, \pi), S, S', \{d_i\}_{i \in S}, \{d'_i\}_{i \in S'} \leftarrow \mathcal{A}_1(1^\lambda, pk, \{sk_i\}_{i \in [n]})$
if Verify(pk, c, π) = 0 then return 0
if $S \not\subseteq [n] \vee S \leq t$ then return 0
if $S' \not\subseteq [n] \vee S' \leq t$ then return 0
if $\exists i \in S: \text{PVerify}(pk, c, d_i) = 0$ then return 0
if $\exists i \in S': \text{PVerify}(pk, c, d'_i) = 0$ then return 0
$k \leftarrow \text{Comb}(S, \{d_i\}_{i \in S}, c)$
$k' \leftarrow \text{Comb}(S', \{d'_i\}_{i \in S'}, c)$
if $k \neq k' \vee k = \perp \vee k' = \perp$ then return 1
return 0

Figure 10: Consistency game for TT-KEM.

Definition 7.1 (Consistency of Traceable Threshold KEM). We call a TT-KEM protocol *consistent*, if for all PPT adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr \left[\text{Game-Consist}_{\mathcal{A}}^{\text{TT-KEM}}(1^\lambda) = 1 \right] \leq \text{negl}(\lambda),$$

where Game-Consist is defined in Figure 10.

Construction from Disjunctive NIZK-PoKs. In order to achieve consistency for our CCA-secure TT-KEM construction from Section 4.2, we need to add a procedure for verification of the partial decryptions and to attach a well-formedness proof of the ciphertext with a trapdoor for the tracer. For simplicity, we present the modifications needed when starting from the concrete BTIB-KEM construction in Section 5.1. The ideas presented here can be generalized to any BTIB-KEM construction.

Verification of Partial Decryption. Given a ciphertext c including vk_e the partial decryption PDec boils down to deriving the partial identity key on the identity $ID = vk_e$ for index j . Looking at the BTIB-KEM construction (Section 5), the partial identity key derivation is computed as $d_i = H(j, vk_e)^{sk_i^{(j)}}$, which is basically a BLS-signature [13] on message (j, vk_e) . Hence the correctness of partial decryptions can be easily verified through the BLS equation $e(g_1, d_i) = e(g_1^{sk_i^{(j)}}, H(j, vk_e))$, where $g_1^{sk_i^{(j)}}$ is part of the public key. We implement this check by letting the PVerify algorithm output 1, if the above equation holds and 0 otherwise.

Well-formedness of Ciphertext via a Trapdoor. In KGen, we generate a discrete-log tuple (q, Q) , where $q \leftarrow \mathbb{Z}_p$ and $Q \leftarrow g_1^q$. In the transformation in Section 4.2, the trapdoor q is included in the tracing key while the challenge Q is included in the public key. In Enc, the encryptor builds a NIZK proof of knowledge for the following relation R . For a statement $\chi = (pk, (c_0, c_1, j, vk_e))$ and a witness ω , we say that $(\chi, \omega) \in R$ if and only if it holds that:

$$(c_0 = Y_j^\omega \wedge c_1 = Z_j^\omega) \vee (Q = g_1^\omega).$$

Hence, the encryptor needs to prove that he knows a witness ω such that $(\chi, \omega) \in R$, where χ is instantiated with the ciphertext. Notice that in normal encryption, the encryptor generates a valid proof by using the randomness r that it used to encrypt c_0 and c_1 as witness. In tracing, on the other hand, the tracer can generate a valid proof by proving that it knows the trapdoor.

As discussed, this proof does not break the traceability of TT-KEM. Indeed, given a valid proof, the adversary cannot distinguish whether it was generated via normal encryption or during tracing. This holds due to the zero-knowledge property of the proof system. Finally, the resulting scheme satisfies the consistency notion, due to the knowledge-soundness of the proof system as well as the verifiability of the partial decryption shares.

THEOREM 7.2 (CONSISTENCY OF THE MODIFIED TT-KEM). *Let M-TT-KEM be the modified TT-KEM-scheme that is described here and in Section 7. M-TT-KEM is consistent, if the proof system Π achieves knowledge-soundness and the DLog assumption holds in $\mathbb{G}_1$.*

A proof of Theorem 7.2 is presented in Appendix I.

We also highlight that the proof system can be efficiently instantiated using disjunctive Schnorr proofs of knowledge, which is a standard technique [27]. The additional proof would add three $\mathbb{G}_1$ elements and four $\mathbb{Z}_p$ elements to the ciphertext over the result for BTIB-KEM-1 in Figure 9.

ACKNOWLEDGEMENTS

This work is supported in part by UKRI grant EP/Y001680/1, by the European Research Council (ERC) under the European Union's Horizon 2020 and Horizon Europe research and innovation programs (grant CRYPTOLAYER-101044770), by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts, through their joint sponsorship of the National Research Center for Applied Cybersecurity ATHENE, and by the German Research Foundation, Deutsche Forschungsgemeinschaft (DFG) via the DFG CRC 1119 CROSSING project S7.

REFERENCES

- [1] Michel Abdalla, Alexander W. Dent, John Malone-Lee, Gregory Neven, Duong Hieu Phan, and Nigel P. Smart. 2007. Identity-Based Traitor Tracing. In *Public Key Cryptography - PKC 2007, 10th International Conference on Practice and Theory in Public-Key Cryptography (Lecture Notes in Computer Science, Vol. 4450)*. Springer, 361–376. https://doi.org/10.1007/978-3-540-71677-8_24
- [2] Amit Agarwal, Rex Fernando, and Benny Pinkas. 2024. Efficiently-Thresholdizable Batched Identity Based Encryption, with Applications. *Cryptology ePrint Archive*, Paper 2024/1575. <https://eprint.iacr.org/2024/1575>
- [3] Giuseppe Ateniese, Jan Camenisch, Susan Hohenberger, and Breno de Medeiros. 2005. Practical Group Signatures without Random Oracles. *Cryptology ePrint Archive*, Paper 2005/385. <https://eprint.iacr.org/2005/385>
- [4] Lucas Ballard, Matthew Green, Breno de Medeiros, and Fabian Monrose. 2005. Correlation-Resistant Storage via Keyword-Searchable Encryption. *Cryptology ePrint Archive*, Paper 2005/417. <https://eprint.iacr.org/2005/417>

- [5] Boaz Barak. 2023. An Intensive Introduction to Cryptography: Chosen Ciphertext Security. https://intensitycrypto.org/public/lec_06_CCA.html. Accessed: 2025-01-09.
- [6] Allison Bishop, Abhishek Jain, and Lucas Kowalczyk. 2015. Function-Hiding Inner Product Encryption. In *Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29 - December 3, 2015, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 9452)*, Tetsu Iwata and Jung Hee Cheon (Eds.). Springer, 470–491. https://doi.org/10.1007/978-3-662-48797-6_20
- [7] Dan Boneh and Xavier Boyen. 2004. Efficient Selective-ID Secure Identity-Based Encryption Without Random Oracles. In *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings (Lecture Notes in Computer Science, Vol. 3027)*, Springer, 223–238. https://doi.org/10.1007/978-3-540-24676-3_14
- [8] Dan Boneh, Xavier Boyen, and Eu-Jin Goh. 2005. Hierarchical identity based encryption with constant size ciphertext. In *Annual international conference on the theory and applications of cryptographic techniques (Lecture Notes in Computer Science, Vol. 3494)*, 440–456. https://doi.org/10.1007/11426639_26
- [9] Dan Boneh, Xavier Boyen, and Shai Halevi. 2006. Chosen Ciphertext Secure Public Key Threshold Encryption Without Random Oracles. In *Topics in Cryptology - CT-RSA 2006, The Cryptographers' Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3860)*, Springer, 226–243. https://doi.org/10.1007/11605805_15
- [10] Dan Boneh, Xavier Boyen, and Hovav Shacham. 2004. Short Group Signatures. In *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings (Lecture Notes in Computer Science, Vol. 3152)*, Matthew K. Franklin (Ed.). Springer, 41–55. https://doi.org/10.1007/978-3-540-28628-8_3
- [11] Dan Boneh, Ran Canetti, Shai Halevi, and Jonathan Katz. 2007. Chosen-Ciphertext Security from Identity-Based Encryption. *SIAM J. Comput.* 36, 5 (2007), 1301–1328. <https://doi.org/10.1137/S009753970544713X>
- [12] Dan Boneh and Matthew K. Franklin. 2001. Identity-Based Encryption from the Weil Pairing. In *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2139)*, Springer, 213–229. https://doi.org/10.1007/3-540-44647-8_13
- [13] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short Signatures from the Weil Pairing. In *Advances in Cryptology - ASIACRYPT 2001*, Colin Boyd (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 514–532.
- [14] Dan Boneh and Moni Naor. 2008. Traitor tracing with constant size ciphertext. In *Proceedings of the 2008 ACM Conference on Computer and Communications Security, CCS 2008, Alexandria, Virginia, USA, October 27-31, 2008*, Peng Ning, Paul F. Syverson, and Somesh Jha (Eds.). ACM, 501–510. <https://doi.org/10.1145/1455770.1455834>
- [15] Dan Boneh, Aditi Partap, and Lior Rotem. 2024. Accountability for Misbehavior in Threshold Decryption via Threshold Traitor Tracing. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII (Lecture Notes in Computer Science, Vol. 14926)*, Springer, 317–351. https://doi.org/10.1007/978-3-031-68394-7_11
- [16] Dan Boneh, Aditi Partap, and Lior Rotem. 2024. Traceable Secret Sharing: Strong Security and Efficient Constructions. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part V (Lecture Notes in Computer Science, Vol. 14924)*, Springer, 221–256. https://doi.org/10.1007/978-3-031-68388-6_9
- [17] Dan Boneh and Hovav Shacham. 2004. Group Signatures With Verifier-Local Revocation. In *ACM CCS 2004: 11th Conference on Computer and Communications Security*, Vijayalakshmi Atluri, Birgit Pfizmann, and Patrick McDaniel (Eds.). ACM Press, Washington, DC, USA, 168–177. <https://doi.org/10.1145/1030083.1030106>
- [18] Dan Boneh and James Shaw. 1995. Collusion-Secure Fingerprinting for Digital Data (Extended Abstract). In *Advances in Cryptology - CRYPTO'95 (Lecture Notes in Computer Science, Vol. 963)*, Don Coppersmith (Ed.). Santa Barbara, CA, USA, 452–465. https://doi.org/10.1007/3-540-44750-4_36
- [19] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. 2024. BEAT-MEV: Epochless Approach to Batched Threshold Encryption for MEV Prevention. *Cryptology ePrint Archive*, Paper 2024/1533. <https://eprint.iacr.org/2024/1533>
- [20] Xavier Boyen. 2008. The uber-assumption family. In *International Conference on Pairing-Based Cryptography (Lecture Notes in Computer Science, Vol. 5209)*, Springer, 39–56. https://doi.org/10.1007/978-3-540-85538-5_3
- [21] Zvika Brakerski, Yael Tauman Kalai, Jonathan Katz, and Vinod Vaikuntanathan. 2010. Overcoming the Hole in the Bucket: Public-Key Cryptography Resilient to Continual Memory Leakage. In *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, 501–510. <https://doi.org/10.1109/FOCS.2010.55>
- [22] Ran Canetti, Shai Halevi, and Jonathan Katz. 2003. A forward-secure public-key encryption scheme. In *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings* 22, 255–271. https://doi.org/10.1007/3-540-39200-9_16
- [23] Ran Canetti, Shai Halevi, and Jonathan Katz. 2003. A forward-secure public-key encryption scheme. In *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings* 22, Springer, 255–271.
- [24] Jie Chen, Hoon Wei Lim, San Ling, Huaxiong Wang, and Hoeteck Wee. 2012. Shorter IBE and Signatures via Asymmetric Pairings. In *Pairing-Based Cryptography - Pairing 2012 - 5th International Conference, Cologne, Germany, May 16-18, 2012, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 7708)*, Michel Abdalla and Tanja Lange (Eds.). Springer, 122–140. https://doi.org/10.1007/978-3-642-36334-4_8
- [25] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. 2024. Mempool Privacy via Batched Threshold Encryption: Attacks and Defenses. *Cryptology ePrint Archive*, Paper 2024/669. <https://eprint.iacr.org/2024/669>
- [26] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. 2024. Practical Mempool Privacy via One-time Setup Batched Threshold Encryption. *Cryptology ePrint Archive*, Paper 2024/1516. <https://eprint.iacr.org/2024/1516>
- [27] Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. 1994. Proofs of partial knowledge and simplified design of witness hiding protocols. In *Annual International Cryptology Conference*, 174–187. https://doi.org/10.1007/3-540-48658-5_19
- [28] Ronald Cramer, Rosario Gennaro, and Berry Schoenmakers. 1997. A Secure and Optimally Efficient Multi-Authority Election Scheme. In *Advances in Cryptology - EUROCRYPT '97, International Conference on the Theory and Application of Cryptographic Techniques, Konstanz, Germany, May 11-15, 1997, Proceeding (Lecture Notes in Computer Science, Vol. 1233)*, Walter Fumy (Ed.). Springer, 103–118. https://doi.org/10.1007/3-540-69053-0_9
- [29] Ronald Cramer and Victor Shoup. 1998. A Practical Public Key Cryptosystem Provably Secure Against Adaptive Chosen Ciphertext Attack. In *Advances in Cryptology - CRYPTO '98, 18th Annual International Cryptology Conference, Santa Barbara, California, USA, August 23-27, 1998, Proceedings (Lecture Notes in Computer Science, Vol. 1462)*, Hugo Krawczyk (Ed.). Springer, 13–25. <https://doi.org/10.1007/BFB0055717>
- [30] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2019. Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. *arXiv:1904.05234 [cs.CR]* <https://arxiv.org/abs/1904.05234>
- [31] Yevgeniy Dodis and Nelly Fazio. 2003. Public Key Trace and Revoke Scheme Secure against Adaptive Chosen Ciphertext Attack. In *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings (Lecture Notes in Computer Science, Vol. 2567)*, 100–115. https://doi.org/10.1007/3-540-36288-6_8
- [32] Stefan Dziembowski, Sebastian Faust, Tomasz Lazurek, and Marcin Mielniczuk. 2024. Secret Sharing with Snitching. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024*, 840–853. <https://doi.org/10.1145/3658644.3690296>
- [33] Stefan Dziembowski, Sebastian Faust, and Jannik Luhn. 2024. Shutter Network: Private Transactions from Threshold Cryptography. *Cryptology ePrint Archive* (2024).
- [34] Shayan Eskandari, Seyedehmahsa Moosavi, and Jeremy Clark. 2020. Sok: Transparent dishonesty: front-running attacks on blockchain. In *Financial Cryptography and Data Security: FC 2019*, 170–189. https://doi.org/10.1007/978-3-030-43725-1_13
- [35] Eiichiro Fujisaki and Tatsuaki Okamoto. 1999. Secure Integration of Asymmetric and Symmetric Encryption Schemes. In *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings (Lecture Notes in Computer Science, Vol. 1666)*, Springer, 537–554. https://doi.org/10.1007/3-540-48405-1_34
- [36] Eiichiro Fujisaki and Tatsuaki Okamoto. 2013. Secure Integration of Asymmetric and Symmetric Encryption Schemes. *J. Cryptol.* 26, 1 (2013), 80–101. <https://doi.org/10.1007/S00145-011-9114-1>
- [37] David Galindo and Ichiro Hasuo. 2005. Security notions for identity based encryption. *Cryptology ePrint Archive* (2005).
- [38] Craig Gentry and Alice Silverberg. 2002. Hierarchical ID-Based Cryptography. In *Advances in Cryptology - ASIACRYPT 2002, 8th International Conference on the Theory and Application of Cryptology and Information Security, Queenstown, New Zealand, December 1-5, 2002, Proceedings (Lecture Notes in Computer Science, Vol. 2501)*, Yuliang Zheng (Ed.). Springer, 548–566. https://doi.org/10.1007/3-540-36178-2_34
- [39] Vipul Goyal, Yifan Song, and Akshayaram Srinivasan. 2021. Traceable Secret Sharing and Applications. In *Advances in Cryptology - CRYPTO 2021 (Lecture Notes in Computer Science, Vol. 12827)*, Springer, 718–747. https://doi.org/10.1007/978-3-030-84252-9_24
- [40] Fuchun Guo, Yi Mu, and Willy Susilo. 2012. Identity-Based Traitor Tracing with Short Private Key and Short Ciphertext. In *Computer Security - ESORICS 2012 (Lecture Notes in Computer Science, Vol. 7459)*, Springer, 609–626. https://doi.org/10.1007/978-3-642-33167-1_35

- [41] Jeremy Horwitz and Ben Lynn. 2002. Toward Hierarchical Identity-Based Encryption. In *Advances in Cryptology - EUROCRYPT 2002 (Lecture Notes in Computer Science, Vol. 2332)*. Springer, 466–481. https://doi.org/10.1007/3-540-46035-7_31
- [42] Antoine Joux. 2004. A One Round Protocol for Tripartite Diffie-Hellman. *J. Cryptol.* 17, 4 (2004), 263–276. <https://doi.org/10.1007/S00145-004-0312-Y>
- [43] Charanjit S. Jutla and Arnab Roy. 2013. Shorter Quasi-Adaptive NIZK Proofs for Linear Subspaces. In *Advances in Cryptology - ASIACRYPT 2013 - 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1-5, 2013, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 8269)*. Kazuo Sako and Palash Sarkar (Eds.). Springer, 1–20. https://doi.org/10.1007/978-3-642-42033-7_1
- [44] Sarasij Maitra and David J. Wu. 2022. Traceable PRFs: Full Collusion Resistance and Active Security. In *Public-Key Cryptography - PKC 2022 - 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, (Lecture Notes in Computer Science, Vol. 13177)*. Springer, 439–469. https://doi.org/10.1007/978-3-030-97121-2_16
- [45] Moni Naor and Moti Yung. 1990. Public-key Cryptosystems Provably Secure against Chosen Ciphertext Attacks. In *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing, May 13-17, 1990, Baltimore, Maryland, USA, Harriet Ortiz (Ed.)*. ACM, 427–437. <https://doi.org/10.1145/100216.100273>
- [46] Duong Hieu Phan and Viet Cuong Trinh. 2011. Identity-Based Trace and Revoke Schemes. In *Provable Security - 5th International Conference, ProvSec 2011, Xi'an, China, October 16-18, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6980)*. Springer, 204–221. https://doi.org/10.1007/978-3-642-24316-5_15
- [47] Amit Sahai and Brent Waters. 2005. Fuzzy Identity-Based Encryption. In *Advances in Cryptology - EUROCRYPT 2005, 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings (Lecture Notes in Computer Science, Vol. 3494)*, Ronald Cramer (Ed.). Springer, 457–473. https://doi.org/10.1007/11426639_27
- [48] Kazuo Sako. 2000. An Auction Protocol Which Hides Bids of Losers. In *Public Key Cryptography, Third International Workshop on Practice and Theory in Public Key Cryptography, PKC 2000, Melbourne, Victoria, Australia, January 18-20, 2000, Proceedings (Lecture Notes in Computer Science, Vol. 1751)*, Hideki Imai and Yuliang Zheng (Eds.). Springer, 422–432. https://doi.org/10.1007/978-3-540-46588-1_28
- [49] Victor Shoup. 1997. Lower bounds for discrete logarithms and related problems. In *Advances in Cryptology—EUROCRYPT’97: International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, May 11–15, 1997 Proceedings 16*. Springer, 256–266.
- [50] Shutter Network. 2021. Introducing Shutter – Combating Front Running and Malicious MEV Using Threshold Cryptography. <https://blog.shutter.network/introducing-shutter-network-combating-front-running-and-malicious-mev-using-threshold-cryptography/>. Accessed: 2025-01-05.
- [51] Gábor Tardos. 2008. Optimal probabilistic fingerprint codes. *J. ACM* 55, 2 (2008), 10:1–10:24. <https://doi.org/10.1145/1346330.1346335>
- [52] Mark Zhandry. 2020. New Techniques for Traitor Tracing: Size $N^{1/3}$ and More from Pairings. In *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020 (Lecture Notes in Computer Science, Vol. 12170)*. 652–682. https://doi.org/10.1007/978-3-030-56784-2_22
- [53] Mark Zhandry. 2021. White Box Traitor Tracing. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part IV (Lecture Notes in Computer Science, Vol. 12828)*. Springer, 303–333. https://doi.org/10.1007/978-3-030-84259-8_11
- [54] Liyi Zhou, Kaihua Qin, Antoine Cully, Benjamin Livshits, and Arthur Gervais. 2021. On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi Protocols. IEEE Computer Society Press, 919–936. <https://doi.org/10.1109/SP40001.2021.00113>
- [55] Liyi Zhou, Kaihua Qin, Christof Ferreira Torres, Duc Viet Le, and Arthur Gervais. 2021. High-Frequency Trading on Decentralized On-Chain Exchanges. IEEE Computer Society Press, 428–445. <https://doi.org/10.1109/SP40001.2021.00027>

A CCA ATTACKS ON BPR24 CONSTRUCTIONS

We briefly discuss the constructions of [15], and present a CCA attack against them. We stress again that the CCA attack is practically relevant to applications such as mempool privacy, since the committee that decrypts transactions basically acts as a decryption oracle that returns partial decryptions. The CCA attack is on the constructions by [15] is the standard attack on homomorphic encryption schemes. For their second construction, the homomorphism is broken by hashing the result of the decryption and using that as key. The attack, however, can still be carried out given the partial decryptions.

First Construction. The first construction of [15] is reminiscent of ElGamal encryption. The j^{th} public key is a triple $X_j = g^{x_j}$, $Y_j = g^{y_j}$, $Z_j = g^{z_j}$. The left and right ciphertexts are Y_j^r and Z_j^r respectively for a randomly sampled r , and the key k is X_j^r . The CCA adversary, on input the challenge ciphertext (Y_j^r, Z_j^r) , asks for decryption of $(\hat{C}_l, \hat{C}_r) = \left((Y_j^r)^2, (Z_j^r)^2 \right)$. The corresponding key would be $\hat{k} = X_j^{2r}$ (can be derived from decryption shares received from the decryption oracle). Now, the the actual key k can be computed by computing $\hat{k}^{1/2}$.

Second Construction. In the second construction, the key is derived by evaluating a hash function H_2 on the result W of the combine-partial-decryptions. However, the adversary still learns the partial decryptions, hence it can derive W which exhibits homomorphic structure. More concretely, in their second construction, each component of the ciphertext is a pair of group elements $C_\ell = (u_\ell, v_\ell) \in \mathbb{G}_1 \times \mathbb{G}_2$, and $C_r = (u_r, v_r) \in \mathbb{G}_1 \times \mathbb{G}_2$. Moreover, for both $b \in \{\ell, r\}$, if $d_{(i,b)}$ is the output of the partial decryption for the i^{th} share of C_b , then it holds that $d_{(i,b)}^2$ is the output of the partial decryption for the i^{th} share of $C_b^2 = (u_b^2, v_b^2)$.

Thus, given the challenge ciphertext (C_ℓ, C_r) the adversary can submit (C_ℓ^2, C_r^2) for the partial decryption. As the submitted ciphertext is different from the challenge ciphertext, the adversary can make the query for t (threshold) number of shares, and get d_i^2 and thus derive d_i for t many parties. The adversary combines t many d_i s to compute W , and therefore derive $K = H_2(W)$.

B ADDITIONAL DEFINITIONS

B.1 Cryptographic Building Blocks

Definition B.1 (One-Time Signature Schemes). A one-time signature scheme Σ is a tuple of algorithms $\Sigma = (\text{KGen}, \text{Sign}, \text{Verify})$ such that for all messages m and all $(\text{sk}, \text{vk}) \leftarrow \text{KGen}(1^\lambda)$ it holds that

$$\text{Verify}(\text{vk}, m, \text{Sign}(\text{sk}, m)) = 1.$$

Further, we require Σ to achieve EUF-OT-CMA-security.

Game-EUF-OT-CMA $_{\mathcal{A}}^{\Sigma}(1^\lambda)$	$\mathcal{O}^{\text{Sign}}(m)$
$M \leftarrow \emptyset$	if $q = 0$ then return $\perp$
$q \leftarrow 1$	$q \leftarrow q - 1$
$(\text{sk}, \text{vk}) \leftarrow \text{KGen}(1^\lambda)$	$M \leftarrow M \cup \{m\}$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Sign}}}(1^\lambda, \text{vk})$	$\sigma \leftarrow \text{Sign}(\text{sk}, m)$
if $m^* \in M$ then return 0	return σ
return $\text{Verify}(\text{vk}, m^*, \sigma^*)$	

Figure 11: EUF-OT-CMA security game for signature schemes.

Definition B.2 (Existential Unforgeability under Chosen Message Attacks for One-Time Signatures (EUF-OT-CMA)). We say that a one-time signature scheme Σ is EUF-OT-CMA-secure if for all PPT adversaries $\mathcal{A}$ there exists a negligible function negl such that

$$\Pr \left[\text{Game-EUF-OT-CMA}_{\mathcal{A}}^{\Sigma}(1^\lambda) = 1 \right] \leq \text{negl}(\lambda),$$

where Game-EUF-OT-CMA is as defined in Figure 11.

Definition B.3 (Non-Interactive Zero-Knowledge Proofs of Knowledge). Let R be an NP-relation and of statement-witness pairs (χ, ω) . A non-interactive zero-knowledge proof of knowledge (NIZK-PoK) system Π for relation R is a tuple of algorithms $\Pi = (\text{Setup}, \text{Prove}, \text{Verify})$ with the following properties:

- Correctness: For all $(\chi, \omega) \in R$ it holds that $\text{Verify}(\chi, \text{Prove}(\chi, \omega)) = 1$.
- Knowledge-soundness: There exists a PPT extraction algorithm Ext such that for all PPT adversaries $\mathcal{A}$ there exists a negligible function negl with

$$\Pr \left[\begin{array}{l} (\chi, \omega) \notin R \wedge \\ \text{Verify}(\chi, \pi) = 1 \end{array} : \begin{array}{l} (\chi, \pi) \leftarrow \mathcal{A}(1^\lambda) \\ \omega \leftarrow \text{Ext}^{\mathcal{A}(\cdot)}(\chi, \pi) \end{array} \right] \leq \text{negl}(\lambda).$$

- Zero-knowledge: There exists a PPT simulator Sim such that for all $(\chi, \omega) \in R$ it holds that

$$(\text{CRS}, \chi, \text{Prove}(\chi, \omega)) \approx_c (\text{CRS}, \chi, \text{Sim}(\chi)).$$

All algorithms receive a common CRS $\leftarrow \text{Setup}(1^\lambda)$ as implicit input.

B.2 Well-Established Bilinear Pairing Assumptions

The Bilinear (Decisional) Diffie-Hellman (BDDH) assumption was introduced in [12] and since been the bedrock of Identity-based cryptography and its extensions, and, in general, pairing-based cryptography ([23, 38, 42, 47] and many more).

Definition B.4 (Bilinear Decisional Diffie-Hellman). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the bilinear decisional Diffie-Hellman (BDDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries $\mathcal{B}$, it holds that

$$\left| \Pr [\mathcal{B}(E, \vec{x}, e(g_1, g_2)^{abc}) = 1] - \Pr [\mathcal{B}(E, \vec{x}, e(g_1, g_2)^v) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, $a, b, c, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^a, g_1^b, g_2, g_2^a, g_2^b).$$

The XDH assumption was first discussed in [10], and formalized in [4]. Since then, it has been used in numerous constructions including Identity-Based Encryption [24], Inner-Product Encryption [6], Group Signatures [3], Leakage-Resilient Encryption [21], and Non-Interactive Zero-knowledge proofs [43].

Definition B.5 (XDH Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the external Diffie-Hellman (XDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries $\hat{\mathcal{B}}$ it holds that

$$\left| \Pr [\mathcal{B}(E, \vec{x}, g_1^{ab}) = 1] - \Pr [\mathcal{B}(E, \vec{x}, g_1^v) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, and $a, b, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^a, g_1^b, g_2).$$

B.3 Traceable Threshold KEM

Definition B.6 (Correctness of TT-KEM). A TT-KEM protocol Π is correct if for all $\lambda \in \mathbb{N}$, all polynomials $\varepsilon(\lambda)$, all $n \geq t > 0$, all $S \subseteq [n]$ with $|S| \geq t$ it holds that

$$\Pr \left[\begin{array}{l} k = k' \wedge \\ \text{Verify}(c, \pi) = 1 \end{array} : \begin{array}{l} (\text{pk}, \{\text{sk}_i\}_{i \in [n]}, \text{tk}) \\ \leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\varepsilon(\lambda)}) \\ (k, c, \pi) \leftarrow \text{Enc}(\text{pk}) \\ \forall i \in S: d_i \leftarrow \text{PDec}(\text{sk}_i, c) \\ k' \leftarrow \text{Comb}(S, \{d_i\}_{i \in S, c}) \end{array} \right] = 1.$$

Further, we require that the keys generated by Enc are uniformly distributed over the key-space $\mathcal{K}(\lambda)$.

Game-IND-CCA $_{\mathcal{A}}^{\text{TT-KEM}}(1^\lambda)$	$\mathcal{O}^{\text{PDec}}((c, \pi), i)$
$b \leftarrow \{0, 1\}$	if $\text{Verify}(c, \pi) = 0$ then
$(\text{pk}, \{\text{sk}_i\}_{i \in [n]}, \text{tk})$	return $\perp$
$\leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\varepsilon(\lambda)})$	if $c = c^*$ then
$(C, \text{st}) \leftarrow \mathcal{A}(\text{st}_1, \text{pk}, \text{tk})$	return $\perp$
if $C \not\subseteq [n] \vee C \geq t$ then	$d_i \leftarrow \text{PDec}(c, \text{sk}_{i,0})$
return 0	return d_i
$k_0 \leftarrow \mathcal{K}(\lambda)$	
$(k_1, c^*, \pi^*) \leftarrow \text{Enc}(\text{pk})$	
$\text{chal} \leftarrow ((c^*, \pi^*), k_b, \{\text{sk}_i\}_C)$	
$b' \leftarrow \mathcal{A}_3^{\mathcal{O}^{\text{PDec}}}(\text{st}, \text{chal})$	
return $b \stackrel{?}{=} b'$	

Figure 12: IND-CCA security game for TT-KEM. The difference from the standard threshold KEM is that the adversary learns the tracing key. This models the fact that the tracer is not trusted for CCA-security.

The most important detail about IND-CCA security of TT-KEM is that the adversary gets access to the tracing key. This reflects the fact that the tracer is not trusted for security. Even if the tracer is corrupt, full CCA security holds as long as less than t shareholders are corrupt.

Definition B.7 (IND-CCA-security of TT-KEM). A TT-KEM protocol π is IND-CCA-secure if for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ there exists a negligible function negl such that

$$\Pr [\text{Game-IND-CCA}_{\mathcal{A}}^{\text{TT-KEM}}(1^\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $\text{Game-IND-CCA}_{\mathcal{A}}^{\text{TT-KEM}}$ is defined in Figure 12.

Definition B.8 ((ε, δ) -Traceability of TT-KEM). Let $\varepsilon(\lambda)$ and $\delta(\lambda)$ be functions in the security parameter. A TT-KEM scheme is (ε, δ) -traceable, if for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ and for all $\lambda \in \mathbb{N}$ the following conditions hold

$$\Pr[\text{GoodTr}] \geq \Pr[\text{GoodDec}] - \delta(\lambda) \quad \text{and} \quad \Pr[\text{BadTr}] \leq \delta(\lambda),$$

where $(\text{pk}, D, \mathcal{J}, \mathcal{J}') \leftarrow \text{Game-Trace}_{\mathcal{A}, \varepsilon}(1^\lambda)$ (Figure 13) and the three events GoodTr, BadTr and GoodDec are defined as follows:

- GoodTr holds when $\mathcal{J}' \neq \emptyset$ and $\mathcal{J}' \subseteq \mathcal{J}$, which indicates that at least one real traitor has been traced and no non-traitors were framed by the tracing algorithm.
- BadTr holds when at least one non-traitor was framed by the tracing algorithm ($\mathcal{J}' \neq \emptyset$ and $\mathcal{J}' \not\subseteq \mathcal{J}$).

Game-Trace$_{\mathcal{A},\epsilon}(1^\lambda)$ $(n, t, st_1) \leftarrow \mathcal{A}_1(1^\lambda)$ $(pk, \{sk_i\}_{i \in [n]}, tk) \leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\epsilon(\lambda)})$ $(\mathcal{J}, st_2) \leftarrow \mathcal{A}_2(st_1, pk)$ if $\mathcal{J} \not\subseteq [n]$ then abort $D \leftarrow \mathcal{A}_3(st_2, \{sk_i\}_{i \in C})$ $\mathcal{J}' \leftarrow \text{Trace}^{D(\cdot)}(pk, tk, 1^{1/\epsilon(\lambda)})$ return $(pk, D, \mathcal{J}, \mathcal{J}')$

Figure 13: Traceability game for TT-KEM.

- GoodDec indicates that the decoder is good for pk. More formally, when $P(D) \geq 1/2 + \epsilon(\lambda)$ where

$$P(D) := \Pr \left[D(c, k_b) = b : \begin{array}{l} ((c, \tau), k_0) \leftarrow \text{Enc}(pk) \\ k_1 \leftarrow \mathcal{K}_\lambda, b \leftarrow \{0, 1\} \end{array} \right].$$

Definition B.9 (Traceability of TT-KEM). A TT-KEM scheme is traceable if there exists a negligible function ν such that TT-KEM is $(1/p, \nu)$ -traceable for every positive polynomial $p(\lambda)$.

C TTIB-KEM FROM BTIB-KEM

In this section, we extend the TT-KEM definition to the identity-based setting, providing the definition of traceable threshold identity-based KEM (TTIB-KEM). We then discuss a transformation from our main building block BTIB-KEM to TTIB-KEM.

C.1 Traceable Threshold Identity-Based KEM

We next present our definition of TTIB-KEM, which extends the standard identity-based KEM by a tracing functionality.

We model traceability for threshold identity-based KEM schemes by lifting the definition from [15] to the identity-based setting. Our model accounts for decoders that can decrypt ciphertexts of a chosen subset of identities (the set I^* in Figure 14), that is determined in the beginning of the scheme.

Definition C.1. A traceable threshold identity-based KEM scheme (TTIB-KEM) is a tuple of protocols (KGen, Enc, DIdk, ComblDk, Dec, Trace) with KEM key-space $\mathcal{K}(\lambda)$ and identity-space IDSpace.

- $(pk, \{sk_i\}_{i \in [n]}, tk) \leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\epsilon(\lambda)})$. The KGen algorithm receives the security parameter 1^λ , the number of parties n , the threshold t where $n \geq t > 0$ and the tracing parameter ϵ , which is a lower bound on the distinguishing advantage of decoders that must be traceable by this scheme. It outputs a public key pk , the secret key shares $sk = \{sk_i\}_{i \in [n]}$, and a tracing key tk .
- $(k, c) \leftarrow \text{Enc}(pk, ID)$. Given a public key pk and an identity $ID \in \text{IDSpace}$, the encryption algorithm returns a key $k \in \mathcal{K}(\lambda)$ and a ciphertext c .
- $\text{idk}_i \leftarrow \text{DIdk}(sk_i, ID)$. The identity key derivation algorithm DIdk receives the secret key share of party i and an identity ID . It returns a partial identity key on ID as idk_i .
- $\text{idk} \leftarrow \text{ComblDk}(S, \{\text{idk}_i\}_{i \in S})$. The ComblDk algorithm aggregates partial identity keys from a set S of size at least t and outputs the resulting identity key idk .

- $k/\perp \leftarrow \text{Dec}(\text{idk}, c)$. The decryption algorithm receives an identity key idk and a ciphertext c . It returns the resulting key k or $\perp$ upon failure.
- $\mathcal{J} \leftarrow \text{Trace}^{D(\cdot)}(pk, tk, I, 1^{1/\epsilon(\lambda)})$. The Trace algorithm is given the public key pk , the tracing key tk , a set of possible identities I , the parameter ϵ , and black-box access to the decoder D . It outputs a set of traitors $\mathcal{J} \subseteq [n]$.

We omit the correctness and IND-SID-CPA definitions as they are standard and not affected by traceability.

Game-Trace-ID$_{\mathcal{A},\epsilon}(1^\lambda)$ $\mathcal{J} \leftarrow \emptyset$ $(n, t, I^*, st_1) \leftarrow \mathcal{A}_1(1^\lambda)$ if $I^* = \emptyset$ then $I^* \leftarrow \text{IDSpace}$ $(pk, \{sk_i\}_{i \in [n]}, tk) \leftarrow \text{KGen}(1^\lambda, n, t, 1^{1/\epsilon(\lambda)})$ $(C, st_2) \leftarrow \mathcal{A}_2(st_1, pk)$ $D \leftarrow \mathcal{A}_3(st_2, \{sk_i\}_{i \in C})$ $\mathcal{J}' \leftarrow \text{Trace}^{D(\cdot)}(pk, tk, I^*, 1^{1/\epsilon(\lambda)})$ return $(pk, I^*, D, \mathcal{J}, \mathcal{J}')$
--

Figure 14: Traceability game for TTIB-KEM.

Definition C.2. $[(\epsilon, \delta)$ -Traceability of TTIB-KEM] For functions in the security parameter $\epsilon(\lambda)$ and $\delta(\lambda)$, a TTIB-KEM scheme is (ϵ, δ) -traceable, if for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ and for all $\lambda \in \mathbb{N}$ the following conditions hold

$$\Pr[\text{GoodTr}] \geq \Pr[\text{GoodDec}] - \delta(\lambda) \quad \text{and} \quad \Pr[\text{BadTr}] \leq \delta(\lambda)$$

with $(pk, I^*, D, \mathcal{J}, \mathcal{J}') \leftarrow \text{Game-Trace-ID}_{\mathcal{A},\epsilon}(1^\lambda)$ (Figure 14) and the three events GoodTr, BadTr and GoodDec defined as follows:

- GoodTr holds when $\mathcal{J}' \neq \emptyset$ and $\mathcal{J}' \subseteq \mathcal{J}$, which indicates that at least one real traitor has been traced and no non-traitors were framed by the tracing algorithm.
- BadTr holds when at least one non-traitor was framed by the tracing algorithm ($\mathcal{J}' \neq \emptyset$ and $\mathcal{J}' \not\subseteq \mathcal{J}$).
- GoodDec indicates that the decoder is good for pk and I^* . More formally, when $P(D) \geq 1/2 + \epsilon(\lambda)$ where

$$P(D) := \Pr \left[D(c, k_b) = b : \begin{array}{l} ID \leftarrow I^*, (c, k_0) \leftarrow \text{Enc}(pk, ID) \\ k_1 \leftarrow \mathcal{K}_\lambda, b \leftarrow \{0, 1\} \end{array} \right].$$

Definition C.3 (Traceability of TTIB-KEM). A TTIB-KEM scheme is traceable if there exist a negligible function ν such that TTIB-KEM is $(1/p, \nu)$ -traceable for every positive polynomial $p(\lambda)$.

Differences from the definition of [1]. Abdalla et al. [1] define traitor tracing in the identity-based setting. We note that their definition considers a different scenario than ours. In particular, in [1] they consider group-based applications of IBE, e.g., email addresses groups, in which all users of the group can decrypt using a decryption key(s) that associated for the group. Note that the decryption keys for every group are constructed and distributed by a trusted authority who holds the master key. In their model, they want to defend against collisions in which users of the group,

using their decryption keys, generate a pirate decoder that is able to decrypt ciphertexts of this specific group. In the tracing definition, the adversary chooses a group ID and the tracing algorithm returns a set of users within group ID that constructed the decoder. In our setting, on the other hand, we consider a scenario where the master key is shared among shareholders, who may collude in order to construct a pirate decoder that is potentially able to decrypt ciphertexts for any identity. Specifically, our definition enables us to trace decoders that are able to decrypt ciphertexts for an identity ID even if the identity key was not derived previously (this is modeled as the set $\mathcal{I}^*$). Observe that this is not captured by the definition in [1] since future identity keys can be derived only using the master key, which is held only by the trusted authority. We emphasize that the possibility to trace decryption for future identity keys is well motivated due to the use case of IBE in threshold mempool encryption, e.g., in [33]. In this application, the identities correspond to epochs and the ability of the adversary to derive future identity keys gains it an advantage in decrypting ciphertexts for future epochs, which allows him to frontrun transactions. As we are motivated mainly by the mempool-privacy application, our traceability definition aims to identify those shareholders, that collude to construct such a decoder.

Remark On identity key derivation oracle. An observant reader may have noted that our traceability game in Figure 14 does not offer an identity key derivation oracle. However, we note that in this game, we consider a stronger adversary who receives enough key shares to derive an identity key for any identity. Indeed, this holds since the adversary receives secret shares for all corrupted parties in C , where $|C| \geq t$. On the other hand, one would consider an extension of our traceability game in which the adversary is offered a *partial identity key derivation oracle*, such that the adversary can choose to receive the partial identity key share from any honest party. This extended traceability notion gives the ability to prove CCA-secure and traceable TT-KEM and TTIB-KEM in the presence of a traceability adversary who has access to partial decryptions. This is interesting, in particular, in encrypted mempool, as it reflects the real setting where the partial decryption shares of the committee are exposed to everyone. That said, we would like to stress that our traceability game is a reasonable model, where we assume that the adversary that attempts to break traceability has access only to the final decryption result (and not the partial decryptions). This reflects a setting where the partial decryption shares are sent to a trusted combiner who aggregates all of them (then the trusted combiner might send a ZK proof for correct aggregation).

We note that currently, it is unclear to us how to simulate the partial identity key oracle in the traceability proof and is left as an interesting open problem.

Remark on selective identity. We highlight the fact that the adversary commits in the beginning to $\mathcal{I}^*$ because it somewhat reflects the selective-identity notion that exists for security of IBE schemes in the context of IND-CPA and IND-CCA [11, 22]. For security, it is well established that the normal notions imply the selective-identity notions, but not the other way around [37]. We note that we would be able to achieve traceability without selective-identity using a stronger one-sided security notion for BTIB-KEM where the adversary can decide on the challenge identity *after* learning

the public key and secret key shares. While our first construction can achieve this stronger notion, our second construction cannot, hence we restrict ourselves to this version of traceability.

C.2 BTIB-KEM to TTIB-KEM Transformation

Next, we present a black-box construction Π for TTIB-KEM from fingerprinting codes and BTIB-KEM. It is very similar to the transformation of [15], but in the identity-based setting. We give the full construction in Figure 15. The construction is black-box with respect to a BTIB-KEM scheme $\Pi^{\text{BTIB-KEM}}$ and a fingerprinting code $\mathcal{F} = (\text{FCGen}, \text{FCTrace})$ as building blocks.

As a direct corollary from the traceability proof of TT-KEM as presented in [15] (which is discussed in Appendix D.3), we achieve the following theorem.

THEOREM C.4. *The TTIB-KEM construction Π is traceable (Definition C.2), if the underlying BTIB-KEM protocol $\Pi^{\text{BTIB-KEM}}$ is one-sided secure (Definition 4.4) and the fingerprinting code $\mathcal{F}$ is fully collusion-resistant.*

D SECURITY PROOFS OF THE TT-KEM CONSTRUCTION

D.1 CCA-security of BT-KEM

PROOF OF LEMMA 4.5. Let Game_1 be the IND-CCA game as introduced in Figure 2. We introduce Game_2 by making a modification to the partial decryption oracle $\mathcal{O}^{\text{PDec}}$. In particular, we add an additional abort condition that causes the oracle to return $\perp$. Given the queried ciphertext $c := ((c_0, c_1), j, \text{vk}_e)$ and the challenge ciphertext $c^* = ((c_0^*, c_1^*), j^*, \text{vk}_e^*)$, the oracle $\mathcal{O}_2^{\text{PDec}}$ aborts, if $\text{vk}_e = \text{vk}_e^*$ but $c \neq c^*$. The modification is laid out in Figure 16.

LEMMA D.1. *If Σ is EUF-OT-CMA-secure following Definition B.2, then Game_1 and Game_2 are computationally indistinguishable.*

PROOF OF LEMMA D.1. We prove lemma D.1 by reducing a successful PPT distinguisher to the unforgeability of Σ . Let $\mathcal{A}$ be a PPT adversary such that

$$\left| \Pr[\text{Game}_{1,\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] - \Pr[\text{Game}_{2,\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] \right| \geq \varepsilon(\lambda)$$

for a non-negligible ε . We construct a PPT reduction $\mathcal{B}$ that runs $\mathcal{A}$ internally to win the EUF-OT-CMA game (Figure 11) of Σ with the same probability ε .

Initially, $\mathcal{B}$ receives a challenge-verification key vk^* from the EUF-OT-CMA game. Then, it simulates Game-IND-CCA to $\mathcal{A}$, embedding vk^* as the vk_e^* of the challenge ciphertext. It computes σ^* by querying the signing oracle of the EUF-OT-CMA game on c^* . When $\mathcal{B}$ receives a partial decryption query on $((c, \sigma), i)$ that triggers the additional abort condition of $\mathcal{O}_2^{\text{PDec}}$ (i.e., $\text{vk}_e = \text{vk}_e^*$ and $c \neq c^*$) and also passes the abort condition $\Sigma.\text{Verify}(\text{vk}_e, c, \sigma)$, then $\mathcal{B}$ outputs (c, σ) as its forgery in the EUF-OT-CMA game.

Analysis. Clearly, $\mathcal{A}$ can only distinguish between the two games by triggering the extra abort condition at least once while sending a valid (c, σ) . In this case, it holds that $\text{vk}_e = \text{vk}_e^*$, $c \neq c^*$ and $\Sigma.\text{Verify}(\text{vk}_e^*, c, \sigma) = 1$, which means that $\mathcal{B}$'s output (c, σ) is a valid forgery, as the message c is new (i.e. not equal to the only signing query c^*) and the signature σ is valid on c . Also, note that $\mathcal{B}$ queries

$\text{KGen}(1^\lambda, n, t, 1^{1/\varepsilon})$ $\delta \leftarrow \frac{1/2\varepsilon}{1/2 - 2/\sqrt{\lambda}}$ $(\Gamma, \ell, \text{tk}) \leftarrow \mathcal{F}.\text{FCGen}(1^n, 2^{-\lambda}, \delta)$ $(\text{pk}, \{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{i \in [n], j \in [\ell]}) \leftarrow \Pi^{\text{BTIB-KEM}}.\text{KGen}(1^\lambda, n, t, \ell)$ for $i \in [n]$ do $w^{(i)} \leftarrow \Gamma_i$ $\text{sk}_i \leftarrow \{\text{sk}_{i,w_j^{(i)}}^{(j)}\}_{j \in [\ell]}$ return $(\text{pk}, \{\text{sk}_i\}_{i \in [n]}, \text{tk})$ $\text{Enc}(\text{pk}, \text{ID})$ $j \leftarrow [\ell]$ $(k, (c_0, c_1)) \leftarrow \Pi^{\text{BTIB-KEM}}.\text{Enc}(\text{pk}, j, \text{ID})$ return $(k, c := (c_0, c_1, j))$ $\text{DIdk}(\text{sk}_i, \text{ID})$ $\text{idk}_i \leftarrow \Pi^{\text{BTIB-KEM}}.\text{DIdk}(\text{sk}_i, \text{ID})$ return idk_i	$\text{ComblDk}(S, \{\text{idk}_i\}_{i \in S})$ $\text{idk} \leftarrow \Pi^{\text{BTIB-KEM}}.\text{ComblDk}(S, \{\text{idk}_i\}_{i \in S})$ return idk $\text{Dec}(\text{idk}, c)$ Parse c as (c_0, c_1, j) $k \leftarrow \Pi^{\text{BTIB-KEM}}.\text{Dec}(\text{idk}, (c_0, c_1), j)$ return k $\text{Tr}^{D(\cdot)}(\text{pk}, j, N, \mathcal{I}, (b_k, b_0, b_1))$ $\text{ctr} \leftarrow 0$ for $r = 1, \dots, N$ do $\text{ID} \leftarrow \mathcal{I}$ $(k^{(0)}, c^{(0)}) := (c_0^{(0)}, c_1^{(0)}) \leftarrow \Pi^{\text{BTIB-KEM}}.\text{Enc}(\text{pk}, j, \text{ID})$ $(k^{(1)}, c^{(1)}) := (c_0^{(1)}, c_1^{(1)}) \leftarrow \Pi^{\text{BTIB-KEM}}.\text{Enc}(\text{pk}, j, \text{ID})$ $c^* \leftarrow (c_0^{(b_0)}, c_1^{(b_1)})$ if $D(c^*, k^{(b_k)}) = 1$ then $\text{ctr} \leftarrow \text{ctr} + 1$ return ctr	$\text{Trace}^{D(\cdot)}(\text{pk}, \text{tk}, \mathcal{I}, 1^{1/\varepsilon})$ $N \leftarrow \lambda^2$ $B \leftarrow \lambda^{3/2}$ for $j \in [\ell]$ do $p_{001} \leftarrow \text{Tr}^{D(\cdot)}(\text{pk}, j, N, \mathcal{I}, (b_k, b_0, b_1) := (0, 0, 1))$ $p_{100} \leftarrow \text{Tr}^{D(\cdot)}(\text{pk}, j, N, \mathcal{I}, (b_k, b_0, b_1) := (1, 0, 0))$ $p_{111} \leftarrow \text{Tr}^{D(\cdot)}(\text{pk}, j, N, \mathcal{I}, (b_k, b_0, b_1) := (1, 1, 1))$ $a_0 \leftarrow p_{001} - p_{100} $ $a_1 \leftarrow p_{001} - p_{111} $ if $a_0 \geq B$ then $w_j \leftarrow 0$ else if $a_1 \geq B$ then $w_j \leftarrow 1$ else $w_j \leftarrow '?'$ $\bar{w} \leftarrow w_1 w_2 \dots w_\ell$ $\mathcal{J} \leftarrow \mathcal{F}.\text{FCTrace}(\text{tk}, \bar{w})$ return $\mathcal{J}$
--	---	---

Figure 15: Our black-box TTIB-KEM construction given a BTIB-KEM protocol $\Pi^{\text{BTIB-KEM}}$ and a fingerprinting code $\mathcal{F}$.

Oracle $\mathcal{O}_1^{\text{PDec}}((c, \sigma), i)$ Parse c as $((c_0, c_1), j, \text{vk}_e)$ if $\Sigma.\text{Verify}(\text{vk}_e, c, \sigma) = 0$ then return $\perp$ if $c = c^*$ then return $\perp$ $d_{i,0} \leftarrow \text{PDec}(c, \text{sk}_{i,0})$ $d_{i,1} \leftarrow \text{PDec}(c, \text{sk}_{i,1})$ return $(d_{i,0}, d_{i,1})$	Oracle $\mathcal{O}_2^{\text{PDec}}((c, \sigma), i)$ Parse c as $((c_0, c_1), j, \text{vk}_e)$ if $\text{vk}_e = \text{vk}_e^* \wedge c \neq c^*$ then return $\perp$ if $\Sigma.\text{Verify}(\text{vk}_e, c, \sigma) = 0$ then return $\perp$ if $c = c^*$ then return $\perp$ $d_{i,0} \leftarrow \text{PDec}(c, \text{sk}_{i,0})$ $d_{i,1} \leftarrow \text{PDec}(c, \text{sk}_{i,1})$ return $(d_{i,0}, d_{i,1})$
--	--

Figure 16: Modified $\mathcal{O}^{\text{PDec}}$ oracle.

the signing oracle $\mathcal{O}^{\text{Sign}}$ once. Hence $\mathcal{B}$ breaks EUF-OT-CMA of Σ with $\mathcal{A}$'s distinguishing advantage

$$\Pr[\text{Game-EUF-OT-CMA}_{\mathcal{B}}^{\Sigma}(1^\lambda) = 1] \geq \left| \Pr[\text{Game}_{1,\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] - \Pr[\text{Game}_{2,\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] \right| \geq \varepsilon(\lambda)$$

which contradicts the assumption that Σ is EUF-OT-CMA secure. $\square$

We conclude our proof with a reduction from Game_2 to the selective ID semantic security of the underlying BTIB-KEM scheme $\mathcal{I}$.

Let $\mathcal{A}$ be a PPT adversary in Game_2 with $\Pr[\text{Game}_{2,\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda)$ for a non-negligible ε . We construct a PPT reduction $\mathcal{B}$ that wins Game-IND-SID-CPA of $\mathcal{I}$ with the same probability. $\mathcal{B}$ first runs $\mathcal{A}_1$ and receives the parameter ℓ as a result. Then, $\mathcal{B}$ generates a signing keypair $(\text{sk}_e^*, \text{vk}_e^*) \leftarrow \Sigma.\text{KGen}(1^\lambda)$ and returns vk_e^* as the selected challenge-identity $\mathcal{I}^*$ as well as ℓ to Game-IND-SID-CPA . In the next step, $\mathcal{B}$ receives the public key pk from Game-IND-SID-CPA , forwards it to $\mathcal{A}_2$ and sends the corrupted set C back to Game-IND-SID-CPA . Next, $\mathcal{B}$ receives a challenge ciphertext $c_{IBE}^* = (c_0^*, c_1^*)$, the challenge key k^* and the challenge index j^* as well as the corrupted secret key shares $\{\text{sk}_i\}_{i \in C}$. $\mathcal{B}$ converts this into a BT-KEM challenge ciphertext $c^* = ((c_0^*, c_1^*), j^*, \text{vk}_e^*)$ and computes $\sigma^* \leftarrow \text{GenTag}(c^*, \text{sk}_e^*)$. It forwards $(c^*, \sigma^*), k^*$ and $\{\text{sk}_i\}_{i \in C}$ to $\mathcal{A}_3$ and returns the same bit as $\mathcal{A}_3$.

Simulating $\mathcal{O}_2^{\text{PDec}}$. $\mathcal{B}$ simulates $\mathcal{O}_2^{\text{PDec}}$ by querying the identity-key derivation oracle $\mathcal{O}^{\text{DIdk}}$ of Game-IND-SID-CPA on the identity vk_e for the PDec step of $\mathcal{O}_2^{\text{PDec}}$. Since $\mathcal{O}_2^{\text{PDec}}$ aborts whenever $\text{vk}_e = \text{vk}_e^* = \mathcal{I}^*$, before even querying $\mathcal{O}^{\text{DIdk}}$ on $\mathcal{I}^*$, the reduction never queries $\mathcal{O}^{\text{DIdk}}$ on the challenge identity and the simulation is always successful.

Analysis. Let $\hat{b}$ be the internal bit of Game-IND-SID-CPA and b the internal bit of Game_2 . If $\hat{b} = 1$, then $\mathcal{B}$ clearly simulates Game_2 for $b = 1$ and vice versa. Consequently, $\mathcal{B}$'s success probability in Game-IND-SID-CPA is equal to $\mathcal{A}$'s success probability in Game_2 and we get

$$\Pr[\text{Game-IND-SID-CPA}_{\mathcal{B}}^{\mathcal{I}}(1^\lambda) = 1] = \Pr[\text{Game}_{2,\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda)$$

which is a contradiction to the IND-SID-CPA security of $\mathcal{I}$.

Overall, we can bound the advantage of any PPT adversary $\mathcal{A}$ in Game-IND-CCA of BT-KEM as follows.

$$\begin{aligned} & \text{Adv}_{\mathcal{A}, \text{Game-IND-CCA}}^{\text{BT-KEM}} \\ & \leq \text{Adv}_{\mathcal{B}, \text{Game-EUF-OT-CMA}}^{\Sigma} + \text{Adv}_{\mathcal{B}, \text{Game-IND-SID-CPA}}^{\mathcal{I}} \end{aligned}$$

Hence the advantage is negligible by our assumption that Σ is EUF-OT-CMA secure and $\mathcal{I}$ is IND-SID-CPA secure $\square$

D.2 One-Sided Security of BT-KEM

PROOF OF LEMMA 4.6. Let $\mathcal{A}$ be a successful adversary against Game-OSS of BT-KEM (Figure 3) such that

$$\Pr[\text{Game-OSS}_{\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda)$$

for a non-negligible ε . We construct a PPT reduction $\mathcal{B}$ to the Game-OSS-ID experiment (Figure 6) of the underlying BTIB-KEM scheme $\mathcal{I}$. First, the reduction runs $\mathcal{A}_1$ and receives ℓ, u and d . Then it obtains a signing key-pair $(\text{sk}_e, \text{vk}_e) \leftarrow \Sigma.\text{KGen}(1^\lambda)$, sets $\text{ID} \leftarrow \text{vk}_e$ and passes ℓ, u, d and ID to Game-OSS-ID. As a result, it receives the secret-key shares S , the public key pk , the challenge ciphertext $c^* = (c_0^*, c_1^*)$, and the challenge key k . $\mathcal{B}$ transforms this into a challenge ciphertext of Game-OSS, by setting $c_{\text{OSS}}^* := ((c_0^*, c_1^*), j, \text{vk}_e)$ and computing $\sigma^* \leftarrow \text{GenTag}(\text{sk}_e, c_{\text{OSS}}^*)$. It forwards $\text{pk}, (c_{\text{OSS}}^*, \sigma^*), k$, and S to $\mathcal{A}_2$ and returns the same bit as $\mathcal{A}_2$.

$\mathcal{B}$'s success probability in Game-OSS-ID is equal to $\mathcal{A}$'s success probability in Game-OSS, as $\mathcal{B}$ perfectly simulates Game-OSS for $b = 1$, whenever the internal bit of Game-OSS-ID is also 1 and vice versa. Hence,

$$\begin{aligned} & \Pr[\text{Game-OSS-ID}_{\mathcal{B}}^{\mathcal{I}}(1^\lambda) = 1] = \\ & \Pr[\text{Game-OSS}_{\mathcal{A}}^{\text{BT-KEM}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda) \end{aligned}$$

which is a contradiction to the one-sided security of $\mathcal{I}$. $\square$

D.3 Traceability Proof of TT-KEM

Here, we present in detail, how the traceability proof of [15] can be adapted to work for our modified TT-KEM transformation from our definition of CCA-secure BT-KEM. The crucial differences here are (a) the modifications to the one-sided security game Game-OSS of BT-KEM (see Figure 3) and (b) the modified Tr subroutine of the TT-KEM construction (see Figure 4).

Recall briefly, that the tracing works by running the $\text{Tr}^{D(\cdot)}$ subroutine for every index $j \in \ell$ with either valid ciphertexts or ciphertexts that are malformed in some way. Afterwards, for every j the results of the different Tr runs are compared to determine, whether the decoder contains left keys for j or right keys for j . The results are compiled into a noisy codeword $\bar{w}$, which is essentially just a string of length ℓ , where 0 encodes “left keys at positions j ”, 1 encodes right keys at position j , and ‘?’ encodes that the tracing algorithm cannot determine. Henceforth, the ‘?’ positions are called *noisy*. In the end, the codeword $\bar{w}$ is fed into the tracing algorithm of the fingerprinting code along with the tracing key tk to determine the set of traitors.

The proof in [15] can be separated into four phases. We next discuss these phases and explain why they are not affected by our modifications.

In the first phase they argue that the codeword $\bar{w}$ produced by the tracing algorithm for any *good* decoder can at most have a fraction of δ noisy positions with overwhelming probability¹². This is a necessary precondition for the tracing algorithm of the fingerprinting code to work. This lemma follows from purely probabilistic arguments that are derived from the definition of good decoders, hence they are entirely unaffected by our modifications.

In the second phase, they introduce an idealized tracing experiment, where they instead use an idealized tracing algorithm Trace' that depends on the *actual* set of traitors $\mathcal{J}$, i.e. they call Tr^D with different (b_k, b_0, b_1) dependent on whether all real traitors have left keys at position j . They argue that the idealized tracing algorithm outputs a codeword $\bar{w}'$ that is feasible for the set of traitors $\mathcal{J}$ with overwhelming probability^{13,14}. The arguments made to prove this lemma rely on the fact that $\text{Tr}^D((b_k, b_0, b_1) = (0, 1, 1))$ and $\text{Tr}^D((b_k, b_0, b_1) = (1, 0, 0))$ are identically distributed. Observe that this is equally the case for our modified Tr subroutine, hence the proof for this lemma is unaffected.

Now that we have the two facts that (1) $\bar{w}$ and $\bar{w}'$ are δ -noisy and (2) that $\bar{w}'$ is feasible for $\mathcal{J}$, it remains to show that the results of the real and the idealized tracing $\bar{w}$ and $\bar{w}'$ are computationally indistinguishably distributed. This is carried out in phases three and four through a reduction to one-sided security. Essentially, they argue that any adversary that can meaningfully distinguish between the two experiments, can be transformed to break the one-sided security game of BT-KEM.

First, the one-sided security game is extended to a more general version, which outputs N challenges for a common bit b instead of just one challenge. This is carried out through a standard hybrid argument, which can be applied to our modified Game-OSS in exactly the same manner. In particular, we get the following bound on the advantage in the generalized game

$$\text{Adv}_{\mathcal{A}}^{\text{Game-OSS-gen}} \leq N \cdot \text{Adv}_{\mathcal{A}}^{\text{Game-OSS}}.$$

Finally, they relate the probabilities that $\bar{w}$ and $\bar{w}'$ are feasible for $\mathcal{J}$ by bounding

$$|\Pr[\bar{w} \text{ is not feasible for } \mathcal{J}] - \Pr[\bar{w}' \text{ is not feasible for } \mathcal{J}]|$$

by the one-sided security of BT-KEM. They reduce any distinguishing adversary $\mathcal{A}$ to the generalized one-sided security game. The reduction $\mathcal{B}$ works by picking a random j^* and d and embedding the N challenge ciphertexts received by Game-OSS-gen as the ciphertexts that would be generated by the Tr subroutine that is called with different (b_k, b_0, b_1) in the real versus the idealized experiment. If the internal bit b of Game-OSS-gen is 0, then the reduction simulates the real tracing experiment. If, however, the internal bit b is 1, then the reduction's simulation amounts to the idealized game. The reduction answers with the bit $\bar{w}_{j^*}$ or outputs a random bit,

¹²The parameter $\delta \in [0, 1]$ is the noisiness limit of the fingerprinting code and it is fixed depending on the desired tracing parameter ε in the TT-KEM construction.

¹³Essentially, $\bar{w}$ is feasible for $\mathcal{J}$, if $\bar{w}$ does not contain a 1 at position j , where the parties in $\mathcal{J}$ have all left keys and vice versa.

¹⁴The feasibility of $\bar{w}$ for $\mathcal{J}$ is important, because the tracing algorithm of the fingerprinting code that is run in the end is guaranteed to output a non-empty subset of $\mathcal{J}$, if $\bar{w}$ is feasible for $\mathcal{J}$ (and $\bar{w}$ is δ -noisy).

should $\mathcal{A}$ corrupt the wrong side for j^* . In the end, the reduction achieves the following bound:

$$\begin{aligned} & |\Pr[\bar{w} \text{ is not feasible for } \mathcal{J}] - \Pr[\bar{w}' \text{ is not feasible for } \mathcal{J}]| \\ & \leq 2\ell \text{Adv}_{\mathcal{B}}^{\text{Game-OSS-gen}} \\ & \leq 2\ell N \text{Adv}_{\mathcal{B}}^{\text{Game-OSS}}. \end{aligned}$$

To see, why the same reduction works for our modifications, we make a close comparison between the (modified) Tr subroutine and the (modified) Game-OSS in Figure 17. One can clearly see that the modifications to Tr and Game-OSS mirror each other. Specifically, the only difference over [15] is the additional tag τ^* that is given to the decoder (resp. the adversary) and it is generated in exactly the same manner in both Tr and Game-OSS. Hence, when $\mathcal{B}$ uses the ciphertext-tag pairs as generated by our modified one-sided security game to simulate the corresponding ciphertexts within the modified Tr subroutine, $\mathcal{B}$ equally simulates either the real tracing experiment or the idealized tracing experiment depending on the interal bit b of Game-OSS.

E SECURITY OF BTIB-KEM-1

E.1 Semantic Security

We prove semantic security in two steps. First, we introduce a new augmented bilinear decisional Diffie-Hellman (ABDDH) assumption that is easier to work with, but is reducible to standard BDDH (Definition B.4). We refer to Appendix F for the full reduction.

Definition E.1 (Augmented Bilinear Decisional Diffie-Hellman).

Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the augmented bilinear decisional Diffie-Hellman (ABDDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries it holds that

$$|\Pr[\mathcal{A}(E, \vec{x}, e(g_1, g_2)^{w\alpha yzr}) = 1] - \Pr[\mathcal{A}(E, \vec{x}, e(g_1, g_2)^v) = 1]|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, $w, \alpha, y, z, r, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_1^{yz}, g_1^{yr}, g_1^{zr}, g_1^{\alpha yz}, g_2, g_2^y, g_2^z, g_2^{yz}, g_2^{\alpha y}, g_2^{\alpha z}, g_2^{\alpha yz}).$$

We conclude the proof with a reduction from IND-SID-CPA-security of BTIB-KEM-1 to the ABDDH assumption.

PROOF OF LEMMA 5.1. Let $\ell_{\max} > 0$ be an upper bound for ℓ , which is an arbitrary but fixed polynomial in the security parameter such that $1 \leq \ell \leq \ell_{\max}$. We prove IND-SID-CPA security of our first BTIB-KEM construction by reduction to the ABDDH assumption (Definition E.1). Let $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ be a PPT adversary against the Game-IND-SID-CPA experiment (Figure 5) such that

$$\Pr[\text{Game-IND-SID-CPA}_{\mathcal{A}}^{\text{BTIB-KEM}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda)$$

for a non-negligible ε . We construct a PPT reduction $\mathcal{B}$ that uses $\mathcal{A}$ internally to break the ABDDH assumption.

First, $\mathcal{B}$ receives the security parameter 1^λ as well as an ABDDH instance

$$(g_1, g_1^y, g_1^z, g_1^{yz}, g_1^{yr}, g_1^{zr}, g_1^{\alpha yz}, (g_2, g_2^y, g_2^z, g_2^{yz}, g_2^{\alpha y}, g_2^{\alpha z}, g_2^{\alpha yz}), T$$

where the challenge T is either $e(g_1, g_2)^{w\alpha yzr}$ or $e(g_1, g_2)^v$ for a random $v \in \mathbb{Z}_p$.

$\mathcal{B}$ samples a random $j^* \in [\ell_{\max}]$ and runs $\mathcal{A}_1(1^\lambda)$ to receive the selected challenge identity I^* and the number of positions ℓ , while simulating the random oracle.

Simulating the random oracle $\mathcal{H}(j, ID)$. For each query $\mathcal{H}(j, ID)$, $\mathcal{B}$ simulates the random oracle $\mathcal{H}$ as follows: If $(j, ID) = (j^*, I^*)$, then output the group element g_2^w from the ABDDH instance. Otherwise, sample a random $w_{j, ID} \leftarrow \mathbb{Z}_p$ and output $g_2^{w_{j, ID}}$. $\mathcal{B}$ answers queries *consistently*, i.e. it saves the values and repeats answers for identical queries.

If $j^* \notin \ell$, then $\mathcal{B}$ stops the simulation and outputs a random bit. One can bound the loss incurred here by $1/\ell_{\max}$. Otherwise, $\mathcal{B}$ simulates KGen as follows:

- For $j \neq j^*$, run the normal key generation procedure.
- For j^* , embed the ABDDH instance into the public key. In particular, set $X_{j^*} \leftarrow g_1^{\alpha yz}$ and set $Y_{j^*} \leftarrow g_1^y$ as well as $Z_{j^*} \leftarrow g_1^z$, which are part of the ABDDH instance.

$\mathcal{B}$ runs $\mathcal{A}_2$ with the resulting public key and receives a set C of corrupted parties. Then, $\mathcal{B}$ sets the challenge encapsulation key k to the ABDDH challenge $k \leftarrow T$ and the challenge ciphertext $c \leftarrow (g_1^{yr}, g_1^{zr})$, which $\mathcal{B}$ also takes from the ABDDH instance. Finally, $\mathcal{B}$ assembles the secret key of the corrupted parties $\{sk_i\}_{i \in C}$ as follows. For $j \neq j^*$, $\mathcal{B}$ uses the normal secret keys from KGen. For j^* , $\mathcal{B}$ just samples a random left and right key from $\mathbb{Z}_p$ on behalf of every corrupted party. W.l.o.g. we assume that $|C| = t - 1$. Let u_i be the left secret key of corrupted party i and v_i be the right secret key of corrupted party i . Note that u_i corresponds to the field element $z \cdot s_i^{(j^*)}$ and v_i corresponds to $y \cdot s_i^{(j^*)}$. It is possible to simulate these with random values, as $\mathcal{A}$ only receives up to $t - 1$ shares.

$\mathcal{B}$ runs $\mathcal{A}_3^{O^{\text{Dldk}}}((c, k), \{sk_i\}_{i \in C})$, receiving a bit b' as a result. In the end, $\mathcal{B}$ returns b' as its guess in the ABDDH instance (if $b' = 0$, it guesses that the challenge T is equal to $e(g_1, g_2)^v$ (i.e., uniformly random in $\mathbb{G}_T$) and if $b' = 1$, it guesses that the challenge is equal to $e(g_1, g_2)^{w\alpha yzr}$). On top of that, $\mathcal{B}$ needs to simulate oracle queries to O^{Dldk} to $\mathcal{A}$.

Simulating the identity key derivation oracle $O^{\text{Dldk}}(ID, x)$. If the target party x is corrupted ($x \in C$), we can trivially simulate identity key derivation using the simulated key shares u_x and v_x . Otherwise, we do the following: For all $j \neq j^*$, $\mathcal{B}$ can simulate O^{Dldk} according to protocol, as it knows all the secret keys. For j^* , we distinguish between 2 cases. If $ID = I^*$, then we abort as mandated by the game. If $ID \neq I^*$ we do the following:

- Define the polynomial f with $f(0) = w_{j^*, ID} \cdot z\alpha$ and $f(i) = u_i$ for $i \in C$.
- Compute $g_2^{f(0)} \leftarrow (g_2^{z\alpha})^{w_{j^*, ID}}$ from the ABDDH instance¹⁵ and $g_2^{f(i)} \leftarrow g_2^{u_i}$ for $i \in C$. Use these t values to interpolate f in $\mathbb{G}_2$ and evaluate $g_2^{f(x)} \leftarrow \prod_{i \in C \cup \{0\}} (g_2^{f(i)})^{l_{i, C \cup \{0\}}(x)}$ where $l_{i, C \cup \{0\}}(x)$ is the Lagrange coefficient of index i for interpolation at position x among the set $C \cup \{0\}$.
- Set $\text{idk}_{i, 0}^{(j^*)} \leftarrow g_2^{f(x)}$.
- Repeat the same for the right key for the function g with $g(0) = w_{j^*, ID} \cdot y\alpha$ and $g(i) = v_i$ for $i \in C$.

¹⁵This is possible, because for $ID \neq I^*$ the field element $w_{j^*, ID}$ is sampled by $\mathcal{B}$ during simulation of the random oracle and thus known to $\mathcal{B}$. In case $\mathcal{A}$ has never queried for $\mathcal{H}(j^*, ID)$, $\mathcal{B}$ just samples a value for $w_{j^*, ID}$ and continues to answer consistently for the random oracle.

$\text{Tr}^{D(\cdot)}(\text{pk}, j, N, b_k, b_0, b_1)$	$\text{Game-OSS}_{\mathcal{A}}(1^\lambda)$
$\text{ctr} \leftarrow 0$	$(\ell, u, d, \text{st}_1) \leftarrow \mathcal{A}_1(1^\lambda)$
for $r = 1, \dots, N$ do	$(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda, n, t, \ell)$
$(\text{sk}_e, \text{vk}_e) \leftarrow \text{TagKeys}(1^\lambda)$	$(\text{sk}_e, \text{vk}_e) \leftarrow \text{TagKeys}(1^\lambda)$
$(k^{(0)}, c^{(0)} := (c_0^{(0)}, c_1^{(0)})) \leftarrow \text{BT-KEM.Enc}(\text{pk}, j, \text{vk}_e)$	$(k^{(0)}, c^{(0)} := (c_0^{(0)}, c_1^{(0)})) \leftarrow \text{Enc}(\text{pk}, u, \text{vk}_e)$
$(k^{(1)}, c^{(1)} := (c_0^{(1)}, c_1^{(1)})) \leftarrow \text{BT-KEM.Enc}(\text{pk}, j, \text{vk}_e)$	$(k^{(1)}, c^{(1)} := (c_0^{(1)}, c_1^{(1)})) \leftarrow \text{Enc}(\text{pk}, u, \text{vk}_e)$
$c^* \leftarrow (c_0^{(b_0)}, c_1^{(b_1)})$	$b \leftarrow \{0, 1\}$
$\pi^* \leftarrow \text{GenTag}(c^*, \text{sk}_e)$	if $b = 0$ then $c^* \leftarrow (c_0^{(0)}, c_1^{(1)})$
if $D((c^*, \pi^*), k^{(b_k)}) = 1$ then $\text{ctr} \leftarrow \text{ctr} + 1$	if $b = 1$ then $c^* \leftarrow (c_0^{(d)}, c_1^{(d)})$
return ctr	$\pi^* \leftarrow \text{GenTag}(c^*, \text{sk}_e)$
	$\text{shares} \leftarrow \{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{i \in [n], j \in [\ell] \setminus \{u\}}, \{\text{sk}_{i,u}^{(u)}\}_{i \in [n]}$
	$b' \leftarrow \mathcal{A}_2(\text{st}_1, \text{pk}, (c^*, \pi^*), k^{(0)}, \text{shares})$
	return $b \stackrel{?}{=} b'$

Figure 17: Comparison between the Tr subroutine and the one-sided security game of BT-KEM. Modifications over [15] are highlighted in gray.

Analysis. Observe that if $\mathcal{B}$ is running in an ABDDH instance where v is random, then $\mathcal{B}$ simulates Game-IND-SID-CPA with $b = 0$ to $\mathcal{A}$. If, however, the ABDDH challenge equals $e(g_1, g_2)^{w\alpha yzr}$, then $\mathcal{B}$ simulates Game-IND-SID-CPA with $b = 1$. Hence, $\mathcal{B}$ has the non-negligible advantage $\varepsilon(\lambda)/\ell_{\max}$ to break ABDDH. $\square$

E.2 One-Sided Security

Just as in the semantic security proof, we introduce the following augmented XDH assumption (AXDH) that is easier to work with in the security proof. A full reduction from AXDH to standard XDH (Definition B.5) is presented in Appendix F.

Definition E.2 (Augmented XDH Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the augmented external Diffie-Hellman (AXDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries it holds that

$$\left| \Pr \left[\mathcal{A}(E, \vec{x}, g_1^{zr}) = 1 \right] - \Pr \left[\mathcal{A}(E, \vec{x}, g_1^{zr'}) = 1 \right] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, and $y, z, r, r' \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_1^{yr}, g_1^{yr'}).$$

Our result of one-sided security is therefore captured by the following Lemma.

LEMMA E.3. *For all PPT adversaries $\mathcal{A}$, there exists a PPT reduction $\mathcal{B}$ such that*

$$\text{Game-OSS-ID}_{\mathcal{A}}^{\text{BTIB-KEM-1}}(1^\lambda) \leq \text{Adv}_{\mathcal{B}, \mathcal{G}}^{\text{AXDH}}$$

for all $\lambda \in \mathbb{N}$.

PROOF OF LEMMA E.3. We make a reduction from Game-OSS-ID of BTIB-KEM-1 (Figure 6) to AXDH (Definition E.2). Let $\mathcal{A}$ be a PPT adversary such that $\Pr[\text{Game-OSS-ID}_{\mathcal{A}}^{\text{BTIB-KEM-1}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda)$. We construct a reduction $\mathcal{B}$, which is solving the AXDH problem. $\mathcal{B}$ receives $g_1, g_1^y, g_1^z, g_1^{yr}, g_1^{yr'} \in \mathcal{G}_1$ and $g_2 \in \mathcal{G}_2$ as input along with a challenge T which is either g_1^{zr} or $g_1^{zr'}$.

$\mathcal{B}$ runs $\mathcal{A}$ to get $(\ell, u, d, \text{ID}^*)$, while simulating the random oracle as follows throughout the reduction. For now, we assume that $d = 0$. At the end of the proof, we explain what changes for $d = 1$. In the $d = 0$ case, the adversary $\mathcal{A}$ gets a matching key k and left ciphertext c_0 and needs to distinguish whether the right ciphertext c_1 fits with k and c_0 (case $b = 1$) or belongs to another encryption (case $b = 0$). In the reduction, we will leverage $\mathcal{A}$'s advantage by embedding the challenge T as the right ciphertext.

Simulating the random oracle $\mathcal{H}$. For each query $\mathcal{H}(j, \text{ID})$, $\mathcal{B}$ simulates the random oracle $\mathcal{H}$ by sampling a random $w_{j, \text{ID}} \leftarrow \mathbb{Z}_p$ and outputting $g_2^{w_{j, \text{ID}}}$. $\mathcal{B}$ answers queries *consistently*, i.e. it saves the values and repeats answers for identical queries. We assume, $\mathcal{A}$ makes a hash query on the target identity ID^* for level u ; if not, the reduction makes the query $\mathcal{H}(u, \text{ID}^*)$ itself.

Simulating key generation. Our simulation differs between the target position u and all other positions as follows:

- For all $j \neq u, j \in [\ell]$, we sample α_j, y_j, z_j , compute the public keys X_j, Y_j, Z_j and all secret key shares for both sides honestly.
- For $j = u$, we need to simulate only left key shares. For this, we sample $\beta \leftarrow \mathbb{Z}_p$ and embed $X_u \leftarrow (g_1^y)^\beta$, as well as $Y_u \leftarrow g_1^y$ and $Z_u \leftarrow g_1^z$ from the AXDH instance. Following the notation of the scheme, we view $X_u = g_1^{\alpha y z}$ where $\alpha = \beta/z$ or $\beta = \alpha z$. Next, we create a Shamir secret sharing $\beta_1, \dots, \beta_n$ of β . We set $sk_{i,0}^{(u)} = \beta_i$

Simulating the key and ciphertext. Let $w = w_{u, \text{ID}^*}$. Recall that $\mathcal{B}$ receives g_1^{yr} as part of the challenge. $\mathcal{B}$ computes $k_0 \leftarrow e(g_1^{yr}, g_2^w)^\beta = e(g_1, g_2)^{\alpha w y z r}$. We set the left ciphertext as $c_0^{(0)} \leftarrow g_1^{yr}$ and the right ciphertext as $c_1 \leftarrow T$, where T is AXDH challenge. We pass k_0, c_0 and c_1 together with the secret shares and the public key to the adversary. In the end, the reduction $\mathcal{B}$ outputs the same bit as $\mathcal{A}$.

Analysis. When $T = g_1^{zr}$, then $\mathcal{B}$ simulates all the parameters for $b = 1$ as $c_1 = T = g_1^{zr}$ fits with the pair $k_0 = e(X_u, \mathcal{H}(u, \text{ID}^*))^r$ and $c_0^{(0)} = g_1^{yr}$. Similarly when $T = g_1^{zr'}$ then $\mathcal{B}$ simulates all the parameters for $b = 0$, where the right ciphertext $c_1 = g_1^{zr'}$ does not fit with k_0 and $c_0^{(0)}$. Hence, $\mathcal{B}$ has the same advantage against AXDH, as $\mathcal{A}$ against Game-OSS-ID of BTIB-KEM-1.

Case d = 1. For this case, we swap Y_u and Y_z in the public key. We essentially treat g_1^y as g_1^z and vice-versa. We set the challenge as follows:

- Set $c_1 = g_1^{yr}$, $c_0 = T$, where T is the challenge $\mathcal{B}$ received. $T \in \{g_1^{zr}, g_1^{zr'}\}$.
- Set $c^* = (c_0, c_1)$
- Let $w = w_{u, \text{ID}^*}$. Recall, $\mathcal{B}$ receives $g_1^{yr'}$ as part of the challenge. $\mathcal{B}$ computes

$$k_0 = (g_1^{yr'}, g_2^w)^\beta = (g_1, g_2)^{\alpha w y z r'}.$$

The correctness of the decapsulation follows similarly as above. Clearly, k_0 does not fit with c_1 . When $T = g_1^{zr}$ then $\mathcal{B}$ simulates all the parameters for $b = 1$. In particular, $c_0 = g_1^{zr}$ matches with $c_1 = g_1^{yr}$. Similarly when $T = g_1^{zr'}$ then $\mathcal{B}$ simulates all the parameters for $b = 0$ as $c_0 = g_1^{zr'}$ matches with $k_0 = e(g_1, g_2)^{\alpha w y z r'}$. This concludes the proof. $\square$

F REDUCTIONS TO WELL-ESTABLISHED ASSUMPTIONS

In this section, we reduce the ABDDH assumption and the AXDH assumption, which were introduced to make a more comprehensible proof of semantic security and one-sided security for BTIB-KEM-1, to the standard BDDH and XDH assumptions

First we recall the two augmented hardness assumptions.

Definition F.1 (Augmented Bilinear Decisional Diffie-Hellman). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the augmented bilinear decisional Diffie-Hellman (ABDDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries it holds that

$$\left| \Pr [\mathcal{A}(E, \vec{x}, e(g_1, g_2)^{w\alpha y z r}) = 1] - \Pr [\mathcal{A}(E, \vec{x}, e(g_1, g_2)^v) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, $w, \alpha, y, z, r, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_1^{yz}, g_1^{yr}, g_1^{zr}, g_1^{\alpha y z}, g_2, g_2^y, g_2^z, g_2^w, g_2^{y\alpha}, g_2^{z\alpha}).$$

The second one is the augmented XDH assumption

Definition F.2 (Augmented XDH Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the augmented external Diffie-Hellman (AXDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries it holds that

$$\left| \Pr [\mathcal{A}(E, \vec{x}, g_1^{zr}) = 1] - \Pr [\mathcal{A}(E, \vec{x}, g_1^{zr'}) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, and $y, z, r, r' \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_1^{yr}, g_1^{yr'}, g_2).$$

F.1 Augmented BDDH

We show a reduction from the standard Bilinear Decisional Diffie-Hellman problem for type-3 pairing [20] as defined in Definition B.4. We redefine it here with different variable names to make the reduction easier to follow.

Definition F.3 (Bilinear Decisional Diffie-Hellman). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the bilinear decisional Diffie-Hellman (BDDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries $\mathcal{B}$, it holds that

$$\left| \Pr [\mathcal{B}(E, \vec{x}, e(g_1, g_2)^{w\alpha r}) = 1] - \Pr [\mathcal{B}(E, \vec{x}, e(g_1, g_2)^v) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, $w, \alpha, r, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_2, g_2^w, g_2^\alpha).$$

The reduction works in the following way. Suppose we are given an adversary $\mathcal{A}$ against the Augmented BDDH problem. We design an adversary $\mathcal{B}$ against the standard BDDH problem. The input to $\mathcal{B}$ is $(E, \vec{x}, \hat{T})$ where either $\hat{T} = e(g_1, g_2)^{w\alpha r}$ (real case) or $\hat{T} = e(g_1, g_2)^v$ (ideal case). $\mathcal{B}$ samples $y, z \leftarrow \mathbb{Z}_p$ and computes the following elements of $\mathbb{G}_1$ in $\vec{x}$.

$$g_1^y, g_1^z, g_1^{yz}, (g_1^r)^y, (g_1^r)^z, (g_1^\alpha)^{yz}.$$

For the elements in $\mathbb{G}_2$, $\mathcal{B}$ already got g_2^w as the input. For the rest of the elements of $\mathbb{G}_2$ in $\vec{x}$, $\mathcal{B}$ computes

$$g_2^y, g_2^z, (g_2^\alpha)^y, (g_2^\alpha)^z.$$

Finally, $\mathcal{B}$ computes $T = \hat{T}^{yz}$ and invokes $\mathcal{A}$ on $(E, \vec{x}, T)$. If $\mathcal{A}$ returns real, $\mathcal{B}$ outputs real. Otherwise, $\mathcal{B}$ outputs random.

For the analysis, we see that all the elements of $\vec{x}$ is computed following the same distribution as in the problem statement. For real case, T is computed truthfully. Further, as $\hat{v}$ is uniformly distributed and independent from y, z , the distribution of $v = \hat{v}yz$ is uniform. Thus distribution of T is identical to that of the problem statement. Hence $\mathcal{B}$ simulates the problem instance perfectly.

We conclude

$$\text{Adv}_{\mathcal{B}, \mathcal{G}}^{\text{BDDH}} = \text{Adv}_{\mathcal{A}, \mathcal{G}}^{\text{ABDDH}}$$

F.2 Augmented XDH

Recall the standard XDH assumption. We redefine it with different variable names.

Definition F.4 (XDH Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the external Diffie-Hellman (XDH) problem is hard for $\mathcal{G}$, if for all PPT adversaries $\mathcal{B}$ it holds that

$$\left| \Pr [\mathcal{B}(E, \vec{x}, g_1^{zr}) = 1] - \Pr [\mathcal{B}(E, \vec{x}, g_1^v) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, and $z, r, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_2).$$

We show a two step reduction from the standard XDH problem. First, we introduce the external 2-Diffie Hellman problem.

Definition F.5 (X2DH Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the external two Diffie-Hellman (X2DH) problem is hard for $\mathcal{G}$, if for all PPT adversaries $\mathcal{B}$ it holds that

$$\left| \Pr [\mathcal{B}(E, \vec{x}, g_1^{zr}) = 1] - \Pr [\mathcal{B}(E, \vec{x}, g_1^{zr'}) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, and $z, r, r' \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^y, g_1^z, g_1^{r'}, g_2).$$

Hardness of X2DH can be established by applying the XDH assumption twice (once between g_1^{zr} and g_1^y and then between $g_1^{zr'}$ and g_1^y). Thus for all PPTM $\mathcal{B}$, it holds that

$$\text{Adv}_{\mathcal{B}, \mathcal{G}}^{\text{X2DH}} \leq 2\text{Adv}_{\mathcal{B}, \mathcal{G}}^{\text{XDH}}$$

For the second step, we show a reduction from the AXDH problem to the X2DH problem. Consider any adversary $\mathcal{A}$ against the AXDH problem. We construct an adversary $\mathcal{B}$ against the X2DH problem. $\mathcal{B}$ receives $(E, \vec{\chi}, T)$ as input where $\vec{\chi} = (g_1, g_1^z, g_1^r, g_1^{r'}, g_2)$, and either $T = g_1^{zr}$ or $T = g_1^{zr'}$. $\mathcal{B}$ samples $y \leftarrow \mathbb{Z}_p$, and computes $g_1^y, (g_1^r)^y$ and $(g_1^{r'})^y$. Note, these three elements along with $\vec{\chi}$ constitutes the required vector $\vec{x}$. $\mathcal{B}$ invokes $\mathcal{A}$ on $(E, \vec{x}, T)$, and returns the response from $\mathcal{A}$. Clearly, $\mathcal{B}$ is correct whenever $\mathcal{A}$ is correct. From the distribution of the input to $\mathcal{B}$ and uniformity of y , the inputs to $\mathcal{A}$ are simulated perfectly to the problem condition. We conclude

$$\text{Adv}_{\mathcal{B}, \mathcal{G}}^{\text{X2DH}} = \text{Adv}_{\mathcal{A}, \mathcal{G}}^{\text{AXDH}}$$

Thus we conclude that for all PPTM $\mathcal{A}$, there exists PPTM $\mathcal{B}$ such that

$$\text{Adv}_{\mathcal{A}, \mathcal{G}}^{\text{AXDH}} \leq 2\text{Adv}_{\mathcal{B}, \mathcal{G}}^{\text{X2DH}}$$

G THRESHOLDIZED BTIB-KEM-2

In this section we present our second BTIB-KEM construction (BTIB-KEM-2) in detail. Upfront, we note that the BTIB-KEM-2 construction is defined with respect to an arbitrary but fixed constant $\ell_{\max}$ such that $1 \leq \ell \leq \ell_{\max}$, where $\ell_{\max}$ is polynomial in the security parameter λ . This requirement is only necessary because of a minor technical detail in the semantic security proof, which requires the reduction to know an upper bound on ℓ upfront. This detail does not in any way impact the practicality of the scheme.

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$
 - Generate a pairing ensemble $E = (\mathbb{G}_1, g_1, \mathbb{G}_2, g_2, \mathbb{G}_T, p, e)$.
 - Sample random group elements $u_1, u_2, v_1, v_2 \leftarrow \mathbb{G}_1$.
 - Additionally, we use a hash function H with output domain in $\mathbb{G}_1$.
 - Output $\text{pp} := (E, (u_1, u_2, v_1, v_2))$.
- $\text{KGen}(1^\lambda, n, t, \ell) \rightarrow (\text{pk}, \text{sk})$
 - Sample $\alpha, y, z \leftarrow \mathbb{Z}_p$.
 - Compute $X \leftarrow g_1^\alpha, Y \leftarrow g_1^y, Z \leftarrow g_1^z$ and $Q \leftarrow g_2^{\alpha y z}$.
 - Set $\text{pk} = \{X, Y, Z, Q\}$.
 - Additionally, we define the functions $\mathcal{L}_1, \mathcal{L}_2, \mathcal{R}_1, \mathcal{R}_2$ for $j \in [\ell], \text{ID} \in \mathbb{Z}_p$ as follows:

$$\mathcal{L}_1(j) = X^j \cdot u_1 [= g_1^{\alpha j} \cdot u_1]$$

$$\mathcal{L}_2(\text{ID}) = X^{\text{ID}} \cdot u_2$$

$$\mathcal{R}_1(j) = X^j \cdot v_1$$

$$\mathcal{R}_2(\text{ID}) = X^{\text{ID}} \cdot v_2$$

Note that they can be evaluated by anyone in possession of the public pk and the public parameters p .

- Generate a (t, n) -sharing $\{\alpha_i\}_{i \in [n]}$ of α .
- For every $(j, i) \in [\ell] \times [n]$:
 - * Sample $r_{1,i}^{(j)}, h_{1,i}^{(j)} \leftarrow \mathbb{Z}_p$.

* Compute the left key-shares as:

$$\text{sk}_{i,0}^{(j)} \leftarrow (H(j)^{\alpha_i z} \cdot \mathcal{L}_1(j)^{r_{1,i}^{(j)}}, Y^{r_{1,i}^{(j)}}, g_2^{r_{1,i}^{(j)}}, g_2^{z\alpha_i})$$

* Compute the right key-shares as

$$\text{sk}_{i,1}^{(j)} \leftarrow (H(j)^{\alpha_i y} \cdot \mathcal{R}_1(j)^{h_{1,i}^{(j)}}, Z^{h_{1,i}^{(j)}}, g_2^{h_{1,i}^{(j)}} g_2^{y\alpha_i})$$

- Set $\text{sk} \leftarrow \{(\text{sk}_{i,0}^{(j)}, \text{sk}_{i,1}^{(j)})\}_{j \in [\ell], i \in [n]}$
- Output pk, sk .
- $\text{Enc}(\text{pk}, j, \text{ID}) \rightarrow (k, c = (c_0, c_1))$
 - Sample $s, t_0, t_1 \leftarrow \mathbb{Z}_p$
 - Compute the key $k \leftarrow e(H(j)^s, Q) [= e(H(j), g_2)^{\alpha y z s}]$.
 - Set the left ciphertext

$$c_0 := (c_{0,1}, c_{0,2}, c_{0,3}, c_{0,4}, c_{0,5}, c_{0,6})$$

$$\leftarrow (Y^s \cdot g_2^{t_0}, \mathcal{L}_1(j)^{t_0}, \mathcal{L}_2(\text{ID})^s, \mathcal{L}_1(j)^{t_1}, \mathcal{L}_2(\text{ID})^{t_1}, H(j)^{t_0})$$

– Set the right ciphertext

$$c_1 := (c_{1,1}, c_{1,2}, c_{1,3}, c_{1,4}, c_{1,5}, c_{1,6})$$

$$\leftarrow (Z^s \cdot g_2^{t_1}, \mathcal{R}_1(j)^s, \mathcal{R}_2(\text{ID})^s, \mathcal{R}_1(j)^{t_1}, \mathcal{R}_2(\text{ID})^{t_1}, H(j)^{t_1})$$

– Assemble $c = (c_0, c_1)$ and output (k, c)

- $\text{Dldk}([\text{sk}_{i,b_{i,j}}^{(j)}]_{j \in [\ell]}, \text{ID}) \rightarrow \text{idk}_i$ The identity key derivation algorithm uses either the left or right secret key share per index j of party i to derive a left or right identity key share respectively. For left shares, the bit $b_{i,j}$ is 0, for right shares it is 1.

– For all $j \in [\ell]$ do:

* Let $\text{sk}_i \leftarrow \text{sk}_{i,b_{i,j}}^{(j)}$

* If $b_{i,j} = 0$, then sample $r_{2,i}^{(j)} \leftarrow \mathbb{Z}_p$ and compute the left partial identity key as

$$(w_{j,i,1}, w_{j,i,2}, w_{j,i,3}, w_{j,i,4}, w_{j,i,5}, w_{j,i,6})$$

$$\leftarrow (\text{sk}_{i,1} \cdot \mathcal{L}_2(\text{ID})^{r_{2,i}^{(j)}}, \text{sk}_{i,2}, Y^{r_{2,i}^{(j)}}, \text{sk}_{i,3}, g_2^{r_{2,i}^{(j)}}, \text{sk}_{i,4})$$

* Otherwise, if $b_{i,j} = 1$, then sample $h_{2,i}^{(j)} \leftarrow \mathbb{Z}_p$ and compute the right partial identity key as

$$(w_{j,i,1}, w_{j,i,2}, w_{j,i,3}, w_{j,i,4}, w_{j,i,5}, w_{j,i,6})$$

$$\leftarrow (\text{sk}_{i,1} \cdot \mathcal{R}_2(\text{ID})^{h_{2,i}^{(j)}}, \text{sk}_{i,2}, Z^{h_{2,i}^{(j)}}, \text{sk}_{i,3}, g_2^{h_{2,i}^{(j)}}, \text{sk}_{i,4})$$

* Set

$$\text{idk}_i^{(j)} \leftarrow (w_{j,i,1}, w_{j,i,2}, w_{j,i,3}, w_{j,i,4}, w_{j,i,5}, w_{j,i,6})$$

– Output $\text{idk}_i \leftarrow [(b_{i,j}, \text{idk}_i^{(j)})]_{j \in [\ell]}$.

- $\text{Combidk}(S, [\text{idk}_i]_{i \in S}) \rightarrow \text{idk}$ This algorithm combines the identity key shares obtained through previous Dldk queries to the Servers in S .

– Require that $S \subseteq [n]$, where $|S| \geq t$.

– For all $j \in [\ell]$ do:

* Denote by $S_0 \leftarrow \{i \in S \mid b_{i,j} = 0\}$ the set of all servers who have provided left identity key shares for index j .

* Denote by $S_1 \leftarrow S \setminus S_0$ the set of all servers who provided right identity key shares of index j . Observe that, by construction, $S_0 \cup S_1 = S$ and $S_0 \cap S_1 = \emptyset$.

- * Compute the left identity key of position j :

$$L^{(j)} := (L_1^{(j)}, \dots, L_6^{(j)}),$$

where $L_m^{(j)} \leftarrow \prod_{i \in S_0} w_{j,i,m}^{l_{i,S}(0)}$ for $m \in [1, 6]$. Here, we partially aggregate the left identity key shares derived by parties from S_0 through interpolation at position 0 among the set S .

- * Compute the right identity key for position j :

$$R^{(j)} := (R_1^{(j)}, \dots, R_6^{(j)}),$$

where $R_m^{(j)} \leftarrow \prod_{i \in S_1} w_{j,i,m}^{l_{i,S}(0)}$ for $m \in [1, 6]$. We aggregate all right shares also by partial interpolation among S .

- Return $\text{idk} = (L^{(j)}, R^{(j)})_{j \in [\ell]}$.
- $\text{Dec}(\text{idk}, c = (c_0, c_1), j) \rightarrow k$
 - Compute

$$k_1 \leftarrow \frac{e(L_1^{(j)}, c_{0,1})}{\prod_{m=2}^6 e(c_{0,m}, L_m^{(j)})},$$

$$k_2 \leftarrow \frac{e(R_1^{(j)}, c_{1,1})}{\prod_{m=2}^6 e(c_{1,m}, R_m^{(j)})}.$$

- Return $k \leftarrow k_1 \cdot k_2$.

G.1 Security of BTIB-KEM-2

We prove semantic security and one-sided security by reduction to the following two assumptions.

Definition G.1 (ABDDH-2 Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the ABDDH-2 problem is hard for $\mathcal{G}$, if for all PPT adversaries it holds that

$$\left| \Pr[\mathcal{A}(E, \vec{x}, e(g_1, g_2)^{w\alpha y z s}) = 1] - \Pr[\mathcal{A}(E, \vec{x}, e(g_1, g_2)^v) = 1] \right|$$

is negligible in λ where $E \leftarrow \mathcal{G}(1^\lambda)$, $w, \alpha, y, z, s, v \leftarrow \mathbb{Z}_p$ and

$$\vec{x} = (g_1, g_1^w, g_1^{wz}, g_1^{wy}, g_1^\alpha, g_1^s, g_2, g_2^y, g_2^z, g_2^{yz}, g_2^{ys}, g_2^{zs}, g_2^{\alpha y}, g_2^{\alpha z}, g_2^{\alpha y z}).$$

Definition G.2 (AXDH-2 Assumption). Let $\mathcal{G}$ be an algorithm to generate ensembles E . We say that the AXDH-2 problem is hard for $\mathcal{G}$, if for all PPT adversaries it holds that

$$\left| \Pr[\mathcal{A}(E, \vec{x}, (g_1^{s+\bar{v}_1 s}, g_1^{s+\bar{v}_2 s}, g_2^{zs+t_1})) = 1] - \Pr[\mathcal{A}(E, \vec{x}, (g_1^{s'+\bar{v}_1 s'}, g_1^{s'+\bar{v}_2 s'}, g_2^{zs'+t_1})) = 1] \right| \leq \text{negl}(\lambda),$$

where $E \leftarrow \mathcal{G}(1^\lambda)$, $w, y, z, s, s', t_0, t_1, \bar{u}_1, \bar{u}_2, \bar{v}_1, \bar{v}_2 \leftarrow \mathbb{Z}_p$ and

$$\begin{aligned} \vec{x} = & (g_1, g_1^y, g_1^z, g_1^w, g_1^{wz}, g_1^{wt_0}, g_1^{\bar{u}_1}, g_1^{\bar{v}_1}, g_1^{\bar{u}_2}, g_1^{\bar{v}_2}, \\ & g_1^{t_1+\bar{v}_1 t_1}, g_1^{t_0+\bar{u}_1 t_0}, g_1^{s+\bar{u}_1 s}, g_1^{t_1+\bar{v}_2 t_1}, g_1^{t_0+\bar{u}_2 t_0}, g_1^{s+\bar{u}_2 s}, \\ & g_2, g_2^y, g_2^z, g_2^{yz}, g_2^{ys+t_0}, \\ & e(g_1, g_2)^{wyzs}, e(g_1, g_2)^{wyzs'}) \end{aligned}$$

Hardness of the Assumptions. Both assumptions can be seen as instances of the Uber problem for bilinear groups [8, 20], which has been shown to be hard in the (bilinear) generic group model (GGM) if there is no trivial way to compute the challenge from elements in $\vec{x}$ using group and pairing operations [8, 49]. Particularly for the more complex AXDH-2 assumption, this was verified by checking that all pairing results between the source-group challenge elements and elements from $\vec{x}$ do not appear as non-trivial linear combinations of pairings between elements from $\vec{x}$ and the challenge¹⁶. As this is the case for both of the above, the ABDDH-2 and AXDH-2 assumptions hold in the GGM.

G.1.1 Semantic Security.

LEMMA G.3 (IND-SID-CPA SECURITY OF BTIB-KEM-2). *Our BTIB-KEM-2 construction is IND-SID-CPA secure if the ABDDH-2 assumption (Definition G.1) holds in the underlying pairing ensemble.*

PROOF. We prove IND-SID-CPA security of BTIB-KEM-2 by reduction to the ABDDH-2 assumption (Definition G.1). Let $\mathcal{A}$ be a PPT adversary against Game-IND-SID-CPA (Figure 5) such that

$$\Pr[\text{Game-IND-SID-CPA}_{\mathcal{A}}^{\text{BTIB-KEM-2}}(1^\lambda) = 1] \geq 1/2 + \varepsilon(\lambda)$$

for a non-negligible ε . We construct a PPT reduction $\mathcal{B}$ that uses $\mathcal{A}$ internally to break the ABDDH-2 assumption.

First, $\mathcal{B}$ receives the security parameter 1^λ as well as an ABDDH-2 instance

$$\begin{aligned} & g_1, g_1^w, g_1^{wz}, g_1^{wy}, g_1^\alpha, g_1^s \\ & g_2, g_2^y, g_2^z, g_2^{yz}, g_2^{ys}, g_2^{zs}, g_2^{\alpha y}, g_2^{\alpha z}, g_2^{\alpha y z} \end{aligned}$$

with challenge T .

Let $\ell_{\max}$ be the upper bound on ℓ for BTIB-KEM-2. First, the reduction $\mathcal{B}$ samples a random $j^* \in [\ell_{\max}]$. We note here that at this point, the adversary $\mathcal{A}$ did not output ℓ yet, hence the BTIB-KEM-2 construction defines an upper bound $\ell_{\max}$ such that $1 \leq \ell \leq \ell_{\max}$. Then, the reduction $\mathcal{B}$ runs $\mathcal{A}_1$ to get ID^* and ℓ while simulating the random oracle as follows.

Simulating the random oracle H . For each query $\mathcal{H}(j)$, $\mathcal{B}$ simulates the random oracle H as follows: If $j = j^*$, then output the group element g_1^w from the assumption instance. Otherwise, sample a random $w_j \leftarrow \mathbb{Z}_p$ and output $g_1^{w_j}$. $\mathcal{B}$ answers queries consistently, i.e. it saves the values and repeats answers for identical queries.

If $j^* \notin [\ell]$, then $\mathcal{B}$ immediately returns a random bit. The probability for this to happen can be bounded by $1/\ell_{\max}$.

Then, $\mathcal{B}$ prepares a simulation of the public parameters and KGen. For this, the challenge is embedded in the public key as $X \leftarrow g_1^\alpha$, $Y \leftarrow g_2^y$, $Z \leftarrow g_2^z$, and $Q \leftarrow g_2^{\alpha y z}$. Further, $\mathcal{B}$ samples $k_{0,1}, k_{0,2}, k_{1,1}, k_{1,2} \leftarrow \mathbb{Z}_p$ to set the public parameters

$$\begin{aligned} & u_1 \leftarrow (g_1^\alpha)^{-j^*} \cdot g_1^{k_{0,1}} \\ & u_2 \leftarrow (g_1^\alpha)^{-\text{ID}^*} \cdot g_1^{k_{0,2}} \\ & v_1 \leftarrow (g_1^\alpha)^{-j^*} \cdot g_1^{k_{1,1}} \\ & v_2 \leftarrow (g_1^\alpha)^{-\text{ID}^*} \cdot g_1^{k_{1,2}} \end{aligned}$$

$\mathcal{B}$ sets $\text{pk} \leftarrow (X, Y, Z, Q)$ and runs $\mathcal{A}_2(\text{st}_1, \text{pk})$ to obtain the set of corrupted parties C . W.l.o.g. we assume that $C = \{1, \dots, t-1\}$. In the next step, the reduction needs to prepare a simulation of the secret key shares for corrupted parties in C as well as the challenge ciphertext and KEM key.

¹⁶We note that many possible combinations can be ruled out right away, as many target monomials do not appear at all in pairings among $\vec{x}$.

Simulating secret key shares. The reduction samples random $s_1, \dots, s_{t-1} \leftarrow \mathbb{Z}_p$ to simulate a sharing of α , which is a standard simulation of Shamir's secret sharing. Hence, this Shamir simulation is with respect to a polynomial $p(i)$, where $p(0) = \alpha$ and $p(i) = s_i$ of $i \in C$. As the reduction only knows g_1^α , but not α , we can only interpolate p in the exponent. We will use this trick extensively in our simulation of identity-key derivation later. We can simulate $t-1$ secret shares for parties in C by sampling $r_{i,1}^{(j)}, h_{i,1}^{(j)} \leftarrow \mathbb{Z}_p$ for all $j \in [\ell], i \in C$ and setting the secret key shares as follows. For the left share, we set $sk_{i,0}^{(j)} \leftarrow (e_{i,0}^{(j)}, Y_{i,1}^{r_{i,1}^{(j)}}, g_2^{r_{i,1}^{(j)}}, g_2^{zs_i})$, where

$$e_{i,0}^{(j)} \leftarrow (g_1^{wz})^{w_j s_i} \cdot (g_1^\alpha)^{j \cdot r_{i,1}^{(j)}} \cdot u_1^{r_{i,1}^{(j)}} [= H(j)^{zs_i} \cdot \mathcal{L}_1(j)^{r_{i,1}^{(j)}}].$$

For the right share, we set $sk_{i,1}^{(j)} \leftarrow (e_{i,1}^{(j)}, Z_{i,1}^{h_{i,1}^{(j)}}, g_2^{h_{i,1}^{(j)}}, g_2^{ys_i})$, where

$$e_{i,1}^{(j)} \leftarrow (g_1^{wy})^{w_j s_i} \cdot (g_1^\alpha)^{j \cdot h_{i,1}^{(j)}} \cdot v_1^{h_{i,1}^{(j)}} [= H(j)^{ys_i} \cdot \mathcal{R}_1(j)^{r_{i,1}^{(j)}}].$$

To prepare the challenge key and ciphertext, the reduction samples $t_0, t_1 \in \mathbb{Z}_p$ and embeds the instance of the ABDDH-2 assumption and sets the key to the ABDDH-2 challenge $k \leftarrow T$. The ciphertext c_0 is simulated as follows.

- $Y^s \leftarrow g_2^{ys} \cdot g_2^{t_0}$
- $\mathcal{L}_1(j^*)^s = (g_1^{\alpha j^*} \cdot u_1)^s = (g_1^{\alpha j^*} \cdot g_1^{-\alpha j^* + k_{0,1}})^s \leftarrow (g_1^s)^{k_{0,1}}$
- $\mathcal{L}_2(\text{ID}^*)^s \leftarrow (g_1^s)^{k_{0,2}}$
- $\mathcal{L}_1(j^*)^{t_0} \leftarrow g_1^{t_0 \cdot k_{0,1}}$
- $\mathcal{L}_2(\text{ID}^*)^{t_0} \leftarrow g_1^{t_0 \cdot k_{0,2}}$
- $H(j^*)^{t_0} \leftarrow (g_1^w)^{t_0}$

and the right ciphertext c_1 is simulated similarly using the elements g_2^{zs} and g_1^s from the assumption.

Finally, $\mathcal{B}$ runs $b' \leftarrow \mathcal{A}_3^{O^{\text{DIdk}}}(\text{st}_2, (c, k), \{\text{sk}_i\}_{i \in C})$, while simulating the O^{DIdk} oracle and returns the same bit b' .

Simulating O^{DIdk} . Given a query on (x, ID) , the identity key shares from $x \in C$ can be simulated honestly from their secret shares. We next show how to simulate the oracle for $x \notin C$. The simulation returns $\perp$ if $\text{ID} = \text{ID}^*$. Here, we focus on how to simulate the left identity-key share. The simulation for the right identity-key share works in a similar manner, using the elements g_1^{wy}, g_2^z , and g_2^{yz} from the assumption and sampling new random elements (the random elements used for the right side are independent from the ones used on the left side). Then:

- For $j \neq j^*$. Sample $r_{x,1}^{(j)} \leftarrow \mathbb{Z}_p$ and let in the following $l_i = l_{i, C \cup \{0\}}(x)$ be the Lagrange coefficient for interpolation at target index x for index i among the set $C \cup \{0\}$. Compute

$$\begin{aligned} w'_{j,x,1} &\leftarrow (g_1^{wz})^{\frac{-w_j k_{0,1} \cdot l_0}{j-j^*}} \cdot \left((g_1^\alpha)^{j-j^*} \cdot g_1^{k_{0,1}} \right)^{r_{x,1}^{(j)}} \cdot \prod_{i=1}^{t-1} (g_1^{wz})^{w_j l_i s_i} \\ &= g_1^{\frac{-w w_j z k_{0,1} \cdot l_0}{j-j^*}} \cdot g_1^{(\alpha j - \alpha j^* + k_{0,1}) \cdot r_{x,1}^{(j)}} \cdot \prod_{i=1}^{t-1} g_1^{w w_j z l_i s_i} \\ &= g_1^{\frac{-w w_j z k_{0,1} \cdot l_0}{j-j^*}} \cdot \mathcal{L}_1(j)^{r_{x,1}^{(j)}} \cdot \prod_{i=1}^{t-1} g_1^{w w_j z l_i s_i} \\ &= g_1^{w w_j z \alpha l_0} \cdot \prod_{i=1}^{t-1} g_1^{w w_j z l_i s_i} \cdot \mathcal{L}_1(j)^{r_{x,1}^{(j)} - \frac{w_j w z l_0}{j-j^*}}. \end{aligned}$$

Further we set

$$w_{j,x,4} \leftarrow (g_1^{wz})^{w_j \frac{-l_0}{j-j^*}} \cdot g_1^{r_{x,1}^{(j)}} = g_1^{\frac{-w w_j z l_0}{j-j^*} + r_{x,1}^{(j)}}.$$

Note that these are valid parts of the identity key share. For this, consider $\tilde{r}_{x,1}^{(j)} = r_{x,1}^{(j)} - \frac{w w_j z l_0}{j-j^*}$. Then we get that

$$\begin{aligned} w'_{j,x,1} &= g_1^{w w_j z \alpha l_0} \cdot \prod_{i=1}^{t-1} g_1^{w w_j z l_i s_i} \cdot \mathcal{L}_1(j)^{r_{x,1}^{(j)} - \frac{w_j w z l_0}{j-j^*}} \\ &= g_1^{w w_j z \alpha x} \cdot \mathcal{L}_1(j)^{\tilde{r}_{x,1}^{(j)}} [= H(j)^{z \alpha x} \cdot \mathcal{L}_1(j)^{\tilde{r}_{x,1}^{(j)}}]. \end{aligned}$$

Also, it holds that $w_{j,x,4} = g_1^{\tilde{r}_{x,1}^{(j)}}$.

Next, we compute $w_{j,x,2} \leftarrow (g_2^y)^{r_{x,1}^{(j)}} \cdot (g_2^{wy})^{w_j \frac{-l_0}{j-j^*}} = Y_{x,1}^{r_{x,1}^{(j)}}$ and $w_{j,x,6} \leftarrow (g_2^{z\alpha})^{l_0} \cdot \prod_{i=1}^{t-1} (g_2^z)^{s_i l_i} = g_2^{\alpha x z}$. To finish, we compute the part of the second layer honestly, that is: we sample $r_{x,2}^{(j)}$ and compute $w_{j,x,1} \leftarrow w'_{j,x,1}$.

$\mathcal{L}_2(\text{ID})^{r_{x,2}^{(j)}}$, $w_{j,x,3} \leftarrow Y_{x,2}^{r_{x,2}^{(j)}}$, and $w_{j,x,5} \leftarrow g_2^{r_{x,2}^{(j)}}$. Note that $(w_{j,x,1}, \dots, w_{j,x,6})$ is a valid left decryption key share from party x for identity ID and position j .

- For $j = j^*$. The case is handled similarly. The only difference is that we now simulate the part of the first layer honestly and apply the previous trick to the second layer.

Analysis. Observe that, if $T = e(g_1, g_2)^{w \alpha y z s}$, then $\mathcal{B}$'s simulates Game-IND-SID-CPA for $b = 1$. If instead $T = e(g_1, g_2)^v$, $\mathcal{B}$'s simulation of k is uniformly random from $\mathcal{K}(\lambda)$, hence $\mathcal{B}$ simulates the game for $b = 0$. Bounding the loss of guessing $j^* \in [\ell]$ by $1/\ell_{\max}$, we get that

$$\Pr[\text{Game-IND-SID-CPA}_{\mathcal{A}}^{\text{BTIB-KEM-2}}(1^\lambda) = 1] \leq \ell_{\max} \cdot \text{Adv}_{\mathcal{G}, \mathcal{B}}^{\text{AXDH-2}} \quad \square$$

G.1.2 One-sided Security. Our result of one-sided security is captured by the following Lemma. We prove it by reduction to the AXDH-2 assumption (Definition G.2).

LEMMA G.4. *For all PPT adversaries $\mathcal{A}$, there exists a PPT reduction $\mathcal{B}$ such that*

$$\text{Game-OSS-ID}_{\mathcal{A}}^{\text{BTIB-KEM-2}}(1^\lambda) \leq \ell_{\max} \text{Adv}_{\mathcal{G}, \mathcal{B}}^{\text{AXDH-2}}$$

for all $\lambda \in \mathbb{N}$.

PROOF. The adversary $\mathcal{B}$, who is solving the AXDH-2 problem, receives a challenge T which is either $(g_1^{s+\tilde{v}_1 s}, g_1^{s+\tilde{v}_2 s}, g_2^{zs+t_1})$ or $(g_1^{s'+\tilde{v}_1 s'}, g_1^{s'+\tilde{v}_2 s'}, g_2^{zs'+t_1})$. It also receives the following group elements:

- From $\mathbb{G}_1$: $g_1, g_1^y, g_1^z, g_1^w, g_1^{wt_1}, g_1^{wt_0}, g_1^{\tilde{u}_1}, g_1^{\tilde{v}_1}, g_1^{\tilde{u}_2}, g_1^{\tilde{v}_2}, g_1^{t_1+\tilde{v}_1 t_1}, g_1^{t_0+\tilde{u}_1 t_0}, g_1^{s+\tilde{u}_1 s}, g_1^{t_1+\tilde{v}_2 t_1}, g_1^{t_0+\tilde{u}_2 t_0}, g_1^{s+\tilde{u}_2 s}$.
- From $\mathbb{G}_2$: $g_2, g_2^y, g_2^z, g_2^{yz}, g_2^{ys+t_0}$.
- From $\mathbb{G}_T$: $e(g_1, g_2)^{w y z s}, e(g_1, g_2)^{w y z s'}$.

First, $\mathcal{B}$ guesses $j^* \leftarrow [\ell_{\max}]$.

Simulating the random oracle H . For each query $H(j)$, $\mathcal{B}$ simulates the random oracle H as follows:

- If $j = j^*$, return g_1^w from the AXDH-2 instance.
- Otherwise, sample a random $w_j \leftarrow \mathbb{Z}_p$ and output $g_1^{w_j}$. $\mathcal{B}$ answers queries *consistently*, i.e. it saves the values and repeats answers for identical queries.
- We assume, $\mathcal{A}$ makes a hash query on j^* ; if not, the reduction makes the query $H(j^*)$ itself.

$\mathcal{B}$ runs $\mathcal{A}$ to get $(\ell, u, d, \text{ID}^*)$ and aborts if $u \neq j^*$.

Simulating Setup. The reduction $\mathcal{B}$ computes the public parameters using elements from the assumption. That is, it first samples $\alpha \leftarrow \mathbb{Z}_p$ and then sets $u_1 = (g_1^{u_1})^{\alpha j^*}$, $u_2 = (g_1^{u_2})^{\alpha \text{ID}^*}$, $v_1 = (g_1^{v_1})^{\alpha j^*}$, and $v_2 = (g_1^{v_2})^{\alpha \text{ID}^*}$. Hence the function $\mathcal{L}$ and $\mathcal{R}$ behave as follows:

$$\begin{aligned}\mathcal{L}_1(j) &= g_1^{\alpha j} \cdot u_1 = g_1^{\alpha j} \cdot g_1^{\alpha j^* u_1} \\ \mathcal{L}_2(\text{ID}) &= g_1^{\alpha \text{ID}} \cdot u_2 = g_1^{\alpha \text{ID}} \cdot g_1^{\alpha \text{ID}^* u_2} \\ \mathcal{R}_1(j) &= g_1^{\alpha j} \cdot v_1 = g_1^{\alpha j} \cdot g_1^{\alpha j^* v_1} \\ \mathcal{R}_2(\text{ID}) &= g_1^{\alpha \text{ID}} \cdot v_2 = g_1^{\alpha \text{ID}} \cdot g_1^{\alpha \text{ID}^* v_2}\end{aligned}$$

We start with the case $d = 0$ first. In this case, the adversary is tasked to determine whether the given right side ciphertext c_1 fits with a correctly derived key and left ciphertext pair (k, c_0) .

Simulating key generation. Given the above public parameters, the AXDH-2 instance, as well as α , we simulate key generation as follow:

- Generate (t, n) -Shamir shares $s_1, \dots, s_n$ of α .
- For the public key, we set $X \leftarrow g_1^\alpha$, $Y \leftarrow g_2^\alpha$, $Z \leftarrow g_2^\alpha$, and $Q \leftarrow (g_2^{yz})^\alpha$.
- For $j \neq u$ the secret shares can be simulated since $H(j)^{zs_i} = (g_1^{z_j})^{w_j s_i}$ where w_j and s_i are known to the reduction and $g_1^{z_j}$ comes from the AXDH-2 instance. Then, $\mathcal{B}$ samples $r_{i,1}^{(j)}, h_{i,1}^{(j)} \leftarrow \mathbb{Z}_p$ for all $(j, i) \in ([\ell] \setminus \{j^*\}) \times [n]$ and sets the left share as

$$sk_{i,0}^{(j)} \leftarrow \left((g_1^{z_j})^{w_j s_i} \cdot \mathcal{L}_1(j)^{r_{i,1}^{(j)}}, (g_2^y)^{r_{i,1}^{(j)}}, g_2^{r_{i,1}^{(j)}}, (g_2^{z_j})^{s_i} \right),$$

and the right share as

$$sk_{i,1}^{(j)} \leftarrow \left((g_1^y)^{w_j s_i} \cdot \mathcal{R}_1(j)^{h_{i,1}^{(j)}}, (g_2^z)^{h_{i,1}^{(j)}}, g_2^{h_{i,1}^{(j)}}, (g_2^y)^{s_i} \right).$$

- For $j = u$ we need to simulate only the left shares. They can be simulated using the element g_1^{zw} from the AXDH-2 instance, since $H(u)^{zs_i} = (g_1^{zw})^{s_i}$. The adversary $\mathcal{B}$ samples $r_{i,1}^{(j)}$ for every $i \in [n]$ and then the left shares are computed as:

$$sk_{i,0}^{(u)} \leftarrow \left((g_1^{zw})^{s_i} \cdot \mathcal{L}_1(u)^{r_{i,1}^{(u)}}, (g_2^y)^{r_{i,1}^{(u)}}, g_2^{r_{i,1}^{(u)}}, (g_2^{z_j})^{s_i} \right)$$

Simulating the KEM key and ciphertext. $\mathcal{B}$ sets the KEM key $k_0 \leftarrow (e(g_1, g_2)^{wyzs})^\alpha$, where $(e(g_1, g_2)^{wyzs})$ is from the AXDH-2 instance. Conceptually, this means that $k_0 = e(H(j^*), g_2)^{\alpha yzs}$. The challenge ciphertext for (u, ID^*) is simulated as follows.

- The components of the left side, we simulate from the instance:

- $c_{0,1} \leftarrow (g_1^{s+u_1 s})^{\alpha j^*} = (g_1^{\alpha j^*} \cdot g_1^{u_1 \cdot \alpha j^*})^s = \mathcal{L}_1(j^*)^s$
- $c_{0,2} \leftarrow (g_1^{s+u_2 s})^{\alpha \text{ID}^*} = (g_1^{\alpha \text{ID}^*} \cdot g_1^{u_2 \cdot \alpha \text{ID}^*})^s = \mathcal{L}_2(\text{ID}^*)^s$
- $c_{0,3} = g_2^{ys+t_0}$
- $c_{0,4} = (g_1^{t_0+u_1 t_0})^{\alpha j^*} = \mathcal{L}_1(j^*)^{t_0}$
- $c_{0,5} = (g_1^{t_0+u_2 t_0})^{\alpha \text{ID}^*} = \mathcal{L}_2(\text{ID}^*)^{t_0}$
- $c_{0,6} = g_1^{wt_0} = H(j^*)^{t_0}$
- In order to set c_1 , we use elements from the challenge. Denote the challenge by $T = (T_1, T_2, T_3)$, then:
 - $c_{1,1} \leftarrow (T_1)^{\alpha j^*}$, which is either $\mathcal{R}_1(j^*)^s$ or $\mathcal{R}_1(j^*)^{s'}$.
 - $c_{1,2} \leftarrow (T_2)^{\alpha \text{ID}^*}$, which is either $\mathcal{R}_2(\text{ID}^*)^s$ or $\mathcal{R}_2(\text{ID}^*)^{s'}$.
 - $c_{1,3} = T_3$, which is either $g_2^{zs+t_1}$ or $g_2^{zs'+t_1}$.
 - $c_{1,4} = (g_1^{t_1+u_1 t_1})^{\alpha j^*} = \mathcal{R}_1(j^*)^{t_1}$
 - $c_{1,5} = (g_1^{t_1+u_2 t_1})^{\alpha \text{ID}^*} = \mathcal{R}_2(\text{ID}^*)^{t_1}$
 - $c_{1,6} = g_1^{wt_1} = H(j^*)^{t_1}$

The adversary $\mathcal{B}$ passes k_0, c_0, c_1 together with the secret shares to $\mathcal{A}$. In the end the reduction $\mathcal{B}$ outputs what $\mathcal{A}$ outputs.

Analysis. Observe that c_0 is a valid ciphertext that is consistent with k_0 . Furthermore, when $T = (g_1^{s+\bar{v}_1 s}, g_1^{s+\bar{v}_2 s}, g_2^{zs+t_1})$, then $\mathcal{B}$ simulates the case $b = 1$ as c_1 is consistent with c_0 and k_0 . On the other hand, when $T = (g_1^{s'+\bar{v}_1 s'}, g_1^{s'+\bar{v}_2 s'}, g_2^{zs'+t_1})$, $\mathcal{B}$ simulates the case $b = 0$ since c_1 is not consistent with c_0 and k_0 . Hence, $\mathcal{B}$ simulates the one-sided security game for $\mathcal{A}$ for the case $d = 0$.

The case $d = 1$ is handled symmetrically, except that now we set $k_0 = e(H(j^*), g_2)^{\alpha yzs'}$ and now we simulate c_0 using the challenge T (instead of c_1). Also, we swap y and z . Hence, when $T = (g_1^{s'+\bar{v}_1 s'}, g_1^{s'+\bar{v}_2 s'}, g_2^{zs'+t_1})$, it holds that c_0 is consistent with k_0 and not with c_1 , thus $\mathcal{B}$ simulates $b = 0$. On the other hand, when $T = (g_1^{s+\bar{v}_1 s}, g_1^{s+\bar{v}_2 s}, g_2^{zs+t_1})$, then c_0 is consistent with c_1 but not with k_0 , thus, $\mathcal{B}$ simulates $b = 1$. $\square$

H COMPUTATIONAL EFFICIENCY

We present the concrete computational efficiency of CCA-secure TT-KEM, when instantiated with our two constructions in Figure 18. We conclude that for both constructions the computational efficiency is sufficient for practical applications. To see this, note that on modern curves that support pairings such as BLS12-381, even the most expensive operations (that is the pairing itself) are computable in less than 1 millisecond on consumer hardware¹⁷.

I CONSISTENCY

PROOF OF THEOREM 7.2. We prove consistency in a series of game-hops. First, we replace the decryption shares of the adversary by honestly generated ones, then we extract the witness ω for the proof that is attached to the adversary's ciphertext and finally, we exclude the possibility that the adversary has generated the proof using the trapdoor. In the end, we show that both decryptions must yield the same key $k \neq \perp$ by correctness of TT-KEM.

Let Game_0 be the consistency game $\text{Game-Consist}^{\text{M-TT-KEM}}$ as displayed in Figure 10.

Game₁: In this game-hop, we generate honest decryption shares on behalf of the parties in S and S' and use the honestly generated

¹⁷These results were obtained on an AMD RYZEN 5800x CPU based on the <https://github.com/kilic/bls12-381> implementation

Algo	With BTIB-KEM-1	With BTIB-KEM-2
Enc	$\Sigma.\text{KGen} + \Sigma.\text{Sign} + \mathcal{H} + 3\text{Exp}_{\mathbb{G}_1} + 1\text{Pair}$	$\Sigma.\text{KGen} + \Sigma.\text{Sign} + 2\mathcal{H} + 4\text{Mul}_{\mathbb{G}_1} + 13\text{Exp}_{\mathbb{G}_1} + 2\text{Mul}_{\mathbb{G}_2} + 4\text{Exp}_{\mathbb{G}_2} + 1\text{Pair}$
PDec	$\Sigma.\text{Verify} + \mathcal{H} + 1\text{Exp}_{\mathbb{G}_2}$	$\Sigma.\text{Verify} + 2\mathcal{H} + 2\text{Mul}_{\mathbb{G}_1} + 2\text{Exp}_{\mathbb{G}_1} + 2\text{Exp}_{\mathbb{G}_2}$
Comb	$(t-2)\text{Mul}_{\mathbb{G}_2} + t\text{Exp}_{\mathbb{G}_2} + 2\text{Pair} + 1\text{Mul}_{\mathbb{G}_T}$	$(t-2)\text{Mul}_{\mathbb{G}_1} + t\text{Exp}_{\mathbb{G}_1} + 5(t-2)\text{Mul}_{\mathbb{G}_2} + 5t\text{Exp}_{\mathbb{G}_2} + 12\text{Pair} + 11\text{Mul}_{\mathbb{G}_T} + 1\text{Exp}_{\mathbb{G}_T}$

Figure 18: Summary of the computational efficiency of important CCA-secure TT-KEM operations, when instantiating with BTIB-KEM-1 or BTIB-KEM-2 and a signature scheme Σ as building blocks.

shares instead of those provided by the adversary when reconstructing k and k' .

LEMMA I.1. $\text{Game}_0^{M\text{-TT-KEM}}$ and $\text{Game}_1^{M\text{-TT-KEM}}$ are equivalent, i.e. for all adversaries $\mathcal{A}$ it holds that $\Pr[\text{Game}_{0,\mathcal{A}}^{M\text{-TT-KEM}}(1^\lambda) = 1] = \Pr[\text{Game}_{1,\mathcal{A}}^{M\text{-TT-KEM}}(1^\lambda) = 1]$

PROOF OF LEMMA I.1. Due to the bilinearity of the pairing operation e , the honestly generated decryption shares must be equal to the decryption shares provided by the adversary, if the share verification PVerify passes (i.e. if $e(g_1, d_i) = e(g_1^{\text{sk}_i^{(j)}}, H(j, \text{vk}_e))$). $\square$

Game₂: Next, we require the adversary to send a witness ω along with the ciphertext c and proof π and abort, if $(\chi, \omega) \notin R$ for $\chi = (\text{pk}, c_0, c_1, j, \text{vk}_e)$. In particular, Game₂ aborts, if $g_1^\omega \neq Q$ and either $c_0 \neq Y_j^\omega$ or $c_1 \neq Z_j^\omega$.

LEMMA I.2. If the proof system Π is knowledge-sound (Definition B.3), then for all PPT adversaries $\mathcal{A}$ there exists a negligible

function $\text{negl}(\lambda)$ and PPT adversary $\mathcal{B}$ such that

$$\begin{aligned} \Pr[\text{Game}_{2,\mathcal{B}}^{M\text{-TT-KEM}}(1^\lambda) = 1] \\ \geq \Pr[\text{Game}_{1,\mathcal{A}}^{M\text{-TT-KEM}}(1^\lambda) = 1] - \text{negl}(\lambda). \end{aligned}$$

PROOF OF LEMMA I.2. Let $\mathcal{A}$ be a PPT adversary such that

$$\Pr[\text{Game}_{1,\mathcal{A}}^{M\text{-TT-KEM}}(1^\lambda) = 1] = \varepsilon(\lambda)$$

for a non-negligible $\varepsilon(\lambda)$. We construct an adversary $\mathcal{B}$ that internally uses $\mathcal{A}$ as well as the extractor Ext to win Game₂. $\mathcal{B}$ runs $\mathcal{A}$ to receive the tuple (c, π) as well as the partial decryptions and then runs $\text{Ext}^{\mathcal{A}(\cdot)}(\chi, \pi)$ to obtain a witness ω . $\mathcal{B}$ then returns c , π , the partial decryptions and the witness ω to Game₂. Note that due to the knowledge-soundness of Π , $\text{Verify}(\text{pk}, c, \pi) = 1$ but $(\chi, \omega) \notin R$ can only happen with negligible probability, hence $\Pr[\text{Game}_{2,\mathcal{B}}^{M\text{-TT-KEM}}(1^\lambda) = 1] \geq \varepsilon - \text{negl}(\lambda)$. $\square$

Game₃: In the next step, we add an additional abort condition. In particular, we abort if $g_1^\omega = Q$.

LEMMA I.3. If the DLog problem holds in $\mathbb{G}_1$ then both games are computationally indistinguishable

$$\text{Game}_2^{M\text{-TT-KEM}} \approx_c \text{Game}_3^{M\text{-TT-KEM}}.$$

PROOF OF LEMMA I.3. Let $\mathcal{A}$ be a PPT adversary with a non-negligible distinguishing advantage $\varepsilon(\lambda)$. Clearly, $\mathcal{A}$ can only distinguish the two games by triggering the additional abort condition, thus providing ω such that $g_1^\omega = Q$. We construct a PPT reduction $\mathcal{B}$ that uses $\mathcal{A}$ to break the DLog assumption with probability $\varepsilon(\lambda)$. Initially, $\mathcal{B}$ receives the group setup as well as a DLog challenge T . $\mathcal{B}$ sets $Q = T$ in the public key and runs $\mathcal{A}$ and returns ω . Whenever $\mathcal{A}$ triggers the additional abort, then $g_1^\omega = Q = T$, hence $\mathcal{B}$ breaks the DLog assumption. $\square$

To close the proof, we observe that no adversary exists that can win Game₃. In particular it holds that $c_0 = Y_j^\omega$ and $c_1 = Z_j^\omega$. Following the correctness of TT-KEM, the Comb operations for both S and S' yield $k = e(g_1, g_2)^{\alpha_j y_j z_j \omega}$. $\square$